22CIPC504 · 21CIPC504

Automata Theory & Compiler Design

Previous Year Question Papers — Complete Solutions, organised unit-wise

Nov–Dec 2025
UVCE · UVCE-UG-2022
V Sem B.Tech CSE/ISE
Dec-25

April 2025
UVCE · UVCE-UG-2022
V Sem B.Tech CSE/ISE
Apr-25

July–Aug 2024
EN-024 · NEP Scheme
V Sem B.Tech CSE & AIML
Aug-24

Every question from all three papers is answered once, grouped by unit and topic. Questions that repeat across papers are merged, and the paper tags show where each one appeared. **Every answer is written to full 10–15-mark depth**: definitions, step-by-step working, diagrams, transition and parsing tables, extra worked examples and traces, verification, and key points to remember. The badge on each question shows the marks it carried in the original paper.

Contents

UNIT I — Finite Automata, Regular Expressions & Grammars

1.1 Define FA. Obtain a DFA to accept strings of a's and b's starting with ab(a+b)* with transition table. · 6 M Aug-24 Q1a

1.2 Draw a DFA to accept the set of all strings on Σ = {0,1} that either begin or end (or both) with the substring 01. · 10 M Dec-25 Q1a

1.3 Draw a DFA to accept the set of all strings of a's and b's ending with ab or ba. · 8 M Apr-25 Q1a

1.4 Design an NFA to recognize the set of strings abc, abd and aacd. · 6 M Dec-25 Q1b

1.5 Obtain an NFA to accept L = {w | w ∈ ababn or aban, n ≥ 0}. · 5 M Apr-25 Q2b

1.6 A toll booth charges a fee based on the sum of coins inserted and allows a car to pass only if the total is … · 10 M Dec-25 Q2a

1.7 Define CFG. Construct a DFA which accepts only those strings representing zero modulo five. · 8 M Aug-24 Q2a

1.8 Obtain a regular expression for L(R) = {w | w ∈ {0,1}*} with at least three consecutive 0's. · 4 M Dec-25 Q1c

1.9 Obtain a regular expression for strings of a's and b's whose third symbol from the right is 'a' and fourth symbol … · 4 M Apr-25 Q1c

1.10 Convert the given ε-NFA to an equivalent NFA (q0 with self-loop 0, q0 —ε→ q1, q1 with self-loop 1, q1 final). · 10 M Dec-25 Q2b Apr-25 Q2c

1.11 Convert the given NFA with ε-moves to DFA (q0: b→q0, ε→q1; q1: a→q1, ε→q2; q2: a→q1, b→q0; q2 final). · 7 M Aug-24 Q1c

1.12 Convert the given NFA to its equivalent DFA (q0 loop 0; q0 —0,1→ q1; q1 —0,1→ q2; q2 loop 1; q2 accepting). · 8 M Apr-25 Q1b

1.13 Convert the following NFA to DFA (P: 0→{P,Q}, 1→{P}; Q: 0,1→{R}; R: 0→{S}, 1→∅; S final: 0,1→{S}). · 7 M Aug-24 Q2b

1.14 Define grammar and explain the different types of grammar. · 7 M Aug-24 Q1b

1.15 Construct the leftmost and rightmost derivation for the string aaabbabbba using S → aB | bA, A → a | aS | bAA, B → b … · 5 M Apr-25 Q2a

1.16 Consider the grammar S → aS | aSbS | ε. Show that the derivation for the string aab is ambiguous. · 5 M Aug-24 Q2c

UNIT II — Introduction to Compilers & Lexical Analysis

2.1 Draw the structure of a compiler and describe the various phases of compilation. Show the output of each phase for … · 10 M Dec-25 Q3a Apr-25 Q3b Aug-24 Q4a

2.2 Analyse the given C program: identify the errors, the compiler phase that detects each one, why each occurs, and … · 6 M Dec-25 Q3b

2.3 Describe the grouping of compiler phases and their significance. · 5 M Apr-25 Q3c

2.4 Describe the role of a compiler. · 6 M Aug-24 Q3c

2.5 Who are the cousins of a compiler? Explain their roles. / Explain the different types of language processors. · 5–6 M Dec-25 Q4b Apr-25 Q3a

2.6 What is the difference between a compiler and an interpreter? · 4 M Dec-25 Q3c

2.7 List and explain the different compiler-construction tools and their purposes. · 6 M Dec-25 Q4a

2.8 Explain input buffering in detail. Describe the role of the forward pointer and the lexeme-begin pointer. How do … · 7–8 M Dec-25 Q4c Apr-25 Q4b Aug-24 Q4c

2.9 Explain the LEX tool with the structure of a Lex program. · 7 M Apr-25 Q4a Aug-24 Q3a

2.10 Explain the process of token recognition in lexical analysis. / Write a regular expression for relational operators … · 5 M Apr-25 Q4c Aug-24 Q5b

2.11 Describe the lexical errors and the various error-recovery strategies, with suitable examples. · 8 M Aug-24 Q6b

UNIT III — Syntax Analysis — Top-Down Parsing

3.1 What is a parser? Explain the role of a parser and its types in detail. · 7–8 M Dec-25 Q5a Apr-25 Q6c Aug-24 Q4a

3.2 Explain left-recursive grammars and left factoring with examples. · 6–7 M Dec-25 Q6a Aug-24 Q4b

3.3 (i) Convert S → a | ^ | (T), T → T,S | S into a non-left-recursive form. (ii) Left-factor S → iEtS | iEtSeS | a, E → b. · 6 M Apr-25 Q6b

3.4 Perform left factoring for the grammar A → abB | aB | cdg | cdeB | cdfB. · 7 M Aug-24 Q3b

3.5 Compute FIRST and FOLLOW for the grammar S → Bb | Cd, B → aB | ε, C → cC | ε. · 6 M Dec-25 Q6c Apr-25 Q5a

3.6 Construct the predictive parser (LL(1) table) for E → E+T | T, T → T*F | F, F → (E) | id (equivalently E → TE', E' → … · 8–10 M Dec-25 Q5b Apr-25 Q5b Aug-24 Q5a

3.7 Write a note on non-recursive descent parsing. / Explain the table-driven predictive parsing model. · 5–6 M Dec-25 Q5c Apr-25 Q6a

3.8 What is backtracking in parsing? Why is it inefficient? · 4 M Apr-25 Q5c

3.9 Discuss error-recovery strategies in predictive parsing. · 7 M Aug-24 Q6c

UNIT IV — Bottom-Up Parsing, SDD & Yacc

4.1 Explain bottom-up parsing in detail. Define augmented grammar and explain handle pruning. · 6 M Dec-25 Q8a Aug-24 Q8b

4.2 Trace the handle-pruning process while parsing the input "abbcdc" with a shift-reduce parser for S → aABc, A → Abc | … · 6 M Apr-25 Q7c

4.3 Perform shift-reduce parsing for the input string (a,(a,a)) using S → (L) | a, L → L,S | S. · 5 M Aug-24 Q5c

4.4 Differentiate between shift-reduce and operator-precedence parsers. · 5 M Aug-24 Q6a

4.5 Construct the LR(1) / CLR(1) parsing table for S → AA, A → aA | b. (Also: S → CC, C → aC | d.) · 7–10 M Dec-25 Q6b, Q8b Apr-25 Q8b Aug-24 Q8a

4.6 Construct the LALR(1) parsing table for S → AA, A → aA, A → b. · 8 M Dec-25 Q7a Apr-25 Q7a

4.7 Construct the LR(0) parsing table for E → E*B | E+B | B, B → 0 | 1. · 8 M Apr-25 Q8c

4.8 Differences between SLR, CLR and LALR parsers. · 4 M Aug-24 Q8c

4.9 Explain Syntax-Directed Definitions (SDD) with an example. Differentiate between synthesized and inherited … · 4–6 M Dec-25 Q7b Apr-25 Q8a Aug-24 Q7b

4.10 What are the applications of SDD? · 6 M Dec-25 Q8c

4.11 Write a note on Yacc. / Explain the working of the Yacc parser generator in detail. · 6–8 M Dec-25 Q7c Apr-25 Q7b Aug-24 Q7a

4.12 Discuss type checking with suitable examples. · 8 M Aug-24 Q7c

UNIT V — Intermediate Code, Run-Time Storage & Code Optimization

5.1 Explain the different types of intermediate code with an example. / Briefly explain the variants of syntax trees. · 7–8 M Aug-24 Q10a Apr-25 Q9a

5.2 Construct the syntax tree for the expression x * y − 5 + z. · 8 M Dec-25 Q9b

5.3 Construct the quadruples, triples and indirect triples of the expression a * −(b + c). · 8 M Dec-25 Q10b

5.4 Write the three-address code, quadruples, triples and indirect triples for: (1) a + b + (c + d) + d + e + a … · 10 M Apr-25 Q10b

5.5 (i) Translate A[i] := b and a := B[i] into triple representation. (ii) Generate the three-address code, compute the … · 8 M **Aug-24 Q9c**

5.6 Generate three-address code for: while (a < c and b > d) do if a = 1 then c = c + 1 else while a <= d do a … · 8 M **Apr-25 Q9b**

5.7 Construct a basic-block flow graph for: if (x > 10) y = x + 5; else y = x − 5; z = y * 2; and identify the number … · 7 M **Dec-25 Q9a**

5.8 Define a flow graph. Differentiate between a basic block and a flow graph. Explain the structure-preserving … · 4–5 M **Dec-25 Q10c** **Aug-24 Q9a**

5.9 What is a Directed Acyclic Graph (DAG)? Draw the DAG for t0 = a + b; t1 = t0 * c; d = t0 + t1. · 7 M **Aug-24 Q9b**

5.10 Construct the DAG for the block: a := b*c; d := b; e := d*c; b := e; f := b + c; g := f + d · 10 M **Apr-25 Q10a**

5.11 Construct the DAG for the basic block: t1 := 4*i; t2 := a[t1]; t3 := 4*i; t4 := b[t3]; t5 := t2*t4; t6 := prod + t5; … · 6 M **Aug-24 Q10b**

5.12 Explain the principal sources of optimization. · 5 M **Dec-25 Q9c**

5.13 Discuss copy propagation, dead-code elimination and strength reduction in detail. · 4–7 M **Apr-25 Q9c** **Aug-24 Q10c**

5.14 Explain storage organization in detail. · 8 M **Dec-25 Q10a**

UNIT I

Finite Automata, Regular Expressions & Grammars

DFA/NFA design · ε-NFA conversion · subset construction · regular expressions · derivations · ambiguity

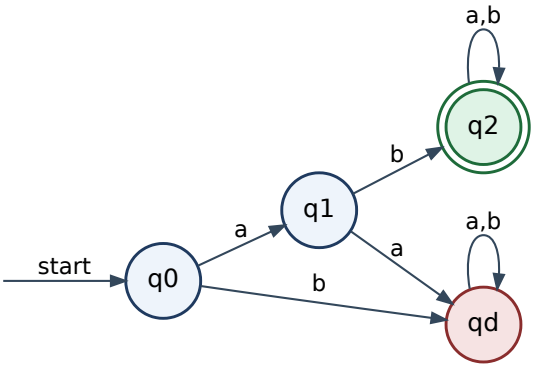
1.1 Define FA. Obtain a DFA to accept strings of a's and b's starting with ab(a+b)* with transition table. 6 M

Aug-24 Q1a

Finite Automaton (FA): An FA is a mathematical model of a machine with a finite number of states. It reads an input string one symbol at a time and accepts or rejects it. A DFA is a 5-tuple **M = (Q, Σ, δ, q0, F)**, where

- **Q** — finite set of states; **Σ** — input alphabet;
- **δ : Q × Σ → Q** — transition function; **q0 ∈ Q** — start state; **F ⊆ Q** — set of final states.

Language: L = ab(a+b)* = strings that begin with *ab*. After reading *a* then *b*, any further input is accepted. Any other prefix goes to a dead state.



δ	a	b
→q0	q1	qd
q1	qd	q2
*q2	q2	q2
qd	qd	qd

Check: "abba": q0→q1→q2→q2→q2 ✔ accepted. "ba": q0→qd→qd ✘ rejected.

Extended δ (processing a whole string)

δ is extended to strings as δ̂ : Q × Σ* → Q, with δ̂(q, ε) = q and δ̂(q, wa) = δ(δ̂(q, w), a). A string w is accepted if δ̂(q0, w) ∈ F, so **L(M) = { w | δ̂(q0, w) ∈ F }**.

Trace for w = "abab": δ̂(q0, ε) = q0 → δ(q0, a) = q1 → δ(q1, b) = q2 → δ(q2, a) = q2 → δ(q2, b) = q2. q2 ∈ F, so it is accepted.

Types of finite automata

DFA	NFA	ε-NFA
$\delta : Q \times \Sigma \to Q$, exactly one move per symbol	$\delta : Q \times \Sigma \to 2^Q$, zero or more moves per symbol	$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$; can move without input
Easy to implement; fast	Easier to design; usually fewer states	Easiest to build from a regular expression (Thompson's construction)

All three accept exactly the **regular languages**. Every NFA or ε-NFA can be converted into a DFA (subset construction).

Why the dead state qd is needed

A DFA must define a move for *every* (state, symbol) pair. A string that starts with 'b' or "aa" can never begin with "ab". So it goes to a **trap (dead) state** qd, which is non-final and loops on all symbols. Without qd the automaton would be incomplete, i.e. a partial DFA.

Minimality

The four states are pairwise distinguishable. q2 is the only final state. q0 and q1 differ on "b" (q1 →b→ q2 accepts, q0 →b→ qd rejects). qd differs from q0 on "ab". So **4 states is the minimum**.

Applications of FA

- Lexical analysers in compilers (token recognition)
- Text search and pattern matching (grep, editors)
- Protocol, vending-machine and traffic-signal controllers
- Spell checkers, digital circuit design (sequential circuits)

1.2 Draw a DFA to accept the set of all strings on Σ = {0,1} that either begin or end (or both) with the substring 01. **10 M**

Dec-25 Q1a

Idea: L = 01(0+1)* ∪ (0+1)*01. We build two "tracks" from one start state:

- **Begins-with-01 track:** q0 →0→ q1 →1→ **qF**. qF loops on 0,1 and stays final, because the string already begins with 01.
- **Ends-with-01 track:** once the string can no longer begin with 01, we only remember the last symbols: r0 = "no useful suffix", r1 = "last symbol is 0", **r2** = "last two symbols are 01" (final).

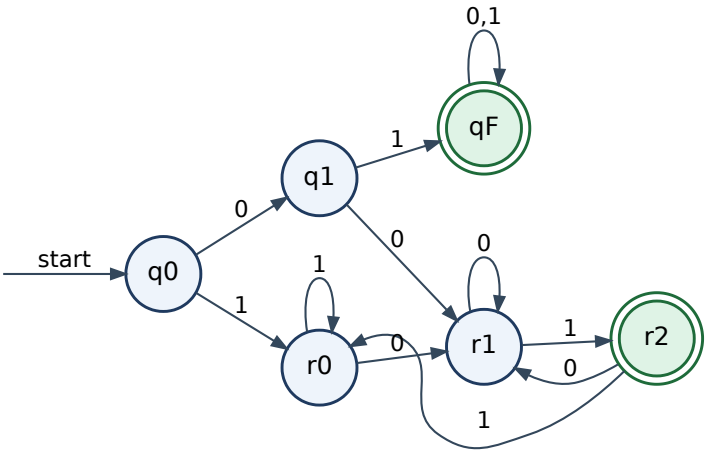


Fig. DFA for strings beginning or ending with 01 (final states: qF, r2)

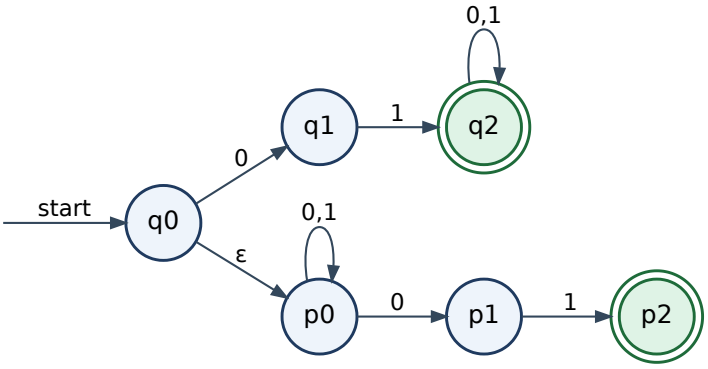
State	0	1	Meaning
→q0	q1	r0	start
q1	r1	qF	read "0"
*qF	qF	qF	began with 01
r0	r1	r0	no suffix
r1	r1	r2	ends in 0
*r2	r1	r0	ends in 01

Verification

- "0110" → q1, qF, qF, qF ✔ (begins with 01)
- "1101" → r0, r0, r1, r2 ✔ (ends with 01)
- "0101" → q1, qF, qF, qF ✔ (both)
- "0010" → q1, r1, r2, r1 ✘
- "10" → r0, r1 ✘

Alternative approach — NFA first, then DFA

The union is easy to express as an NFA. One branch checks the "begins with 01" prefix. The other guesses the point where the final "01" starts.



ε-NFA for 01(0+1)* + (0+1)*01. Applying subset construction to it gives the 6-state DFA above.

Formal definition of the DFA

M = (Q, Σ, δ, q0, F) with Q = {q0, q1, qF, r0, r1, r2}, Σ = {0, 1}, start state q0, F = {qF, r2}, and δ as in the table.

Proof that it is correct (state invariants)

State	Strings that reach it
q0	ε only
q1	"0" only
qF	every string beginning with 01
r0	does not begin with 01, length ≥ 1, ends in 1 but not in 01
r1	does not begin with 01, length ≥ 2 or starts with 1, ends in 0
r2	does not begin with 01 and ends in 01

The final states are exactly the ones whose strings begin with 01 (qF) or end with 01 (r2). So L(M) = L.

Minimality

No two states are equivalent. qF accepts every continuation, but r2 rejects "0". q1 and r1 differ on "10" (qF accepts, r1 rejects). q0 and r0 differ on "010". So **6 is the minimum number of states**.

Regular expression

R = 01(0 + 1)* + (0 + 1)*01

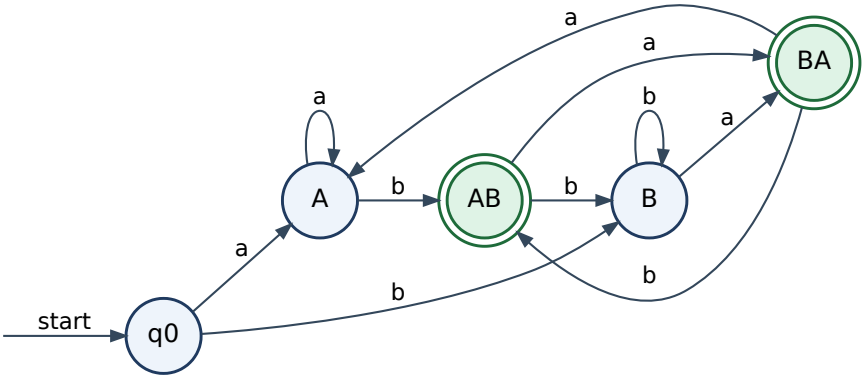
More test strings

String	Path	Result
01	q0→q1→qF	✔
101	q0→r0→r1→r2	✔
1010	r0→r1→r2→r1	✘
0001	q1→r1→r1→r2	✔
ε	q0	✘

1.3 Draw a DFA to accept the set of all strings of a's and b's ending with ab or ba. — 8 M

Apr-25 Q1a

The DFA only needs to remember the **last two symbols**. States: **q0** (empty input), **A** (ends in a, not ba), **B** (ends in b, not ab), **AB** (ends with ab, final), **BA** (ends with ba, final).



δ	a	b
→q0	A	B
A	A	AB
B	BA	B
*AB	BA	B
*BA	A	AB

Traces

- "aab": A, A, AB ✔
- "abba": A, AB, B, BA ✔
- "aa": A, A ✘ "bbb": B,B,B ✘

Equivalent RE: (a+b)*(ab+ba)

Construction strategy

The language depends only on the **last two symbols**, so the DFA tracks "the relevant suffix of the input read so far". Each state stands for one equivalence class:

State	Class of strings	Final?
q0	ε	No
A	"a" or strings ending in "aa"	No
B	"b" or strings ending in "bb"	No
AB	strings ending in "ab"	Yes
BA	strings ending in "ba"	Yes

How each transition is found: from AB (ends ab), reading *a* gives "...aba", which ends in "ba", so go to BA. Reading *b* gives "...abb", which ends in "bb", so go to B. The other transitions follow the same way.

Formal definition

M = ({q0, A, B, AB, BA}, {a, b}, δ, q0, {AB, BA}).

Minimality check

Final vs non-final states are distinct. AB vs BA: on "a", AB → BA (accept) while BA → A (reject), so they differ. A vs B: on "b", A → AB (accept) while B → B (reject). q0 vs A: on "b", q0 → B (reject) while A → AB (accept). q0 vs B: on "a", q0 → A (reject) while B → BA (accept). So the **5-state DFA is minimal**.

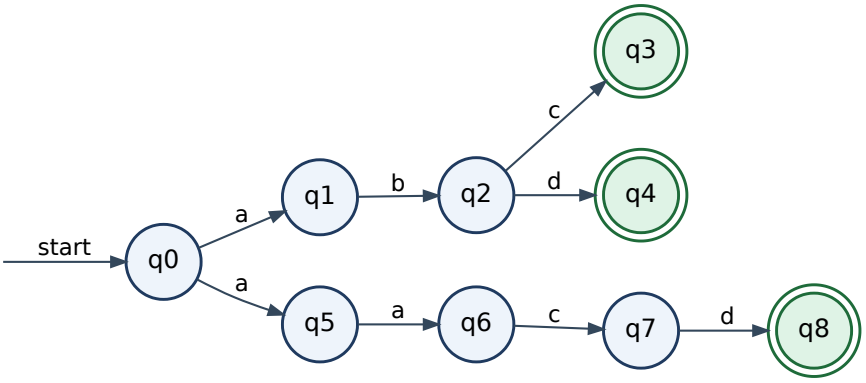
Alternative: NFA for (a+b)*(ab + ba)

An NFA has a start state looping on a, b, with two branches: start →a→ p →b→ (final) and start →b→ r →a→ (final). Applying subset construction gives the DFA above.

1.4 Design an NFA to recognize the set of strings abc, abd and aacd. 6 M

Dec-25 Q1b

L = {abc, abd, aacd} is finite. The NFA starts one path per word, sharing the common prefix *ab*. The state q0 has **two moves on 'a'** (to q1 and q5), so the machine is non-deterministic.



δ	a	b	c	d
→q0	{q1, q5}	∅	∅	∅
q1	∅	{q2}	∅	∅
q2	∅	∅	{q3}	{q4}
q5	{q6}	∅	∅	∅
q6	∅	∅	{q7}	∅
q7	∅	∅	∅	{q8}
*q3, *q4, *q8	∅	∅	∅	∅

M = ({q0…q8}, {a,b,c,d}, δ, q0, {q3, q4, q8}). For "aacd" the path q0→q5→q6→q7→q8 accepts. The branch through q1 dies on the second 'a'.

Formal 5-tuple

N = (Q, Σ, δ, q0, F) where Q = {q0, …, q8}, Σ = {a, b, c, d}, start state q0, F = {q3, q4, q8}, and δ : Q × Σ → 2^Q as tabulated.

Acceptance by an NFA

An NFA accepts w if **at least one** path labelled w leads from q0 to a final state: $\hat{\delta}(q0, w) \cap F \neq \emptyset$.

Input	$\hat{\delta}$ computation (sets of states)	Result
abc	{q0} →a→ {q1, q5} →b→ {q2} →c→ {q3}	✔ (q3 ∈ F)
abd	{q0} → {q1, q5} → {q2} →d→ {q4}	✔
aacd	{q0} → {q1, q5} →a→ {q6} →c→ {q7} →d→ {q8}	✔
aac	… → {q7}	✘ (q7 not final)
abcd	… → {q3} →d→ ∅	✘

Equivalent DFA (subset construction)

DFA state	a	b	c	d
→{q0}	{q1,q5}	∅	∅	∅
{q1,q5}	{q6}	{q2}	∅	∅
{q2}	∅	∅	{q3}	{q4}
{q6}	∅	∅	{q7}	∅
{q7}	∅	∅	∅	{q8}
*{q3}, *{q4}, *{q8}	∅	∅	∅	∅

The three final states have identical behaviour (they go to ∅ on every symbol), so they can be merged into one in the minimal DFA.

Regular expression

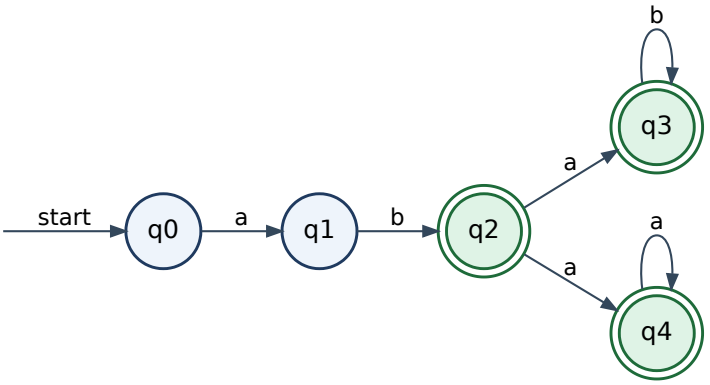
R = abc + abd + aacd = a(b(c + d) + acd)

Why an NFA here? The NFA mirrors the word list directly: one branch per keyword, all leaving the start state. This is exactly how a lexical analyser handles keywords (a "trie" of keywords).

1.5 Obtain an NFA to accept L = {w | w ∈ $abab^n$ or aba^n, n ≥ 0}. 5 M

Apr-25 Q2b

aba^n = {ab, aba, abaa, …} and $abab^n$ = {aba, abab, ababb, …}. Both share the prefix **ab**. After it, the NFA guesses which branch to follow.



- **q2** final → accepts "ab" (aba^0).
- q2 →a→ **q3** (final, loop on b) → aba, abab, ababb … ($abab^n$)
- q2 →a→ **q4** (final, loop on a) → aba, abaa, … (aba^n, n ≥ 1)

Two separate branches are needed. One shared state looping on both a and b would wrongly accept strings like "abaab".

Formal definition

N = ({q0, q1, q2, q3, q4}, {a, b}, δ, q0, {q2, q3, q4}) with δ(q0,a) = {q1}, δ(q1,b) = {q2}, δ(q2,a) = {q3, q4}, δ(q3,b) = {q3}, δ(q4,a) = {q4}. All other moves go to ∅.

δ	a	b
→q0	{q1}	∅
q1	∅	{q2}
*q2	{q3, q4}	∅
*q3	∅	{q3}
*q4	{q4}	∅

Traces

String	Set of states	Result
ab	{q0}→{q1}→{q2}	✔ (aba^0)
ababb	{q2} →a→ {q3,q4} →b→ {q3} →b→ {q3}	✔
abaaa	{q2} →a→ {q3,q4} →a→ {q4} →a→ {q4}	✔
abaab	{q3,q4} →a→ {q4} →b→ ∅	✘ (correctly rejected)

Regular expression and DFA

R = ab(a* + ab*) = ab + aba(a* + b*) — equivalently abab* + aba*

Subset construction gives the DFA states {q0}, {q1}, {q2}*, {q3,q4}*, {q3}*, {q4}* and ∅. The state {q3,q4} remembers that we do not yet know which branch we are in. The first symbol after "aba" decides the branch.

1.6 A toll booth charges a fee based on the sum of coins inserted and allows a car to pass only if the total is divisible by 3. Design a DFA to decide whether the sum is divisible by 3.

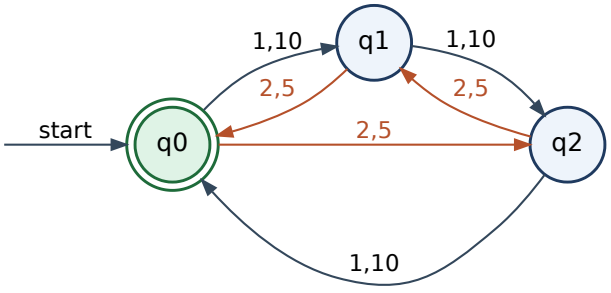
10 M

Dec-25 Q2a

Assumption: the machine accepts Indian coins **Σ = {1, 2, 5, 10}** (rupees). We never need the actual total, only **total mod 3**. So the DFA needs 3 states:

- **q0** — sum ≡ 0 (mod 3) → start and **final** (gate opens)
- **q1** — sum ≡ 1 (mod 3) **q2** — sum ≡ 2 (mod 3)

Transition rule: $\delta(q_r, c) = q_{(r + c) \bmod 3}$. Since 1 ≡ 10 ≡ 1 and 2 ≡ 5 ≡ 2 (mod 3), coins {1,10} add 1 and coins {2,5} add 2.



δ	1	2	5	10
→*q0	q1	q2	q2	q1
q1	q2	q0	q0	q2
q2	q0	q1	q1	q0

M = ({q0,q1,q2}, {1,2,5,10}, δ, q0, {q0})

- Coins 5, 10: q0 →5→ q2 →10→ q0 ✔ total 15, gate opens
- Coins 2, 2: q0 → q2 → q1 ✘ total 4
- Coins 1, 2, 5, 1: q1, q0, q2, q0 ✔ total 9

Note: The same 3-state design works for any set of coin values. A coin of value c simply moves the machine from residue r to (r + c) mod 3.

Formal definition

M = (Q, Σ, δ, q0, F), where Q = {q0, q1, q2}, Σ = {1, 2, 5, 10}, start state q0, F = {q0}, and $\delta(q_r, c) = q_{(r + c) \bmod 3}$.

Why only 3 states are needed (the key idea)

Let S be the running total. When a coin c is inserted, the new remainder is **(S + c) mod 3 = ((S mod 3) + (c mod 3)) mod 3**. So the next remainder depends only on the current remainder and the coin, not on the full total. The machine therefore only has to remember one of 3 remainders. This keeps the DFA finite even though the total can grow without limit.

Detailed trace — coins 2, 5, 10, 1, 2

Step	Coin	Total	Total mod 3	State	Gate
0	—	0	0	q0	(start)
1	2	2	2	q2	closed
2	5	7	1	q1	closed
3	10	17	2	q2	closed
4	1	18	0	q0	**open ✔**
5	2	20	2	q2	closed

Correctness (by induction)

Claim: after reading coins $c_1 \ldots c_k$, the DFA is in state q_r where $r = (c_1 + \ldots + c_k) \bmod 3$. **Base:** $k = 0$ gives total 0 and state q0. **Step:** if the claim holds after k coins (state q_r), then $\delta(q_r, c_{k+1}) = q_{(r + c_{k+1}) \bmod 3}$, which matches the new total. The DFA accepts exactly when $r = 0$, i.e. when the sum is divisible by 3.

Variants

- Using only coins {1, 2}: the same 3 states, with 1 moving forward one state and 2 moving forward two.
- To reject an empty sequence (no coins), add a separate non-final start state that moves to q1 or q2 on the first coin, or to a final copy of q0 when the first coin is a multiple of 3.
- Divisible by n in general: n states $q0 \ldots q_{n-1}$, $\delta(q_r, c) = q_{(r+c) \bmod n}$.

Real-world meaning: the toll booth is a *Moore-style controller*. The gate output depends only on the state: OPEN in q0, CLOSED in q1 and q2.

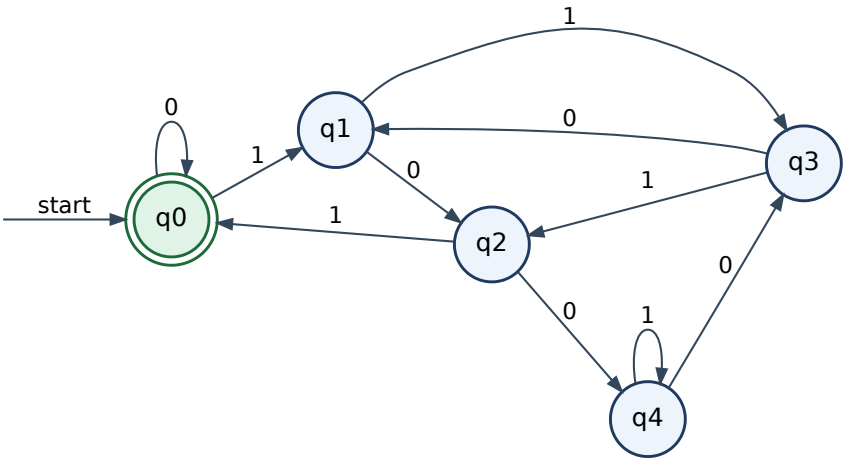
1.7 Define CFG. Construct a DFA which accepts only those strings representing zero modulo five.

8 M

Aug-24 Q2a

CFG: A context-free grammar is G = (V, T, P, S), where V = variables, T = terminals, P = productions of the form **A → α** ($A \in V$, $\alpha \in (V \cup T)^*$), and S = start symbol. The left side always has a single variable, so a rule can be applied regardless of context. Example: S → aSb | ε generates $\{a^n b^n\}$.

DFA for binary numbers ≡ 0 (mod 5): If the value read so far is v with remainder r, reading bit d gives value 2v + d. So the new remainder is **(2r + d) mod 5**. The states q0…q4 stand for the remainders, and q0 is both the start state and the final state.



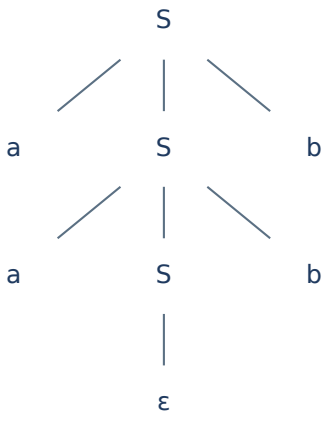
δ	0	1
→*q0	q0	q1
q1	q2	q3
q2	q4	q0
q3	q1	q2
q4	q3	q4

Check

- 1010 (=10): q1→q2→q0→q0 ✔
- 1111 (=15): q1→q3→q2→q0 ✔
- 111 (=7): q1→q3→q2 ✘ (rem 2)

CFG — more detail

Derivation: αAβ ⇒ αγβ when A → γ ∈ P. The language is L(G) = { w ∈ T* | S ⇒* w }. **Example:** G = ({S}, {a, b}, {S → aSb | ε}, S): S ⇒ aSb ⇒ aaSbb ⇒ aabb.



Parse tree for "aabb". CFGs can describe nested structures (balanced brackets, $a^n b^n$, nested if-else) that no finite automaton can recognise. That is why programming-language syntax is specified with CFGs.

Why the DFA works — the remainder recurrence

For a binary string w with value v(w), appending bit d gives v(wd) = 2·v(w) + d. Taking both sides mod 5: r' = (2r + d) mod 5. This recurrence fills the table directly:

r	2r + 0	mod 5	2r + 1	mod 5
0	0	0	1	1
1	2	2	3	3
2	4	4	5	0
3	6	1	7	2
4	8	3	9	4

Detailed trace — 11001 (= 25)

Bit read	Prefix	Value	Value mod 5	State
1	1	1	1	q1
1	11	3	3	q3
0	110	6	1	q1
0	1100	12	2	q2
1	11001	25	0	**q0 ✔**

Formal definition: M = ({q0,...,q4}, {0,1}, δ, q0, {q0}). All 5 states are needed, because the 5 remainders are pairwise distinguishable. **Note:** ε is accepted (value 0). If ε must be rejected, add a separate non-final start state.

Generalisation: a number in base b is divisible by n exactly when the DFA with states q0... q_{n-1} and $\delta(q_r, d) = q_{(b\cdot r + d) \bmod n}$ ends in q0. For decimal numbers mod 3: $\delta(q_r, d) = q_{(10r + d) \bmod 3}$.

1.8 Obtain a regular expression for L(R) = {w | w ∈ {0,1}*} with at least three consecutive 0's. 4 M

Dec-25 Q1c

The string must contain the substring **000** somewhere. Anything may come before it and anything after it.

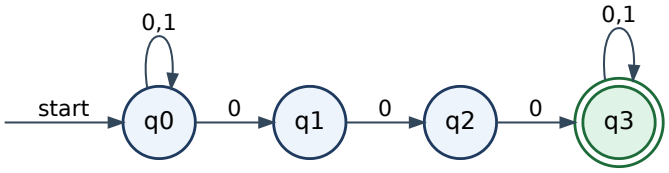
R = (0 + 1)* 000 (0 + 1)*

Examples in L: 000, 10001, 110000. Not in L: 0010, 1001. Any extra 0's (e.g. 0000) are absorbed by (0+1)*.

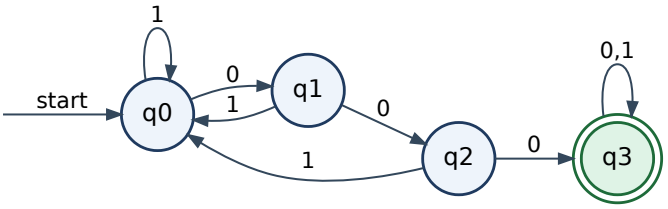
Building the RE step by step

- The required substring is **000**.
- Before it: any string, i.e. (0+1)*.
- After it: any string, i.e. (0+1)*.
- Concatenate the three parts: (0+1)* 000 (0+1)*.

NFA and DFA for this RE



NFA: guesses where 000 starts



DFA: counts consecutive 0's (q3 = found)

δ (DFA)	0	1	Meaning
→q0	q1	q0	no trailing 0
q1	q2	q0	one trailing 0
q2	q3	q0	two trailing 0's
*q3	q3	q3	000 seen

Related variants (often asked)

Language	RE
Exactly three consecutive 0's somewhere (not four)	Harder; usually not asked
At least three 0's (not necessarily consecutive)	(0+1)*0(0+1)*0(0+1)*0(0+1)* = 1*01*01*0(0+1)*
No three consecutive 0's	(1 + 01 + 001)*(ε + 0 + 00)
Ending with 000	(0+1)*000

Identities used: R + R = R, (R*)* = R*, ε·R = R. Note that (0+1)* is the same as Σ*.

1.9 Obtain a regular expression for strings of a's and b's whose third symbol from the right is 'a' and fourth symbol from the right is 'b'.

4 M

Apr-25 Q1c

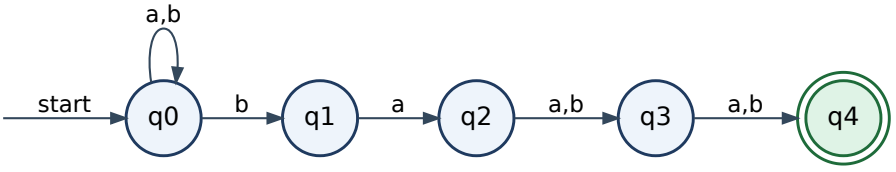
Write the string's last four positions as: … $\mathbf{x_4\ x_3\ x_2\ x_1}$. We need x_4 = b and x_3 = a, while x_2 and x_1 can be anything. The prefix before them is arbitrary.

R = (a + b)* b a (a + b)(a + b)

Examples: "baab", "aababb", "babb". "abab" ✘ (its 4th symbol from the right is a).

Derivation of the RE

The positions counted from the right are x_1 (last), x_2, x_3, x_4. The conditions are x_4 = b and x_3 = a. Reading left to right: (any prefix) · b · a · (any symbol) · (any symbol).



NFA: guesses the position of "ba" exactly two symbols before the end

Verification table

String	4th from right	3rd from right	In L?
baab	b	a	✔
abbaba	b	a	✔
abab	a	b	✘
bab	— (length 3)	b	✘

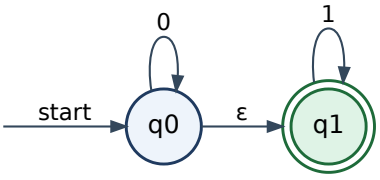
Minimum length is 4, so every string in L has length ≥ 4. **DFA note:** an equivalent DFA must remember the last 4 symbols, so it needs up to $2^4 = 16$ states. This is a classic example of an NFA being exponentially smaller than the equivalent DFA.

Related forms

- 3rd symbol from the right is a: (a+b)* a (a+b)(a+b)
- 3rd symbol from the left is a: (a+b)(a+b) a (a+b)*
- Starts with b and ends with a: b(a+b)*a

1.10 Convert the given ε-NFA to an equivalent NFA (q0 with self-loop 0, q0 —ε→ q1, q1 with self-loop 1, q1 final). 10 M

Dec-25 Q2b Apr-25 Q2c



Given ε-NFA

Step 1 — ε-closures

ε-closure(q) = the set of states reachable from q using only ε-moves (q itself included).

- ε-closure(q0) = {q0, q1}
- ε-closure(q1) = {q1}

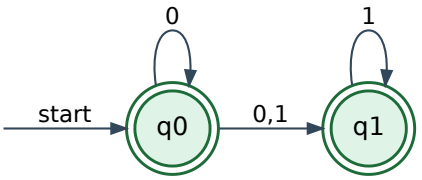
Step 2 — New transitions: δ'(q, a) = ε-closure(δ(ε-closure(q), a))

Computation	Result
δ'(q0,0) = ε-cl(δ({q0,q1},0)) = ε-cl({q0})	{q0, q1}
δ'(q0,1) = ε-cl(δ({q0,q1},1)) = ε-cl({q1})	{q1}
δ'(q1,0) = ε-cl(δ({q1},0)) = ε-cl(∅)	∅
δ'(q1,1) = ε-cl(δ({q1},1)) = ε-cl({q1})	{q1}

Step 3 — Final states

A state is final in the NFA if its ε-closure contains an original final state. ε-closure(q0) contains q1, so **q0 also becomes final**. F' = {q0, q1}.

δ'	0	1
→*q0	{q0,q1}	{q1}
*q1	∅	{q1}



Result: the NFA has no ε-moves and accepts the same language, 0*1*. q0 is final because the empty string is accepted.

Definitions used

- **ε-NFA:** E = (Q, Σ, δ, q0, F) with δ : Q × (Σ ∪ {ε}) → 2^Q.
- **ε-closure(q):** q, plus every state reachable from q by ε-moves alone. For a set S, ε-closure(S) = ⋃ ε-closure(q) over q ∈ S.
- **NFA without ε:** N = (Q, Σ, δ', q0, F'), where δ'(q, a) = ε-closure(δ(ε-closure(q), a)) and F' = { q | ε-closure(q) ∩ F ≠ ∅ }.

Transition table of the given ε-NFA

δ	0	1	ε
→q0	{q0}	∅	{q1}
*q1	∅	{q1}	∅

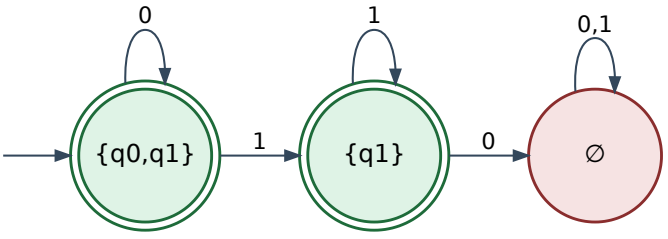
Verification on sample strings

String	ε-NFA run	NFA run	Accepted?
ε	q0 —ε→ q1 (final)	q0 is final	✔ both
0011	q0, q0, (ε) q1, q1, q1	{q0}→{q0,q1}→{q0,q1}→{q1}→{q1}	✔ both
10	q0 —ε→ q1 —1→ q1 —0→ ✘	{q0}→{q1}→∅	✘ both

Language: L = 0*1*, i.e. any number of 0's followed by any number of 1's.

Further: equivalent DFA

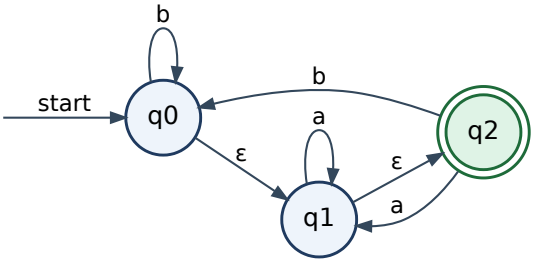
DFA state	0	1
→*{q0,q1}	{q0,q1}	{q1}
*{q1}	∅	{q1}
∅	∅	∅



Exam tip: always (1) write the ε-closures, (2) compute every δ' entry with the formula, (3) mark as final any state whose closure contains an old final state, and (4) draw the final NFA. Most of the marks are for these steps.

1.11 Convert the given NFA with ε-moves to DFA (q0: b→q0, ε→q1; q1: a→q1, ε→q2; q2: a→q1, b→q0; q2 final). 7 M

Aug-24 Q1c



Given ε-NFA

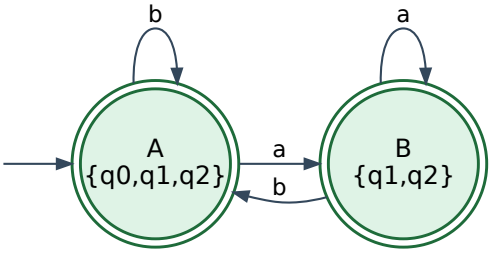
ε-closures: ε-cl(q0) = {q0, q1, q2}, ε-cl(q1) = {q1, q2}, ε-cl(q2) = {q2}

Subset construction (start state = ε-cl(q0) = **A = {q0,q1,q2}**):

DFA state	on a	on b
A = {q0,q1,q2}	δ = {q1} (from q1, q2) → ε-cl = {q1,q2} = **B**	δ = {q0} (from q0, q2) → ε-cl = {q0,q1,q2} = **A**
B = {q1,q2}	{q1} → ε-cl = {q1,q2} = **B**	{q0} (from q2) → ε-cl = **A**

Both A and B contain the final state q2, so **both are final**.

δ_D	a	b
→*A	B	A
*B	B	A



Observation: every DFA state is final and the DFA is complete, so it accepts (a+b)*. A and B are equivalent, so the minimal DFA has one state.

Algorithm (subset construction with ε-closure)

```
Dstates = { ε-closure(q0) } (unmarked)
while there is an unmarked T in Dstates:
    mark T
    for each input symbol a:
        U = ε-closure( move(T, a) )
        if U not in Dstates: add U as unmarked
        Dtran[T, a] = U
Final DFA states: the sets that contain an NFA final state
```

move() computations in detail

T	move(T, a)	move(T, b)
A = {q0,q1,q2}	q0: –, q1: q1, q2: q1 → {q1}	q0: q0, q1: –, q2: q0 → {q0}
B = {q1,q2}	q1: q1, q2: q1 → {q1}	q1: –, q2: q0 → {q0}

Verification

ε-NFA on "ba": the closure of q0 contains q2 (accepting); b → q0 → closure {q0,q1,q2}; a → q1 → closure {q1,q2} ∋ q2 → accepted. DFA: A →b→ A →a→ B (final) ✔. Every string can be extended so that q2 is reached through ε-moves, so the language really is (a+b)*.

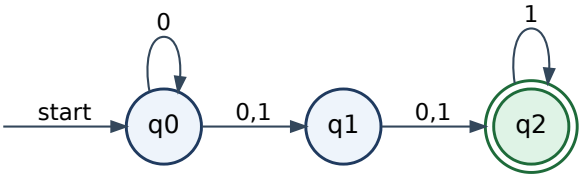
Minimization

A and B are both final, and on each symbol they go to the same states (a → B, b → A). So they are equivalent. **Minimal DFA:** one final state with a self-loop on a and b, which accepts Σ*.

Remember: for an ε-NFA, the DFA start state is **ε-closure(q0)**, not {q0}. Forgetting this is the most common mistake.

1.12 Convert the given NFA to its equivalent DFA (q0 loop 0; q0 —0,1→ q1; q1 —0,1→ q2; q2 loop 1; q2 accepting). 8 M

Apr-25 Q1b



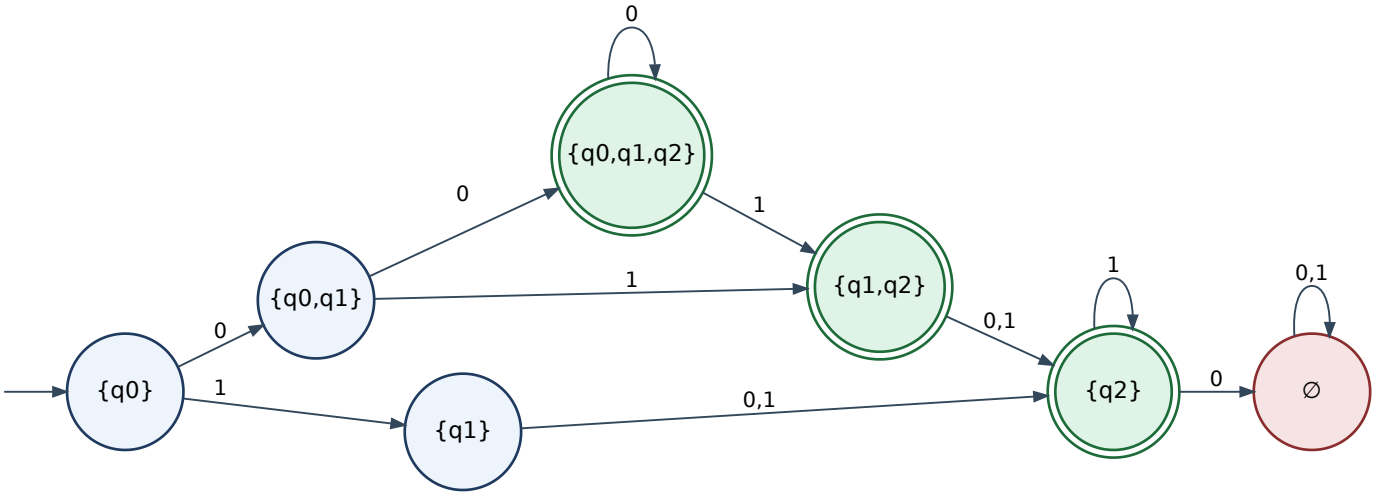
NFA transition table

δ	0	1
→q0	{q0,q1}	{q1}
q1	{q2}	{q2}
*q2	∅	{q2}

Subset construction

DFA state	0	1
→[q0]	[q0,q1]	[q1]
[q0,q1]	[q0,q1,q2]	[q1,q2]
[q1]	[q2]	[q2]
*[q0,q1,q2]	[q0,q1,q2]	[q1,q2]
*[q1,q2]	[q2]	[q2]
*[q2]	∅	[q2]
∅ (dead)	∅	∅

Final DFA states = every subset that contains q2: [q0,q1,q2], [q1,q2], [q2].



Equivalent DFA (7 states including the dead state ∅)

Subset-construction rule

For a DFA state S (a set of NFA states): $\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$. Start with {q0} and add new subsets as they appear (lazy evaluation), so only reachable states are built. In the worst case there are 2^n states. Here only 7 of the $2^3 = 8$ subsets are reachable: {q0, q2} never appears.

Worked computation of a few entries

- $\delta_D([q0,q1], 0) = \delta(q0,0) \cup \delta(q1,0) = \{q0,q1\} \cup \{q2\} = [q0,q1,q2]$
- $\delta_D([q0,q1,q2], 1) = \{q1\} \cup \{q2\} \cup \{q2\} = [q1,q2]$
- $\delta_D([q2], 0) = \emptyset \rightarrow$ dead state

Language and verification

L = 0*(0+1)(0+1)1*. That is, any 0's, then two symbols (each 0 or 1), then any 1's. Length ≥ 2.

String	DFA path	Result
01	[q0]→[q0q1]→[q1q2]	✔
11	[q0]→[q1]→[q2]	✔
0110	…→[q1q2]→[q2]→∅	✘
1	[q0]→[q1]	✘

Minimality

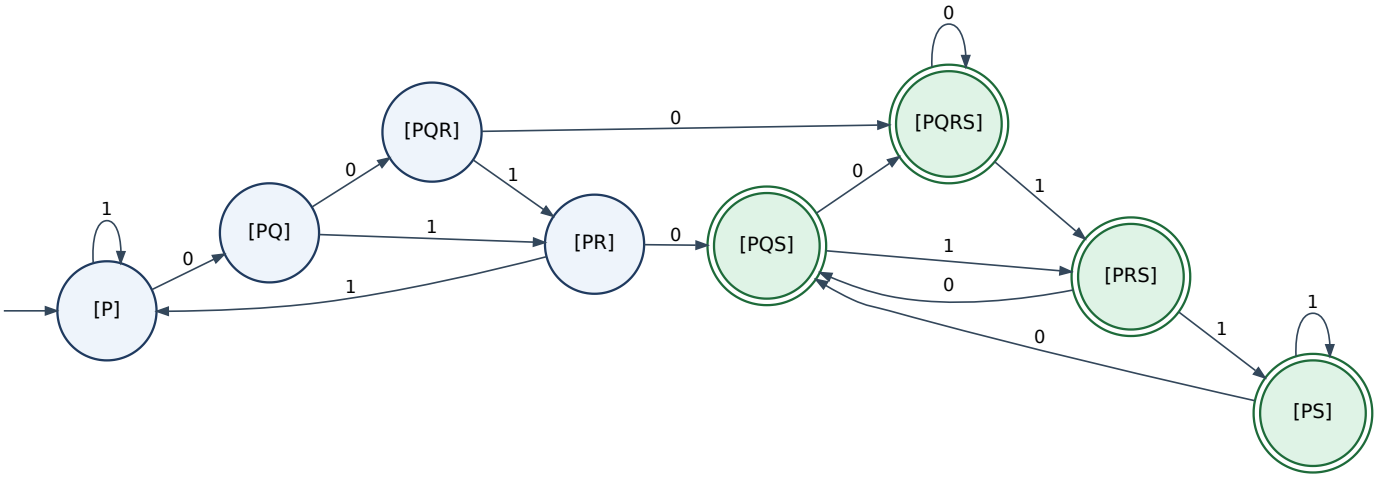
The final states [q0q1q2], [q1q2] and [q2] are distinguishable: on "0" they go to final, final and dead respectively, and [q0q1q2] vs [q1q2] differ on "00". The non-final states [q0], [q0q1], [q1] and ∅ are also distinguishable ([q0q1] vs [q1] differ on "00"). So the **7-state DFA is minimal**.

1.13 Convert the following NFA to DFA (P: 0→{P,Q}, 1→{P}; Q: 0,1→{R}; R: 0→{S}, 1→∅; S final: 0,1→{S}). — 7 M

Aug-24 Q2b

Start with [P] and compute only the reachable subsets, one at a time (lazy subset construction):

DFA state	0	1	Working for 0
→[P]	[PQ]	[P]	δ(P,0) = {P,Q}
[PQ]	[PQR]	[PR]	{P,Q} ∪ {R}
[PQR]	[PQRS]	[PR]	{P,Q} ∪ {R} ∪ {S}
[PR]	[PQS]	[P]	{P,Q} ∪ {S}
*[PQRS]	[PQRS]	[PRS]	{P,Q} ∪ {R} ∪ {S} ∪ {S}
*[PQS]	[PQRS]	[PRS]	{P,Q} ∪ {R} ∪ {S}
*[PRS]	[PQS]	[PS]	{P,Q} ∪ {S} ∪ {S}
*[PS]	[PQS]	[PS]	{P,Q} ∪ {S}



Resulting DFA — 8 states; final states are the ones containing S

Understanding the NFA

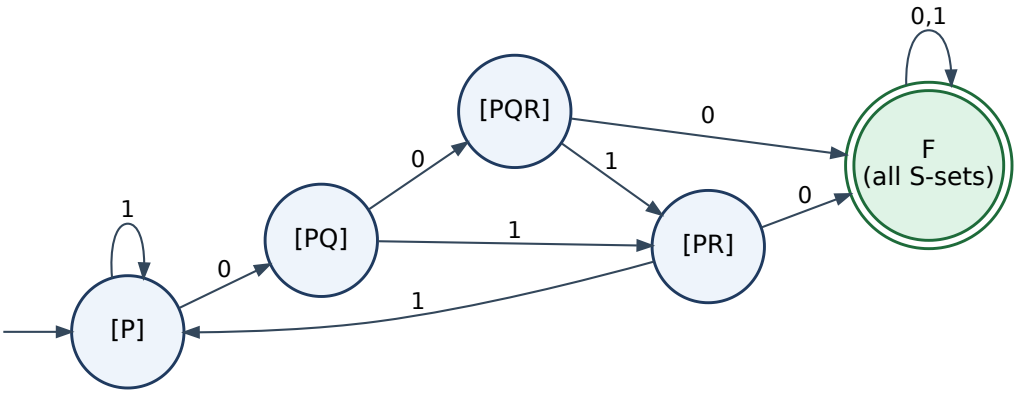
P loops on 0 and 1. P —0→ Q (guess) → R on any symbol → S on 0, and S is an accepting trap. So the NFA accepts exactly the strings that contain a **0, followed by any one symbol, followed by 0**:

L = (0+1)* 0 (0+1) 0 (0+1)*

Minimization of the 8-state DFA

Every final state contains S, and S loops on both symbols. So once S is reached the string is always accepted. All four final states go only to final states, so they are **equivalent** and merge into one state F. The non-final states are pairwise distinguishable:

Pair	Distinguishing string
[P] vs [PQ]	"00": [P]→[PQR] (non-final), [PQ]→F
[PQR] vs [PR]	"10": [PQR]→[PR]→F, [PR]→[P]→[PQ]
[P] vs [PR]	"0": [P]→[PQ], [PR]→F
[PQ] vs [PQR]	"0": [PQ]→[PQR], [PQR]→F



Minimal DFA (5 states)

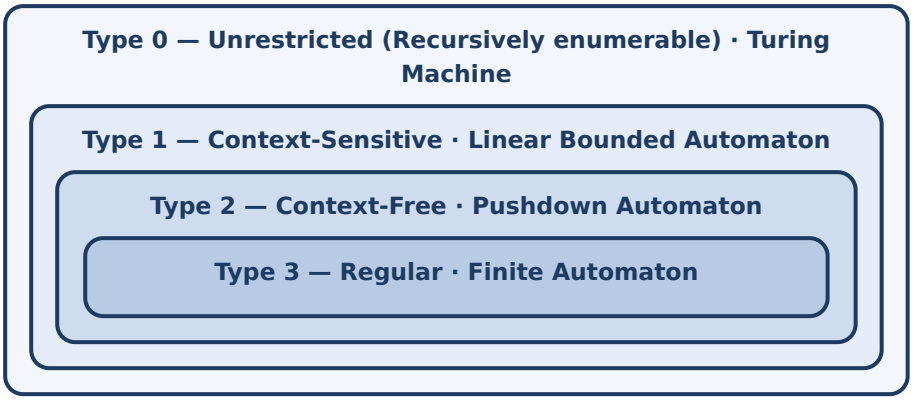
Tests: "1010" → [P] →1→ [P] →0→ [PQ] →1→ [PR] →0→ F ✔ (contains 0·1·0). "1001" → [P] → [PQ] → [PQR] → [PR], which is non-final ✘ (no pattern 0·x·0). "000" → [PQ] → [PQR] → F ✔.

1.14 Define grammar and explain the different types of grammar. 7 M

Aug-24 Q1b

Grammar: A grammar is a 4-tuple **G = (V, T, P, S)**. V is a finite set of variables (non-terminals), T is a finite set of terminals (V ∩ T = ∅), P is a finite set of productions α → β, and S ∈ V is the start symbol. The language of G is L(G) = { w ∈ T* | S ⇒* w }.

Chomsky Hierarchy — grammars are classified by the form of their productions:



Type	Production form	Machine	Example
Type 0 Unrestricted	α → β, α ∈ (V∪T)⁺ contains at least one variable; no restriction on β	Turing machine	AB → b
Type 1 Context-sensitive	αAβ → αγβ, |γ| ≥ 1 (non-contracting: |lhs| ≤ |rhs|)	Linear bounded automaton	aAb → aab; L = $a^n b^n c^n$
Type 2 Context-free	A → α, A ∈ V, α ∈ (V∪T)*	Pushdown automaton	S → aSb | ε
Type 3 Regular	A → aB | a (right-linear) or A → Ba | a (left-linear)	Finite automaton	S → aS | b

Each type is a proper subset of the one above it: Regular ⊂ CFL ⊂ CSL ⊂ REL. CFGs are used to specify programming-language syntax, and regular grammars or expressions are used to specify tokens.

Examples of each type

Type	Grammar	Language
3 Regular	S → 0S | 1S | 1	binary strings ending in 1
2 Context-free	S → (S)S | ε	balanced parentheses
1 Context-sensitive	S → aSBC | aBC, CB → BC, aB → ab, bB → bb, bC → bc, cC → cc	$a^n b^n c^n$, n ≥ 1
0 Unrestricted	any rules, e.g. Sa → aS, AB → ε	all recursively enumerable languages

Derivations and parse trees

A **derivation** is a sequence of rule applications from S. A **leftmost derivation** always replaces the leftmost variable; a **rightmost** one, the rightmost variable. A **parse tree** shows the derivation graphically: the root is S, interior nodes are variables, leaves are terminals or ε, and the yield is the derived string. A grammar is **ambiguous** if some string has two parse trees.

Closure and decision properties (summary)

Property	Regular	CFL	CSL	REL
Union, concatenation, Kleene star	✔	✔	✔	✔
Intersection	✔	✘	✔	✔
Complement	✔	✘	✔	✘
Membership decidable	✔	✔ (CYK)	✔	✘ (semi-decidable)

Role in compilers

Regular grammars or expressions → lexical analysis (Lex). Context-free grammars → syntax analysis (Yacc). Context-sensitive checks such as "declare before use" and type matching → semantic analysis, using attributes and the symbol table.

1.15 Construct the leftmost and rightmost derivation for the string aaabbabbba using S → aB | bA, A → a | aS | bAA, B → b | bS | aBB.

5 M

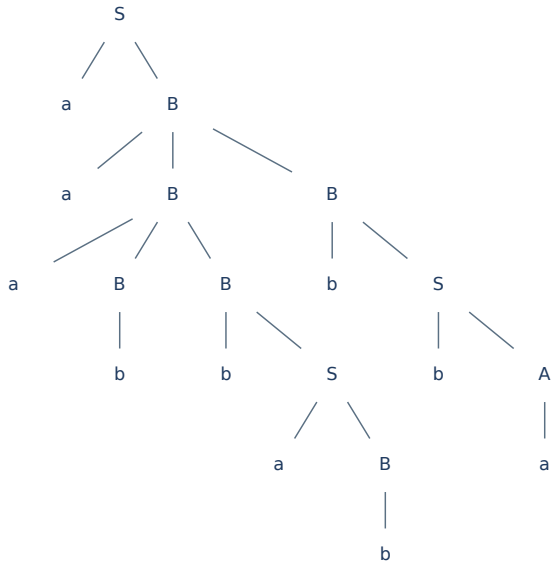
Apr-25 Q2a

Leftmost Derivation (LMD)

```
S ⇒ aB            (S→aB)
  ⇒ aaBB          (B→aBB)
  ⇒ aaaBBB        (B→aBB)
  ⇒ aaabBB        (B→b)
  ⇒ aaabbSB       (B→bS)
  ⇒ aaabbaBB      (S→aB)
  ⇒ aaabbabB      (B→b)
  ⇒ aaabbabbS     (B→bS)
  ⇒ aaabbabbbA    (S→bA)
  ⇒ aaabbabbba    (A→a)
```

Rightmost Derivation (RMD)

```
S ⇒ aB            (S→aB)
  ⇒ aaBB          (B→aBB)
  ⇒ aaBbS         (B→bS)
  ⇒ aaBbbA        (S→bA)
  ⇒ aaBbba        (A→a)
  ⇒ aaaBBbba      (B→aBB)
  ⇒ aaaBbSbba     (B→bS)
  ⇒ aaaBbaBbba    (S→aB)
  ⇒ aaaBbabbba    (B→b)
  ⇒ aaabbabbba    (B→b)
```



Parse tree (the same tree for both derivations; reading the leaves left to right gives a a a b b a b b b a)

About this grammar

It generates exactly the strings with **an equal number of a's and b's** (length ≥ 2). The intended meaning of the variables is: A derives strings with one more a than b, B derives strings with one more b than a, and S derives strings with equal numbers. Check: S → aB (one a plus one extra b gives equal counts), A → bAA (one b plus two extra a's gives one extra a), and so on.

Check for "aaabbabbba": 5 a's and 5 b's ✔.

Choosing the right production (strategy)

- Read the string left to right and keep track of what each pending variable still has to derive.
- After "aaa" three B's are pending, so we need three more b's than a's in the remainder "bbabbba" (b = 5, a = 2 ✔).
- Use B → b when B can finish, B → bS when a balanced substring follows, and B → aBB when another a comes first.

LMD vs RMD

Leftmost derivation	Rightmost derivation
Always expands the leftmost variable	Always expands the rightmost variable
Built by top-down (LL) parsers	Built in reverse by bottom-up (LR) parsers
Same number of steps (10 here)	Same parse tree, different order

Both derivations have **10 steps** because the parse tree has 10 interior-node expansions: one each for S → aB, B → aBB (twice), B → b (twice), B → bS (twice), S → aB, S → bA and A → a.

Verification: reading the leaves of the parse tree from left to right gives a a a b b a b b b a, which is the required string. Both derivations correspond to this single tree.

1.16 Consider the grammar S → aS | aSbS | ε. Show that the derivation for the string aab is ambiguous. 5 M

Aug-24 Q2c

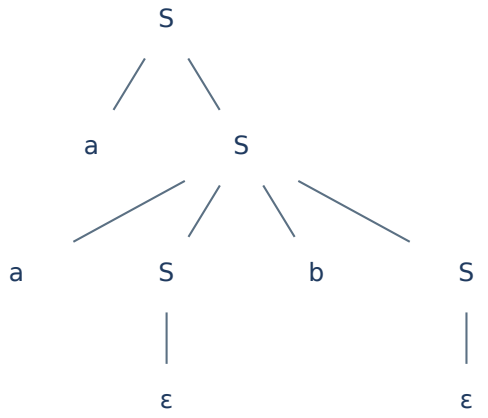
A grammar is **ambiguous** if some string has more than one leftmost derivation, or equivalently more than one parse tree. For w = aab:

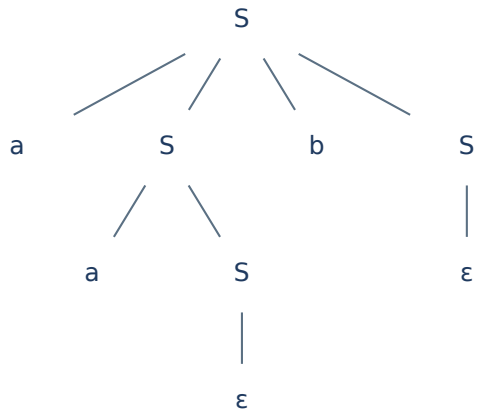
LMD 1

```
S ⇒ aS        (S→aS)
  ⇒ aaSbS     (S→aSbS)
  ⇒ aabS      (S→ε)
  ⇒ aab       (S→ε)
```

LMD 2

```
S ⇒ aSbS      (S→aSbS)
  ⇒ aaSbS     (S→aS)
  ⇒ aabS      (S→ε)
  ⇒ aab       (S→ε)
```





There are two different parse trees, and two different leftmost derivations, for the same string "aab". Hence the grammar **S → aS | aSbS | ε is ambiguous**. This is the "dangling-else" situation: 'b' can pair with either 'a'.

Second string showing ambiguity

Ambiguity appears whenever an unmatched a can pair with either of two b-positions. For example, "aabab" also has several parse trees.

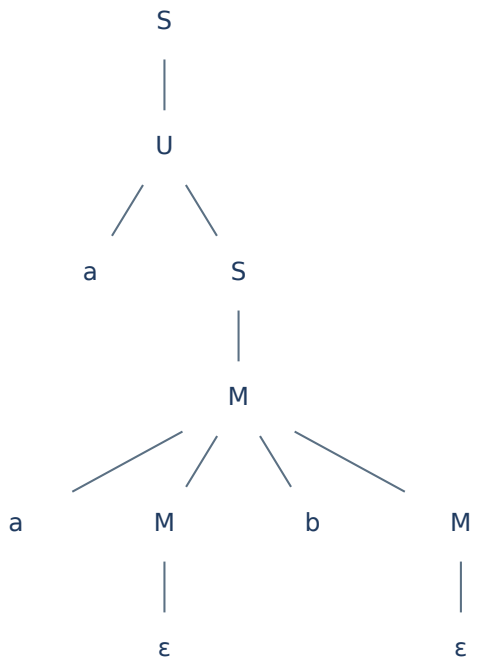
Consequences of ambiguity

- The same program could get two meanings (e.g. which *if* an *else* belongs to).
- Deterministic parsers (LL(1), LR(1)) cannot be built: the tables get conflicts.
- Ambiguity of a CFG is **undecidable** in general, so no algorithm can detect it for every grammar.

Removing the ambiguity (matched / unmatched method)

Read a as "if-then" and b as "else". Each b must pair with the **nearest unmatched a**:

```
S → M | U
M → a M b M | ε          (matched: every a has its b)
U → a S | a M b U        (unmatched: some a has no b)
```



With this grammar, "aab" has **only one** parse tree: S ⇒ U ⇒ aS ⇒ aM ⇒ aaMbM ⇒ aabM ⇒ aab. Here b pairs with the inner (second) a. This is the same rule C and Java use: an else matches the nearest unmatched if.

Summary: (1) state the definition of ambiguity, (2) give two leftmost derivations, (3) draw two parse trees, (4) conclude. For full marks, add the unambiguous equivalent grammar.

UNIT II

Introduction to Compilers & Lexical Analysis

Phases · language processors · compiler tools · input buffering · LEX · token recognition · lexical errors

2.1 Draw the structure of a compiler and describe the various phases of compilation. Show the output of each phase for the statement a = b + c * 60. 10 M

Dec-25 Q3a Apr-25 Q3b Aug-24 Q4a

A compiler translates a program written in a source language into an equivalent program in a target language. The work is done in two parts. **Analysis** (front end) breaks the source into pieces and builds an intermediate representation. **Synthesis** (back end) builds the target program from that representation. Each part is made of phases, and all phases share the **symbol table** and the **error handler**.

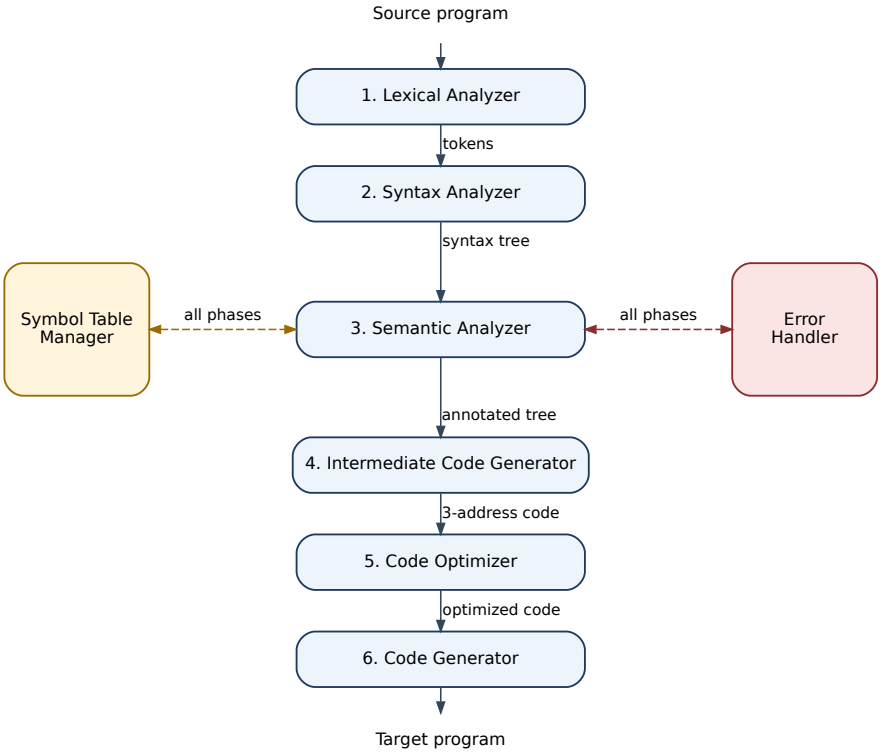


Fig. Phases of a compiler

1. **Lexical Analysis (Scanning):** reads the character stream and groups it into *lexemes*. For each lexeme it emits a token <token-name, attribute>. It also removes white space and comments and enters identifiers in the symbol table.
2. **Syntax Analysis (Parsing):** uses the tokens to build a *syntax tree* according to the grammar (CFG), and reports syntax errors.
3. **Semantic Analysis:** checks the tree for semantic consistency, mainly *type checking*. It inserts type conversions (coercions) where needed and records type information.
4. **Intermediate Code Generation:** produces a machine-independent, easy-to-translate IR, such as three-address code.

5. **Code Optimization:** improves the IR so the program runs faster or uses less memory (constant folding, removing redundant temporaries, etc.).
6. **Code Generation:** maps the IR to target instructions, allocating registers and memory.

Symbol table: stores each identifier with its attributes (type, scope, storage). **Error handler:** lets every phase report errors and recover.

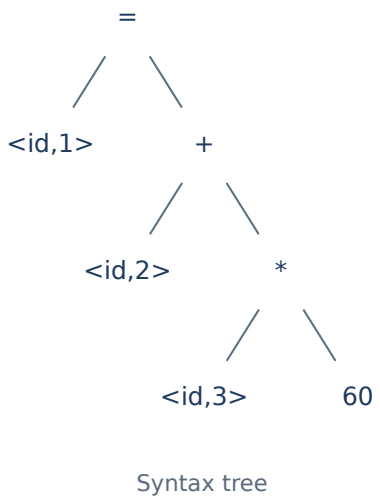
Translation of a = b + c * 60 (a, b, c are float)

Phase	Output
Lexical analyzer	`<id,1> <=> <id,2> <+> <id,3> <*> <number,60>` `Symbol table: 1 → a, 2 → b, 3 → c`
Syntax analyzer	Syntax tree (left figure below). * binds tighter than +.
Semantic analyzer	The integer 60 is converted to float with *inttofloat* (right figure).

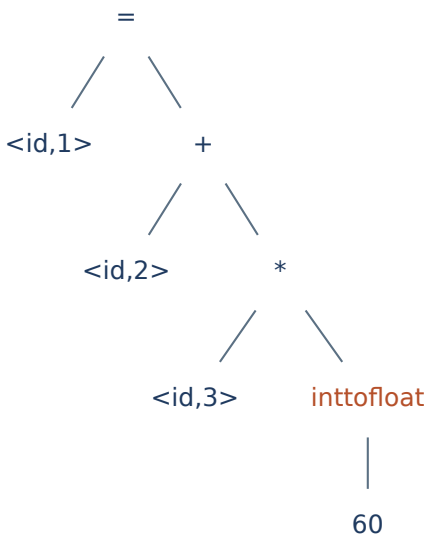
Intermediate code generator	`t1 = inttofloat(60)`
	`t2 = id3 * t1`
	`t3 = id2 + t2`
	`id1 = t3`

Code optimizer	`t1 = id3 * 60.0` `id1 = id2 + t1`

Code generator	`LDF R2, id3`
	`MULF R2, R2, #60.0`
	`LDF R1, id2`
	`ADDF R1, R1, R2`
	`STF id1, R1`



Syntax tree



After semantic analysis

For the Dec-25 variant a = b + c − 60, the steps are the same: tokens <id,1> <=> <id,2> <+> <id,3> <−> <num,60>; TAC t1 = inttofloat(60), t2 = id2 + id3, t3 = t2 − t1, id1 = t3; optimized code t1 = id2 + id3, id1 = t1 − 60.0.

Symbol table after lexical analysis

Index	Name	Token	Type	Other attributes
1	a	id	float	storage offset, scope
2	b	id	float	…
3	c	id	float	…

Lexemes and tokens

Lexeme	a	=	b	+	c	*	60
Token	<id,1>	<=>	<id,2>	<+>	<id,3>	<*>	<num,60>

The blanks between the lexemes are discarded by the lexical analyser.

Errors detected by each phase

Phase	Typical errors	Example
Lexical	illegal characters, malformed numbers or identifiers	`a = b # c;`, `2x = 5;`
Syntax	missing or extra tokens, unbalanced parentheses	`a = b + ;`
Semantic	type mismatch, undeclared variable, wrong argument count	`int x = "str";`
Code optimisation / generation	unreachable code (warning), constant overflow, resource limits	`return 0; x++;`

Analysis–synthesis model

- **Analysis part (front end):** lexical, syntax and semantic analysis plus intermediate code generation. It depends on the source language.
- **Synthesis part (back end):** code optimisation and code generation. It depends on the target machine.
- The **intermediate representation** separates the two parts, which makes retargeting possible.

Why each step of the example makes sense

- The parser puts * deeper in the tree than + because * has higher precedence. So c * 60 is evaluated first.
- The semantic analyser finds an int (60) inside a float operation and inserts the coercion *inttofloat*.
- The ICG produces one operation per instruction, with at most three addresses and new temporaries t1, t2, t3.
- The optimiser performs the int-to-float conversion at compile time (constant folding: 60 becomes 60.0) and removes the copy through t3.
- The code generator chooses registers (R1, R2) and float instructions (LDF, MULF, ADDF, STF). The # marks an immediate constant.

Exam tip: for 10–15 marks, draw the phase diagram, explain all six phases plus the symbol table and error handler, and show the output of every phase for the given statement, including both trees. That covers the whole question.

2.2 Analyse the given C program: identify the errors, the compiler phase that detects each one, why each occurs, and write the corrected code. 6 M

Dec-25 Q3b

```
#include <stdio.h>
int main () {
    int 123 num = 10;
    int x = 5
    float y = "hello";
    print ("value of x: %d", x);
    return 0;
}
```

```
/* Corrected version */
#include <stdio.h>
int main() {
    int num123 = 10;
    int x = 5;
    char *y = "hello";   /* or: float y =
5.0; */
    printf("value of x: %d", x);
    return 0;
}
```

#	Error	Phase that detects it	Why it occurs
1	`int 123 num` — invalid identifier	Lexical (if written as one word, `123num` is a malformed token) / Syntax (with the space, the parser sees `int` followed by the number 123)	An identifier must start with a letter or _. A declaration expects an identifier after the type, not a constant.
2	`int x = 5` — missing semicolon	Syntax analysis	The grammar requires a declaration to end with ';'. The parser sees `float` where it expects ';'.
3	`float y = "hello";` — type mismatch	Semantic analysis (type checking)	A string literal (char*) cannot be assigned to a float. The types are incompatible.
4	`print(...)` — undefined function	Semantic analysis (implicit-declaration warning) and then the **linker** ("undefined reference to print")	The standard library function is `printf`. No function named `print` is declared or defined.

Typical compiler messages (gcc)

```
prog.c:3:9: error: expected identifier or '(' before numeric constant     int 123 num = 10;
prog.c:5:5: error: expected ',' or ';' before 'float'                       int x = 5
prog.c:5:15: error: incompatible types when initializing type 'float' using type 'char *'
prog.c:6:5: warning: implicit declaration of function 'print'; did you mean 'printf'?
/usr/bin/ld: prog.o: undefined reference to `print'
```

Error 1 in detail — identifiers

The regular definition of an identifier is `letter_ (letter_ | digit)*`. "123" matches the *number* pattern, not the identifier pattern. With a space, the lexer produces the tokens `int`, `num(123)`, `id(num)`. The grammar rule *declaration → type id = expr ;* expects an identifier after `int`, so the

parser reports the error. If the programmer wrote `123num`, some lexers flag it as a **malformed lexeme** (a lexical error).

Error 2 — missing semicolon

Statement terminators are part of the grammar. The parser reads `5` and then sees `float`, which cannot follow an expression in a declaration. With panic-mode recovery it skips to the next ';' and continues. That is why the following errors are still reported.

Error 3 — type mismatch

The statement is *syntactically* valid (type id = expression;), so the parser accepts it. The **semantic analyser** checks the types: the right side has type `char *` (array of char) and the left side is `float`. No implicit conversion exists between them, so it reports a type error.

Error 4 — undefined function

Syntactically `print(...)` is a valid function call. In C89 an undeclared function is implicitly declared to return int (C99 and later: warning or error). The linker then searches the libraries for `print`, finds nothing, and reports an **undefined reference**.

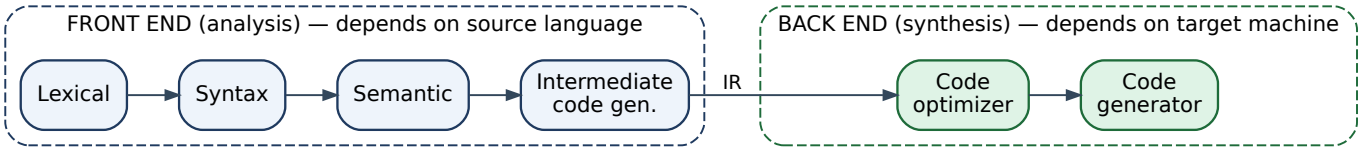
Classification summary

Error kind	Count here	Detected by
Lexical / syntax	2	Scanner / parser
Semantic	1 (+1 warning)	Type checker
Link time	1	Linker

Output of the corrected program: `value of x: 5`

2.3 Describe the grouping of compiler phases and their significance. 5 M

Apr-25 Q3c



- **Front end (analysis):** lexical, syntax and semantic analysis plus intermediate code generation. It depends on the *source language* and is independent of the target machine.
- **Back end (synthesis):** code optimization and code generation. It depends on the *target machine* and is independent of the source language.
- **Passes:** several phases can be grouped into one pass that reads one input file and writes one output file. For example, lexing, parsing, semantic analysis and IR generation can form a single front-end pass. Fewer passes make compilation faster; more passes use less memory and allow more optimization.

Significance: (1) **Retargeting** — one front end + several back ends = one language on many machines. (2) Several front ends + one back end = many languages on one machine (e.g. GCC,

LLVM). For *m* languages and *n* machines we need only m + n components instead of m × n compilers. (3) Each phase can be designed, tested and optimized separately.

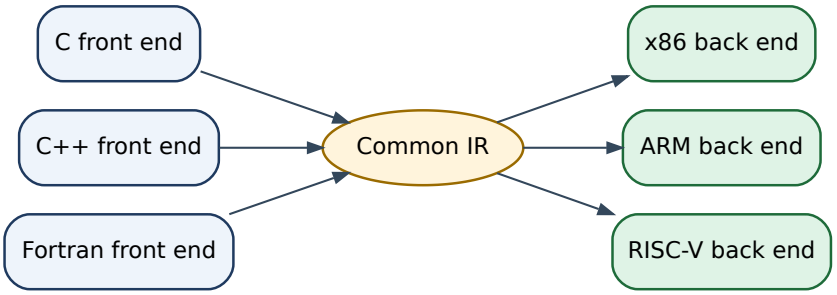


Fig. Sharing front ends and back ends through a common IR (m + n components instead of m × n compilers)

One-pass vs multi-pass compilers

One-pass	Multi-pass
All phases are done in one scan of the source	The source or IR is scanned several times; each pass does one or more phases
Faster compilation; less I/O	Slower, but allows global optimisation
Needs more memory (all phases loaded); forward references are hard (needs backpatching)	Each pass uses less memory; forward references are easy
e.g. early Pascal compilers	e.g. GCC, modern optimising compilers

Typical grouping of a 3-pass compiler

1. **Pass 1:** lexical + syntax + semantic analysis + IR generation (the front end).
2. **Pass 2:** machine-independent optimisation on the IR (optional).
3. **Pass 3:** code generation + machine-dependent optimisation (the back end).

Advantages of grouping into front end and back end

- **Portability / retargeting:** to support a new processor, write only a new back end.
- **Reusability:** one optimiser works for all languages (LLVM: Clang, Rust and Swift share LLVM's back ends).
- **Modularity:** phases are easier to design, test and maintain separately.
- **Optimisation:** machine-independent optimisation is done once, on the IR.

Example: for 3 languages and 3 machines, separate compilers would need 3 × 3 = 9 compilers. With a common IR we need only 3 + 3 = 6 components.

2.4 Describe the role of a compiler. 6 M

Aug-24 Q3c

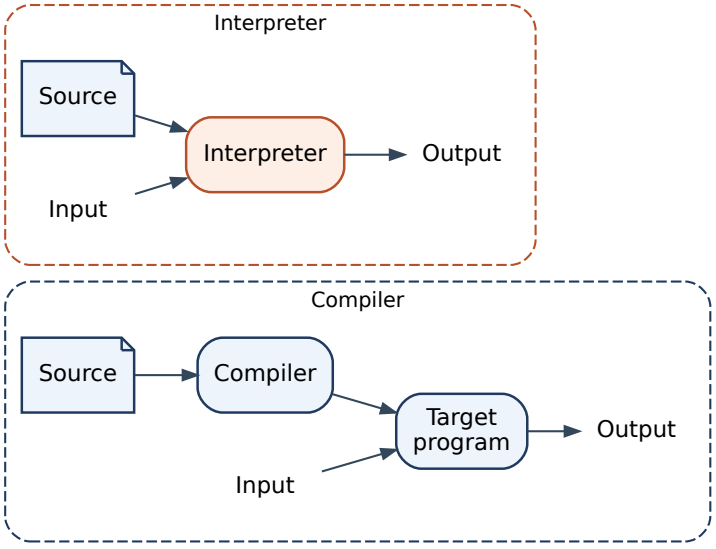
A **compiler** is a program that reads a program in one language (the *source*) and translates it into an equivalent program in another language (the *target*, usually machine code). While doing so, it reports any errors in the source program.

- **Translation:** converts high-level constructs (loops, functions, objects) into low-level instructions the hardware can execute.

- **Error detection and reporting:** finds lexical, syntax and semantic errors and reports them with line numbers, so the programmer can fix them before the program runs.
- **Optimization:** produces efficient code, such as fewer instructions or better register use, without changing what the program means.
- **Abstraction:** lets programmers write portable, readable code without knowing machine details.
- **Symbol management:** keeps the symbol table (names, types, scopes, storage) used for type checking and memory allocation.
- **Support for the toolchain:** works with the preprocessor, assembler, linker and loader to produce an executable.

Source program → **Compiler** → Target program; then Input → **Target program** → Output. The target program runs independently, without the compiler.

Diagram — compiler vs running the target



Requirements of a good compiler

- **Correctness:** the target code must keep exactly the meaning of the source program. This is the most important requirement.
- **Efficiency of the generated code:** fast and small.
- **Fast compilation**, which matters for large projects.
- **Good diagnostics:** clear error messages that give the right location.
- **Portability and maintainability** of the compiler itself.

Types of compilers

Type	Description
Single-pass / multi-pass	Number of times the source is processed
Cross compiler	Runs on one machine and produces code for another (e.g. compiling on x86 for an ARM microcontroller)
Source-to-source (transpiler)	Translates one high-level language into another (TypeScript → JavaScript)
JIT compiler	Compiles at run time (Java HotSpot, V8 for JavaScript)
Incremental compiler	Recompiles only the changed parts
Bootstrapping compiler	A compiler written in the language it compiles

Applications of compiler technology

Implementing high-level languages, optimising for new computer architectures (parallelism, memory hierarchies), translating between languages, software productivity tools (static analysers, type checkers, bounds checking), database query compilers (SQL), and hardware synthesis (Verilog/VHDL).

2.5 Who are the cousins of a compiler? Explain their roles. / Explain the different types of language processors. 5-6 M

Dec-25 Q4b Apr-25 Q3a

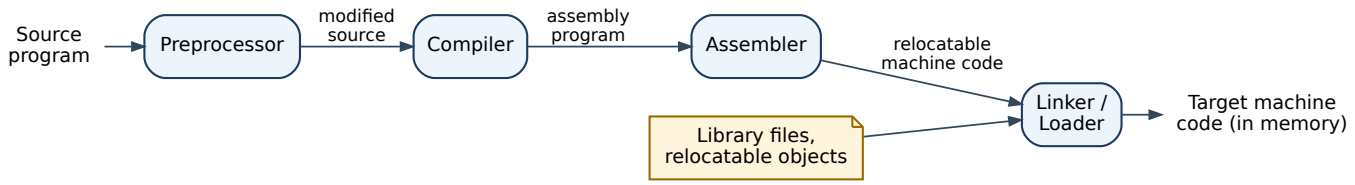


Fig. A language-processing system

Processor	Role
Preprocessor	Collects the source modules and expands macros (`#define`), file inclusion (`#include`), conditional compilation and language extensions. Its output is pure source code for the compiler.
Compiler	Translates the whole high-level program into assembly or machine code and reports errors.
Assembler	Translates assembly mnemonics into *relocatable machine code* (object files). It is usually two-pass: pass 1 builds the symbol table of labels, pass 2 generates the code.
Linker	Combines several object files and library routines. It resolves external references (for example a call to `printf`) and produces one executable.
Loader	Places the executable in memory, adjusts relocatable addresses (relocation), and starts execution.
Interpreter	Executes the source program statement by statement, without producing a separate target program (e.g. Python). Java uses a hybrid: it is compiled to bytecode, which the JVM interprets or JIT-compiles.

The preprocessor, assembler, linker and loader are called the **cousins of the compiler**: they work alongside it to turn source code into a running program.

Preprocessor — functions with examples

- **Macro processing:** `#define MAX 100`, `#define SQ(x) ((x)*(x))`; each use is textually replaced.
- **File inclusion:** `#include <stdio.h>` copies the header into the source.
- **Conditional compilation:** `#ifdef DEBUG … #endif`.
- **Rational preprocessors and language extensions:** they add constructs such as structured loops to older languages, or embed SQL in C.

Assembler

Example input: `MOV R1, a; ADD R1, #2; MOV a, R1`. **Pass 1** assigns addresses to labels and variables and fills the symbol table. **Pass 2** translates each mnemonic into its opcode bits and uses the symbol table to fill in addresses. The output is *relocatable*: addresses are relative to 0 and adjusted later.

Linker — static vs dynamic linking

Static linking	Dynamic linking
Library code is copied into the executable at link time	The library (.so/.dll) is loaded at run time and shared between programs
Larger executable, no runtime dependency	Smaller executable; the library can be updated separately

Loader — functions

Allocation (reserve memory), **linking** (resolve symbols, if not done already), **relocation** (adjust relocatable addresses), and **loading** (copy the code into memory). Types: absolute loader, relocating loader, direct linking loader, dynamic loader.

Interpreter and hybrid systems

An interpreter takes the source program and the input together and directly produces the output. A **hybrid compiler** (Java) first translates the source into bytecode (javac). The JVM then interprets it, or JIT-compiles hot code to machine code. This gives portability ("write once, run anywhere") and good speed.

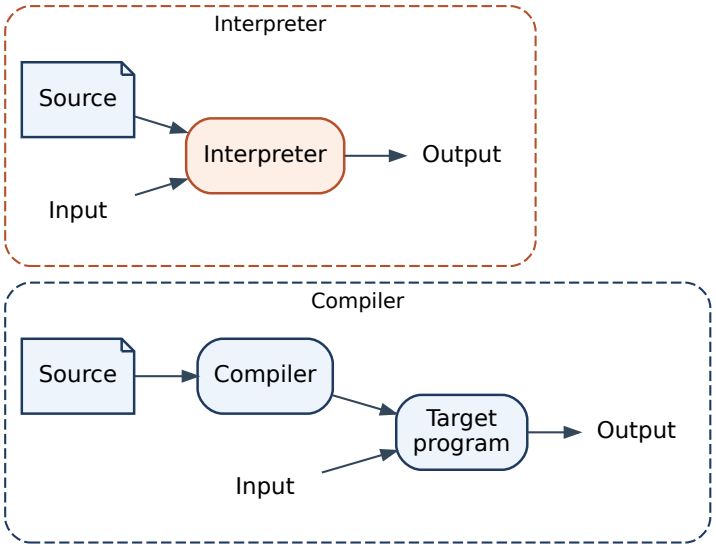
Flow for a C program: prog.c → (cpp) prog.i → (cc1) prog.s → (as) prog.o → (ld, + libc) a.out → (loader) process in memory.

2.6 What is the difference between a compiler and an interpreter? 4 M

Dec-25 Q3c

Compiler	Interpreter
Translates the entire program at once into target code	Translates and executes one statement at a time
Produces a separate object/executable file	Produces no target file
Reports errors after analysing the whole program	Stops at the first error it finds — easier to debug
Generated code runs **faster**	Execution is **slower** (it re-analyses code each time)
Uses more memory at compile time (builds object code)	Uses less memory; the source is needed at every run
Examples: C, C++, Go	Examples: Python, Ruby, JavaScript (classically)

Diagram



Further points of difference

Basis	Compiler	Interpreter
Program analysis	Analyses the whole program; can optimise globally	Analyses each statement as it runs; little optimisation
Intermediate object code	Generated	Not generated
Recompilation	Needed after every change	Not needed; just run again
Security of the source	Only the binary is distributed (source hidden)	The source must be distributed
Typical use	Production, performance-critical software	Scripting, prototyping, teaching, REPLs

Example

For `for (i = 0; i < 1000; i++) s += i;`, a compiler translates the loop body once into machine code that runs 1000 times. An interpreter re-examines and re-dispatches the statement `s += i` on each of the 1000 iterations, so it is much slower.

Hybrid approach: Java and C# compile to bytecode and then interpret or JIT-compile it, combining the portability of interpretation with the speed of compilation.

2.7 List and explain the different compiler-construction tools and their purposes. 6 M

Dec-25 Q4a

1. **Scanner generators** (e.g. *Lex, Flex*): take regular-expression descriptions of tokens and produce a lexical analyzer (a DFA-driven C program).
2. **Parser generators** (e.g. *Yacc, Bison, ANTLR*): take a grammar description (CFG) and automatically produce a syntax analyzer, usually an LALR or LL parser.
3. **Syntax-directed translation engines:** produce routines that walk the parse tree and generate intermediate code, using translations attached to grammar productions.
4. **Code-generator generators:** take rules for translating each IR operation into target-machine instructions (template matching) and produce a code generator.
5. **Data-flow analysis engines:** collect information about how values flow through the program (live variables, reaching definitions). This is the key requirement for code optimization.
6. **Compiler-construction toolkits** (e.g. LLVM, GCC infrastructure): integrated sets of routines for building the different phases of a compiler.

Summary table

Tool	Input	Output	Phase supported
Scanner generator (Lex/Flex)	Regular expressions + actions	Lexical analyser (C code)	Lexical analysis
Parser generator (Yacc/Bison/ANTLR)	Grammar + semantic actions	LALR / LL parser	Syntax analysis
Syntax-directed translation engine	Parse tree + translation rules	Intermediate code	Semantic analysis / ICG
Code-generator generator (e.g. BURG, iburg)	Target-instruction patterns + costs	Code generator (tree-pattern matcher)	Code generation
Data-flow analysis engine	Flow graph + transfer functions	Data-flow facts (live variables, reaching definitions)	Optimisation
Compiler-construction toolkit (LLVM, GCC)	—	Reusable IR, optimisation passes, back ends	All phases

How they fit together

```
lex scanner.l     →  lex.yy.c   (yylex)
yacc -d parser.y  →  y.tab.c    (yyparse, with semantic actions that build the IR)
cc lex.yy.c y.tab.c -o mycompiler
```

Modern toolkits such as **LLVM** provide the optimiser and code generator as libraries. A new language then only needs a front end that emits LLVM IR.

Advantages of using tools

- Faster development: the hard algorithms (DFA construction, LALR tables) are automated.
- Fewer bugs, since the generated code is well tested.
- Easy to change the language: edit the specification and regenerate.
- Formal specifications (regular expressions, grammars) double as documentation.

2.8 Explain input buffering in detail. Describe the role of the forward pointer and the lexeme-begin pointer. How do they work together to recognize tokens? 7–8 M

Dec-25 Q4c Apr-25 Q4b Aug-24 Q4c

Why buffering? The lexer reads the source one character at a time, and often must *look ahead* one or more characters to decide the token. For example, after '>' it must check whether the next character is '=', and an identifier ends only when a non-letter is seen. Reading characters one by one from disk would be very slow. So the input is read in large blocks into a buffer.

Buffer pairs

Two buffers of N characters each (N = one disk block, e.g. 4096 bytes) are filled alternately with a single system read each. Two pointers are maintained:

- **lexemeBegin** — marks the start of the current lexeme being recognized.
- **forward** — scans ahead until a pattern match is found.



Fig. Buffer pair with sentinels (eof) at the end of each half

How the two pointers work together

1. At the start of each lexeme, both pointers point to its first character.
2. **forward** moves right one character at a time, as long as the characters can still form a valid token (following the DFA or transition diagram).
3. When a lexeme is complete (e.g. a blank or an operator follows an identifier), forward may have gone one character too far, so it is **retracted** by one.
4. The characters between lexemeBegin and forward form the **lexeme**. The token <name, attribute> is returned to the parser.
5. lexemeBegin is set to the character just after the lexeme, and scanning continues.

Sentinels

Without sentinels, every time forward advances we must make *two* tests: "is it the end of the buffer?" and "which character is it?". Placing a special **eof sentinel** at the end of each half means usually only one test is needed:

```
switch (*forward++) {
  case eof:
     if (forward is at end of first buffer)  { reload second buffer; forward = beginning of
second buffer; }
     else if (forward is at end of second buffer) { reload first buffer; forward = beginning
of first buffer; }
     else  /* eof within a buffer: real end of input */  terminate lexical analysis;
     break;
  /* cases for the other characters */
}
```

> **Limitation:** a lexeme longer than N characters cannot be handled, and lookahead is limited to N characters.

Step-by-step scan of "E = M * C ** 2"

Step	lexemeBegin at	forward moves over	Lexeme	Token returned
1	E	E, then '=' (retract one)	E	<id, ptr-E>
2	=	=, then 'M' (retract)	=	<assign_op>
3	M	M, '*' (retract)	M	<id, ptr-M>
4	*	*, 'C' (retract)	*	<mult_op>
5	C	C, '*' (retract)	C	<id, ptr-C>
6	*	*, *, then '2' (retract)	**	<exp_op> (longest match)
7	2	2, then eof (retract)	2	<number, 2>

Step 6 shows why lookahead is needed. After the first '*' the lexer cannot decide between * and ** until it sees the next character.

Lookahead code without sentinels (two tests per character)

```
if (forward at end of first buffer) { reload second buffer; forward = beginning of second
buffer; }
else if (forward at end of second buffer) { reload first buffer; forward = beginning of first
buffer; }
else forward++;
/* then: test which character *forward is */
```

One buffer vs buffer pair

Single buffer	Buffer pair
When the buffer is refilled, a lexeme that spans the boundary is lost	While one half is being processed the other half holds the next block, so a lexeme can cross halves
Simple	Needs two end-of-buffer tests; sentinels reduce this to about one test per character

Why it matters

The lexical analyser is the only phase that reads every character of the source. It can take a large share of compilation time, so reducing the cost per character with block reads, two buffers and sentinels speeds up the whole compiler noticeably.

Exam points to cover: (1) the need for buffering, (2) the buffer-pair diagram, (3) the two pointers and their movement, (4) retraction, (5) the sentinel code, (6) the limitation. These are enough for 10–15 marks.

2.9 Explain the LEX tool with the structure of a Lex program. 7 M

Apr-25 Q4a Aug-24 Q3a

Lex (or its newer version Flex) is a **scanner generator**. The programmer writes token patterns as regular expressions together with actions. Lex converts them into an NFA, then a DFA, and outputs C code, `lex.yy.c`, that simulates the DFA.

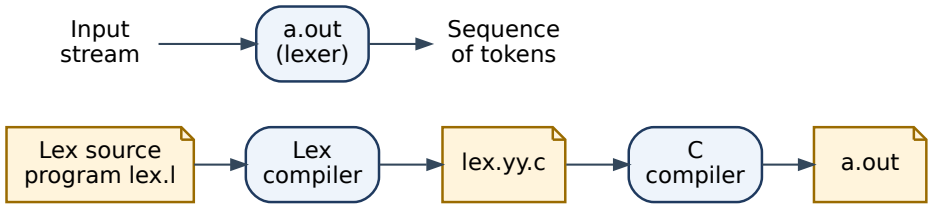


Fig. Creating a lexical analyzer with Lex

Structure of a Lex program

```
declarations
%%
translation rules
%%
auxiliary functions
```

- **Declarations:** C declarations in `%{ … %}` (header files, variables, token constants) and *regular definitions* such as `digit [0-9]`.
- **Translation rules:** each has the form `Pattern { Action }`. The action is C code that runs when the pattern matches.
- **Auxiliary functions:** user functions such as `main()`, `yywrap()`, `installID()`.

```
%{
#include <stdio.h>
%}
digit   [0-9]
letter  [a-zA-Z_]
%%
[ \t\n]+                    { /* skip white space */ }
if|else|while               { printf("KEYWORD: %s\n", yytext); }
{letter}({letter}|{digit})* { printf("ID: %s\n", yytext); }
{digit}+                    { printf("NUMBER: %s\n", yytext); }
"<="|">="|"=="|"<"|">"     { printf("RELOP: %s\n", yytext); }
.                           { printf("OTHER: %s\n", yytext); }
%%
int yywrap() { return 1; }
int main()   { yylex(); return 0; }
```

Conflict rules: (1) the *longest* matching prefix is preferred; (2) if two patterns match the same length, the one listed *first* wins. That is why keywords are listed before identifiers. Useful built-ins: `yytext` (the lexeme), `yyleng` (its length), `yylex()` (the scanner), `yylval` (the attribute value passed to the parser).

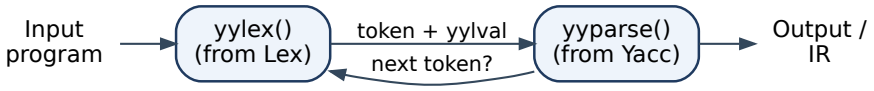
How Lex works internally

1. Each pattern p_i is converted into an NFA N_i (Thompson's construction).
2. A new start state with ε-moves to every N_i combines them into one NFA.
3. The NFA is converted to a DFA (subset construction) and minimised.
4. The DFA is stored as a transition table, together with a driver routine in lex.yy.c.
5. At run time the driver simulates the DFA, remembers the **last accepting state** (for the longest match), and runs the action of the earliest pattern matching at that state.

Sample run of the Lex program above

```
Input :  if count1 <= 25 then
Output:  KEYWORD: if
         ID: count1
         RELOP: <=
         NUMBER: 25
         ID: then        (then is not in our keyword list)
```

Lex used together with Yacc




```
{digit}+   { yylval = atoi(yytext); return NUMBER; }   /* token code comes from y.tab.h */
"+"        { return PLUS; }
```

Special features

- **Lookahead operator /**: `IF/\(` matches "IF" only when it is followed by '('. This is needed for languages like Fortran.
- **Start conditions** (`%x COMMENT`) for context-dependent scanning, e.g. inside comments.
- `yymore()`, `yyless(n)`, `ECHO`, `REJECT`.

Commands: `lex file.l` → lex.yy.c; `cc lex.yy.c -ll` → a.out; `./a.out < input.txt`.

2.10 Explain the process of token recognition in lexical analysis. / Write a regular expression for relational operators and design a transition diagram for them. — 5 M

Apr-25 Q4c Aug-24 Q5b

Terms: a **token** is a pair <token-name, attribute> (e.g. <id, ptr>). A **pattern** describes the form of a token's lexemes (usually a regular expression). A **lexeme** is the actual character sequence that matches a pattern (e.g. "count").

Process: (1) token patterns are written as regular definitions; (2) each one is converted into a **transition diagram** (a DFA with actions); (3) the lexer simulates these diagrams on the input buffer, uses the longest match, and retracts when it has read one extra character (shown by *); (4) it returns the token and installs identifiers or numbers in the symbol table. Keywords are either pre-loaded into the symbol table or given their own diagrams.

Regular definitions:

```
relop  →  <  |  <=  |  =  |  <>  |  >  |  >=
id     →  letter ( letter | digit )*
number →  digit+ ( . digit+ )? ( E (+|-)? digit+ )?
```

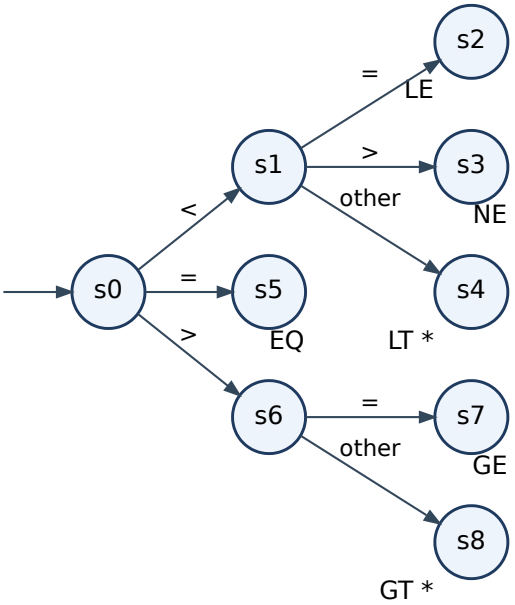


Fig. Transition diagram for relop (* = retract forward by one; return <relop, attribute>)

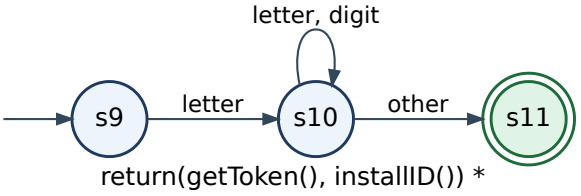


Fig. Transition diagram for identifiers and keywords

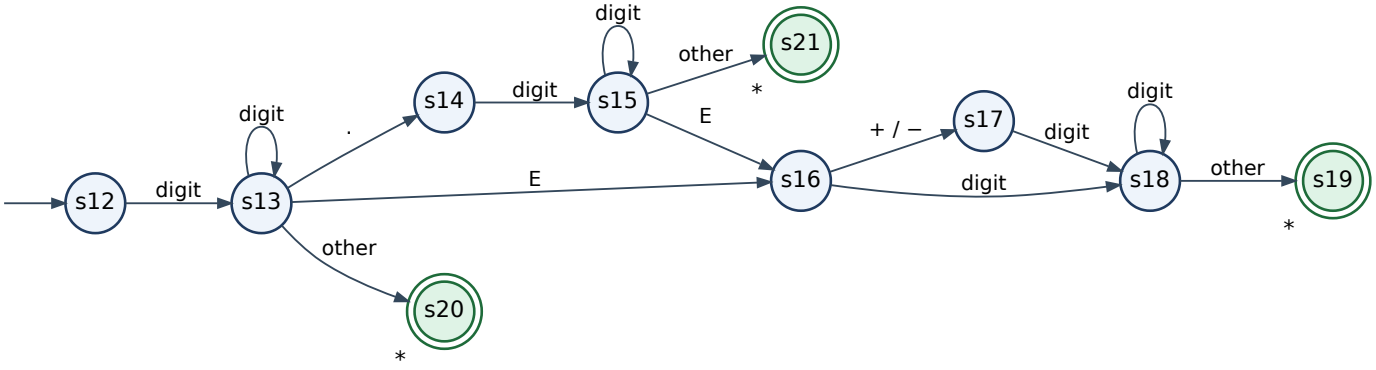


Fig. Transition diagram for unsigned numbers (e.g. 25, 3.14, 6.02E23)

Implementing the relop diagram — getRelop()

```
TOKEN getRelop() {
    TOKEN retToken = new(RELOP);
    while (1) {
        switch (state) {
        case 0: c = nextChar();
                if (c == '<') state = 1;
                else if (c == '=') state = 5;
                else if (c == '>') state = 6;
                else fail();              /* lexeme is not a relop */
                break;
        case 1: c = nextChar();
                if (c == '=') { retToken.attribute = LE; return retToken; }
                else if (c == '>') { retToken.attribute = NE; return retToken; }
                else { retract(); retToken.attribute = LT; return retToken; }
        /* ... cases 5, 6, 7, 8 similarly ... */
        }
    }
}
```

Handling keywords vs identifiers

- **Method 1:** pre-install the reserved words (if, then, else) in the symbol table. When the id diagram accepts, `getToken()` looks the lexeme up and returns either the keyword token or id.
- **Method 2:** create separate transition diagrams for each keyword, and check that the next character is not a letter or digit (so that `ifx` is an id).

White space

`ws → (blank | tab | newline)+`. When ws is recognised, no token is returned; the lexer restarts at the next lexeme.

Token table for the relational operators

Lexeme	<	<=	=	<>	>	>=
Token	relop					
Attribute	LT	LE	EQ	NE	GT	GE

2.11 Describe the lexical errors and the various error-recovery strategies, with suitable examples. 8 M

Aug-24 Q6b

A **lexical error** occurs when no token pattern matches a prefix of the remaining input. The lexer sees the source only locally, so it can catch few errors. For example, in `fi (a == f(x))` it cannot tell that `fi` is a misspelled `if`: it is a valid identifier. Typical lexical errors:

- Illegal characters: `int a = 5 $ 3;` or `@count`
- Malformed identifiers or numbers: `2abc`, `3.14.5`, `1e`
- Unterminated strings or comments: `"hello`, `/* …` with no closing `*/`
- Identifiers or literals that are too long (exceed the compiler's limit)

Error-recovery strategies in the lexer

Strategy	Description	Example
Panic mode	Delete successive characters from the remaining input until a well-formed token is found. Simple and never loops forever.	`a = 5 $$ + 3` → skip `"$$"`
Delete one character	Remove an extraneous character	`whi#le` → `while`
Insert a missing character	Add a character to form a valid token	`"hello` → `"hello"`
Replace a character	Change one character into another	`1o0` → `100` (letter o replaced by digit 0)
Transpose two adjacent characters	Swap neighbours	`fi` → `if`, `esle` → `else`

The **minimum-distance** idea: choose the smallest number of such single-character transformations that turns the input into a valid lexeme. This is expensive, so in practice panic mode is used most.

General (parser-level) strategies: panic mode (skip to a synchronizing token such as ';' or '}'), phrase-level recovery (local corrections such as inserting a missing ';'), error productions (grammar rules for common mistakes), and global correction (the least-cost change to the whole program).

Example session

```
int main() {
    int a = 10$;          → lexical error: stray '$' in program  (panic mode deletes '$')
    char c = 'ab;         → lexical error: missing terminating ' character
    float f = 3.14.15;    → lexical error: too many decimal points in number
    int 9lives = 3;       → invalid suffix "lives" on integer constant
}
```

Error recovery at other phases (for context)

Strategy	Used at	Idea	Example
Panic mode	Parser	Discard tokens until a synchronising token (; or })	`x = = 5;` → skip to ;
Phrase level	Parser	Local correction: insert, delete or replace a token	insert a missing ;
Error productions	Parser	Add rules for common mistakes	`if E S else else S`
Global correction	Parser (theoretical)	Minimum number of changes to make the whole program valid	too costly in practice
Type recovery	Semantic	Give an erroneous expression the type type_error and suppress cascading messages	undeclared x: declare it as int

Goals of error handling

- Report errors clearly and accurately, with line and column.

- Recover quickly enough to detect later errors.
- Add little overhead to the processing of correct programs.
- Avoid cascades of spurious errors caused by one real error.

UNIT III

Syntax Analysis — Top-Down Parsing

Role of the parser · left recursion & factoring · FIRST/FOLLOW · LL(1) tables · backtracking · error recovery

3.1 What is a parser? Explain the role of a parser and its types in detail. 7–8 M

Dec-25 Q5a Apr-25 Q6c Aug-24 Q4a

A **parser** (syntax analyzer) is the second phase of a compiler. It takes the token stream from the lexical analyzer and checks whether the tokens can be generated by the grammar of the source language. If they can, it builds a **parse tree** (or syntax tree) for the rest of the front end.

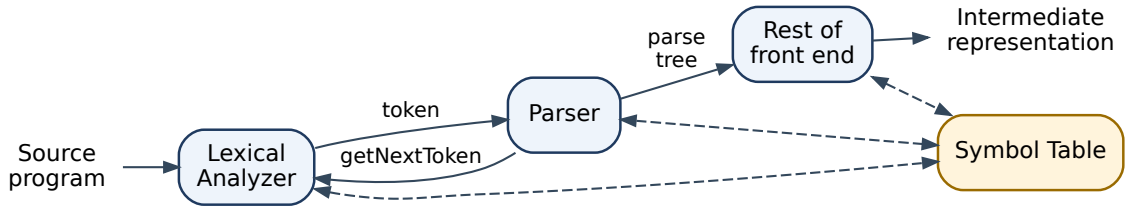


Fig. Position of the parser in the compiler model

Role of the parser

- Gets tokens on demand from the lexer (`getNextToken`) and verifies the syntax using the CFG.
- Builds the parse tree or syntax tree and passes it to semantic analysis and IR generation.
- Reports syntax errors clearly (line and position) and **recovers** from common errors so it can continue and find more of them.
- Collects information into the symbol table and may perform type checking or translation (syntax-directed translation).

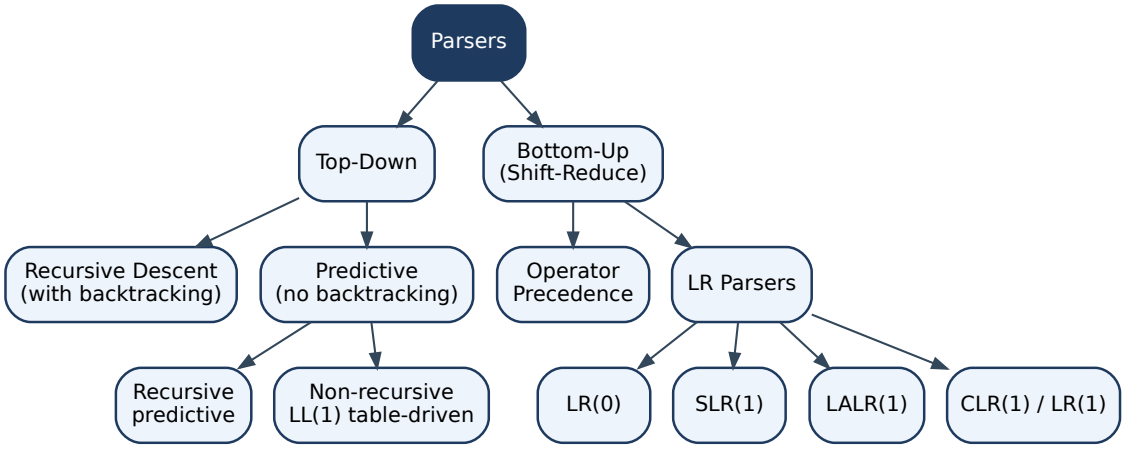


Fig. Classification of parsers

1. Top-down parsers build the parse tree from the root (start symbol) down to the leaves. They follow a *leftmost derivation*.

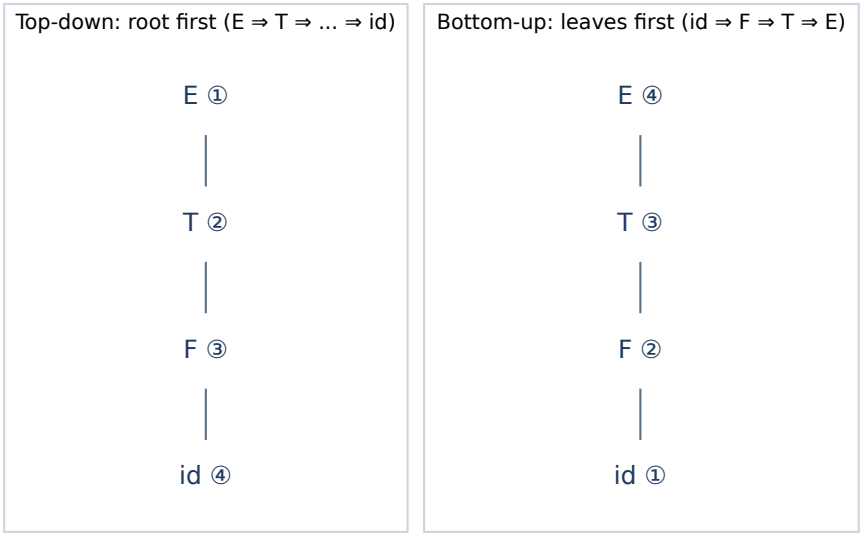
- **Recursive descent with backtracking:** one procedure per non-terminal. It tries alternatives one by one and backtracks on failure. Simple but slow.

- **Predictive parsing (no backtracking):** the right production is chosen by looking at one lookahead token. It needs a grammar with no left recursion that has been left-factored, i.e. an LL(1) grammar. It can be recursive-predictive or **non-recursive (table-driven) LL(1)**, which uses an explicit stack and a parsing table.

2. Bottom-up parsers build the tree from the leaves up to the root. They trace a *rightmost derivation in reverse*, using shift and reduce actions.

- **Operator-precedence parser:** for operator grammars (no ε-productions and no two adjacent non-terminals). It uses the precedence relations <·, ≐, ·>.
- **LR parsers:** LR(0), SLR(1), LALR(1) and CLR(1). They are the most powerful practical parsers and are used by Yacc and Bison.

Top-down vs bottom-up — the order in which the tree is built



Top-down parsing	Bottom-up parsing
Starts at the root (S) and works towards the leaves	Starts at the leaves (the input) and works towards the root
Follows a leftmost derivation	Follows a rightmost derivation in reverse
Main decision: *which production to expand*	Main decision: *when to reduce, and by which production* (finding the handle)
Cannot handle left recursion; needs left factoring	Handles left recursion naturally
Works for the smaller class of LL(k) grammars	Works for the larger class of LR(k) grammars
Easy to write by hand (recursive descent)	Usually generated by tools (Yacc)
e.g. recursive descent, LL(1)	e.g. operator precedence, SLR, LALR, CLR

Error-recovery strategies of a parser

- **Panic mode:** discard input until a synchronising token is found (e.g. ';').
- **Phrase level:** make a local correction (insert ';', delete an extra ',').
- **Error productions:** extend the grammar with rules for common errors.

- **Global correction:** find the minimum change needed to make the input valid (theoretical).

Parse tree vs syntax tree

A **parse tree** shows every grammar symbol used in the derivation, including chain rules such as E → T → F. A **syntax tree** (AST) keeps only the operators and operands, so it is more compact. Parsers usually build the AST directly.

Summary: the parser checks syntax, builds the tree, reports and recovers from errors, and drives later translation. Parsers are classified as top-down (backtracking or predictive) and bottom-up (operator precedence or LR family).

3.2 Explain left-recursive grammars and left factoring with examples. 6–7 M

Dec-25 Q6a Aug-24 Q4b

Left recursion

A grammar is **left recursive** if it has a non-terminal A such that $A \Rightarrow^{+} A\alpha$. With *immediate* left recursion, A → Aα. A top-down parser expanding A would call A again without consuming any input, so it loops forever. Left recursion must therefore be removed.

Rule: $A \to A\alpha_1 \mid A\alpha_2 \mid \ldots \mid \beta_1 \mid \beta_2 \mid \ldots \implies \mathbf{A \to \beta_1 A' \mid \beta_2 A' \mid \ldots}$ and $\mathbf{A' \to \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \varepsilon}$

Example: E → E + T | T ⟹ E → TE', E' → +TE' | ε. Similarly T → T * F | F ⟹ T → FT', T' → *FT' | ε.

Left factoring

When two alternatives of A start with the same prefix, a predictive parser cannot choose between them with one lookahead symbol. **Left factoring** takes out the common prefix and delays the decision until enough input has been seen.

Rule: $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma \implies \mathbf{A \to \alpha A' \mid \gamma}, \quad \mathbf{A' \to \beta_1 \mid \beta_2}$

Example (dangling else): S → iEtS | iEtSeS | a ⟹ S → iEtSS' | a, S' → eS | ε.

Difference: left recursion causes *infinite looping* in top-down parsers; a missing left factoring causes *non-determinism* (backtracking). Both must be fixed to obtain an LL(1) grammar.

Indirect left recursion

Left recursion can also arise through several steps, e.g. S → Aa | b, A → Ac | Sd | ε. Here S ⇒ Aa ⇒ Sda, so S is left recursive. To remove it, order the non-terminals (S, A), substitute the earlier ones into the later ones, and then remove the immediate left recursion.

```
Algorithm (non-terminals ordered A1 … An):
for i = 1 to n:
    for j = 1 to i-1:
        replace each Ai → Aj γ by Ai → δ1 γ | δ2 γ | …   (where Aj → δ1 | δ2 | … )
    eliminate immediate left recursion among the Ai-productions
```

Working: substitute S into A → Sd: A → Ac | Aad | bd | ε. Then remove the immediate recursion:

S → Aa | b A → bdA' | A' A' → cA' | adA' | ε

Why left recursion loops — the recursive-descent view

```
void E() {  E();  match('+');  T();  }   /* E calls E without reading input → infinite
recursion */
```

After the transformation, the procedure `E() { T(); Eprime(); }` always reads input through T before it can recurse.

Left factoring — algorithm

```
for each non-terminal A:
    find the longest prefix α common to two or more A-alternatives
    if α ≠ ε: replace  A → αβ1 | … | αβn | γ   by   A → αA' | γ ,  A' → β1 | … | βn
repeat until no two alternatives share a prefix
```

Effect on the language

Both transformations keep the language the same; only the structure of the parse tree changes. E → TE' still generates exactly the strings of E → E + T | T. Associativity information is no longer visible in the tree, so it has to be restored with attributes in the translation scheme.

	Left recursion	Left factoring
Form	A → Aα | β	A → αβ₁ | αβ₂
Problem	Infinite loop in top-down parsing	Cannot choose an alternative with 1 lookahead (backtracking)
Fix	A → βA', A' → αA' | ε	A → αA', A' → β₁ | β₂
Needed for	All top-down parsers	Predictive (LL(1)) parsers

3.3 (i) Convert S → a | ^ | (T), T → T,S | S into a non-left-recursive form. (ii) Left-factor S → iEtS | iEtSeS | a, E → b. — 6 M

Apr-25 Q6b

(i) S has no left recursion. T → T,S | S is immediately left recursive, with α = ",S" and β = "S". Applying the rule:

S → a | ^ | (T) T → S T' T' → , S T' | ε

(ii) The common prefix of the first two S-alternatives is α = iEtS, with β₁ = ε and β₂ = eS:

S → iEtS S' | a S' → eS | ε E → b

Checking that (i) is now LL(1)

NT	FIRST	FOLLOW
S	{ a, ^, (}	{ $, ",",) }
T	{ a, ^, (}	{) }
T'	{ ",", ε }	{) }

NT	a	^	(	)	,	$
S	S→a	S→^	S→(T)			
T	T→ST'	T→ST'	T→ST'			
T'				T'→ε	T'→,ST'	

There are no conflicts, so the converted grammar is **LL(1)**. **Example derivation** of (a,^): S ⇒ (T) ⇒ (ST') ⇒ (aT') ⇒ (a,ST') ⇒ (a,^T') ⇒ (a,^).

Checking (ii) — the dangling-else conflict

FIRST(S) = {i, a}, FIRST(S') = {e, ε}, FOLLOW(S) = FOLLOW(S') = {$, e}. For S' → ε we enter it under FOLLOW(S'), which includes e. But S' → eS is also entered under e. So **M[S', e] = { S' → eS, S' → ε }** is a conflict, and the grammar is **not LL(1)**, even after left factoring. That is because the grammar is ambiguous.

NT	i	t	a	e	b	$
S	S→iEtSS'		S→a			
S'				S'→eS S'→ε		S'→ε
E					E→b	

Practical resolution: keep only S' → eS in M[S', e]. This matches each else with the nearest unmatched then, which is the rule used by C and Java.

3.4 Perform left factoring for the grammar A → abB | aB | cdg | cdeB | cdfB. — 7 M

Aug-24 Q3b

Step 1: Group the alternatives by common prefix: {abB, aB} share **a**; {cdg, cdeB, cdfB} share **cd**.

Step 2: Factor out **a**: A → aA' | cdg | cdeB | cdfB, A' → bB | B

Step 3: Factor out **cd**: A → aA' | cdA'', A'' → g | eB | fB

A → aA' | cdA''
A' → bB | B
A'' → g | eB | fB

Now no two alternatives of any non-terminal share a common prefix, so a predictive parser can choose a production with one lookahead symbol.

The general left-factoring rule applied

A → αβ₁ | αβ₂ | … | γ becomes A → αA' | γ, A' → β₁ | β₂ | …

- Round 1: α = **a**, β₁ = bB, β₂ = B. The other alternatives (cdg, cdeB, cdfB) form the γ's.
- Round 2: in the remaining alternatives, α = **cd**, β = g, eB, fB.
- Round 3: check A', A'' and A again. bB and B start with different symbols (b vs the first symbol of B); g, eB and fB start with g, e and f. No common prefix is left, so we stop.

Why this helps — before and after

Before	After
On input "cd...", the parser must choose among cdg, cdeB and cdfB, but all start with c. It has to guess and backtrack.	On "c" the only choice is A → cdA''. After reading "cd", the next symbol (g, e or f) selects the A'' alternative directly.

Derivation check

Original: A ⇒ cdeB. Factored: A ⇒ cdA'' ⇒ cdeB ✔. Original: A ⇒ abB. Factored: A ⇒ aA' ⇒ abB ✔. Every original sentential form can still be derived, so the language is unchanged.

Note on B

Here B is another non-terminal of the grammar. If FIRST(B) contains b, then A' → bB | B still needs a second round of factoring (e.g. if B → bX, then A' → bA''' with A''' → B | X). For full marks, mention that factoring is repeated until no common prefixes remain.

Final answer: A → aA' | cdA'' ; A' → bB | B ; A'' → g | eB | fB

3.5 Compute FIRST and FOLLOW for the grammar S → Bb | Cd, B → aB | ε, C → cC | ε. — 6 M

Dec-25 Q6c Apr-25 Q5a

FIRST rules

- If X is a terminal, FIRST(X) = {X}.
- If X → ε, add ε to FIRST(X).
- If X → Y_1Y_2..., add FIRST(Y_1) − {ε}. If Y_1 ⇒* ε, also add FIRST(Y_2), and so on.

FOLLOW rules

- Put $ in FOLLOW(S).
- For A → αBβ, add FIRST(β) − {ε} to FOLLOW(B).
- For A → αB, or A → αBβ with β ⇒* ε, add FOLLOW(A) to FOLLOW(B).

FIRST:

- FIRST(B) = {a, ε} (B → aB gives a; B → ε gives ε)
- FIRST(C) = {c, ε}
- FIRST(S): from S → Bb: FIRST(B) − ε = {a}, and B ⇒ ε so add b. From S → Cd: {c}, and C ⇒ ε so add d. ⇒ FIRST(S) = {a, b, c, d}

FOLLOW:

- FOLLOW(S) = {$} (start symbol; S never appears on a right side)
- FOLLOW(B): in S → Bb, b follows B; in B → aB, add FOLLOW(B) itself. ⇒ {b}
- FOLLOW(C): in S → Cd ⇒ {d}

Non-terminal	FIRST	FOLLOW
S	{a, b, c, d}	{$}
B	{a, ε}	{b}
C	{c, ε}	{d}

The predictive table built from these has no conflicts (M[S,a] = M[S,b] = S→Bb; M[S,c] = M[S,d] = S→Cd; M[B,a] = B→aB; M[B,b] = B→ε; M[C,c] = C→cC; M[C,d] = C→ε), so the grammar is LL(1).

Step-by-step computation

Step	Reasoning	Result
FIRST(B)	B → aB adds a; B → ε adds ε	{a, ε}
FIRST(C)	C → cC adds c; C → ε adds ε	{c, ε}
FIRST(Bb)	FIRST(B) − ε = {a}; B is nullable, so add FIRST(b) = {b}	{a, b}
FIRST(Cd)	{c}; C is nullable, so add {d}	{c, d}
FIRST(S)	FIRST(Bb) ∪ FIRST(Cd)	{a, b, c, d}
FOLLOW(S)	start symbol	{$}
FOLLOW(B)	S → B**b** adds b; B → aB: FOLLOW(B) ⊆ FOLLOW(B), nothing new	{b}
FOLLOW(C)	S → C**d** adds d	{d}

LL(1) parsing table

NT	a	b	c	d	$
S	S→Bb	S→Bb	S→Cd	S→Cd	
B	B→aB	B→ε			
C			C→cC	C→ε	

Parsing "aab"

Stack	Input	Action
$S	aab$	S → Bb
$bB	aab$	B → aB
$bBa	aab$	match a
$bB	ab$	B → aB
$bBa	ab$	match a
$bB	b$	B → ε
$b	b$	match b
$	$	**accept**

Language: L = a*b ∪ c*d. Each table cell has at most one entry, so the grammar is **LL(1)**.

> **Why FIRST and FOLLOW matter:** FIRST tells the parser which production to use for a given lookahead. FOLLOW tells it when an ε-production may be used. Both are also used to build SLR tables and synchronising sets for error recovery.

3.6 Construct the predictive parser (LL(1) table) for E → E+T | T, T → T*F | F, F → (E) | id (equivalently E → TE', E' → +TE' | ε, T → FT', T' → *FT' | ε, F → id | (E)). Check whether it is LL(1) and show the moves of the parser on id + id * id. 8-10 M

Dec-25 Q5b Apr-25 Q5b Aug-24 Q5a

Step 1 — Remove left recursion

E → E+T | T and T → T*F | F are left recursive. A top-down parser would loop forever, so **the original grammar is not LL(1)**. Using A → Aα | β ⇒ A → βA', A' → αA' | ε:

```
E  → T E'          E' → + T E' | ε
T  → F T'          T' → * F T' | ε
F  → ( E ) | id
```

Step 2 — FIRST and FOLLOW

NT	FIRST	FOLLOW
E	{ (, id }	{), $ }
E'	{ +, ε }	{), $ }
T	{ (, id }	{ +,), $ }
T'	{ *, ε }	{ +,), $ }
F	{ (, id }	{ *, +,), $ }

Step 3 — Parsing table M[A, a]

For each A → α: add it to M[A, a] for every a ∈ FIRST(α). If ε ∈ FIRST(α), add it to M[A, b] for every b ∈ FOLLOW(A).

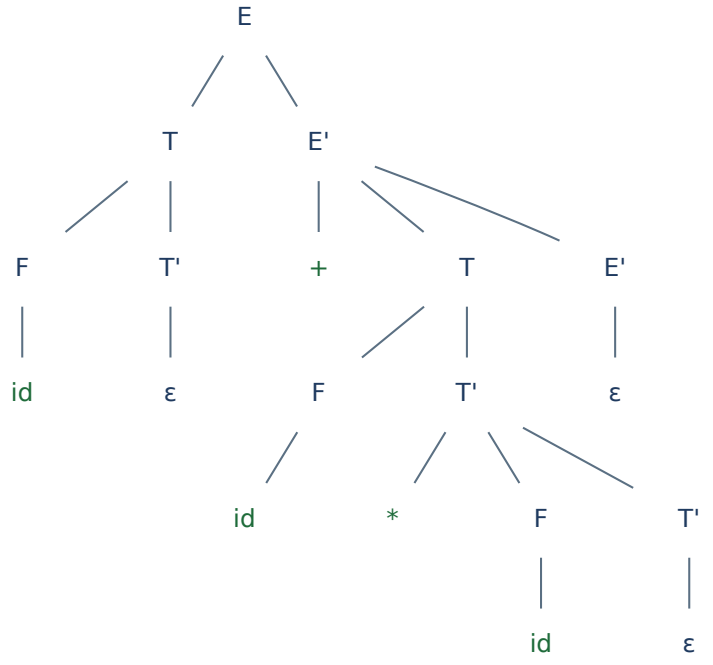
NT	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

LL(1) check: every cell has at most one production (no multiple entries), so the transformed grammar **is LL(1)**.

Step 4 — Moves on w = id + id * id $

Stack (top at right)	Input	Action / Output
$E	id+id*id$	E → TE'
$E'T	id+id*id$	T → FT'
$E'T'F	id+id*id$	F → id
$E'T'id	id+id*id$	match id
$E'T'	+id*id$	T' → ε
$E'	+id*id$	E' → +TE'
$E'T+	+id*id$	match +
$E'T	id*id$	T → FT'
$E'T'F	id*id$	F → id
$E'T'id	id*id$	match id
$E'T'	*id$	T' → *FT'
$E'T'F*	*id$	match *
$E'T'F	id$	F → id
$E'T'id	id$	match id
$E'T'	$	T' → ε
$E'	$	E' → ε
$	$	**ACCEPT**

Parse tree produced



Parse tree for id + id * id (the leftmost derivation traced above)

Leftmost derivation (the output sequence)

```
E ⇒ TE' ⇒ FT'E' ⇒ id T'E' ⇒ id E' ⇒ id + TE' ⇒ id + FT'E' ⇒ id + id T'E'
  ⇒ id + id * FT'E' ⇒ id + id * id T'E' ⇒ id + id * id E' ⇒ id + id * id
```

Equivalent recursive-descent parser

```
void E()  { T(); Eprime(); }
void Eprime() { if (look == '+') { match('+'); T(); Eprime(); } /* else ε */ }
void T()  { F(); Tprime(); }
void Tprime() { if (look == '*') { match('*'); F(); Tprime(); } /* else ε */ }
void F()  { if (look == ID) match(ID);
            else if (look == '(') { match('('); E(); match(')'); }
            else error(); }
```

Conditions for a grammar to be LL(1)

For every pair of productions A → α | β:

1. FIRST(α) ∩ FIRST(β) = ∅.
2. At most one of α, β can derive ε.
3. If β ⇒* ε, then FIRST(α) ∩ FOLLOW(A) = ∅.

Check E' → +TE' | ε: FIRST(+TE') = {+} and FOLLOW(E') = {), $}, which are disjoint ✔. T' → *FT' | ε: {*} ∩ {+,), $} = ∅ ✔. F → id | (E): {id} ∩ {(} = ∅ ✔. So the grammar is LL(1).

Error example: id + * id

After id and + have been matched, the stack is $E'T and the input is `* id $`. M[T, *] is empty, so a syntax error is reported ("operand expected before *"). With panic-mode recovery (see 3.9), * is not in FOLLOW(T), so the entry is blank rather than synch, and the parser **skips the input symbol** *. It then continues with T → FT', F → id and reaches the end normally, having reported one error.

Answer structure for 10–15 marks: (1) remove left recursion, (2) FIRST and FOLLOW, (3) the parsing table, (4) the LL(1) conclusion, (5) the trace of moves, (6) the parse tree. All six are shown above.

3.7 Write a note on non-recursive descent parsing. / Explain the table-driven predictive parsing model.

5–6 M

Dec-25 Q5c Apr-25 Q6a

A **non-recursive predictive parser** (LL(1) parser) simulates a leftmost derivation. Instead of recursive procedure calls, it keeps an **explicit stack**. It is driven by a parsing table built from FIRST and FOLLOW.

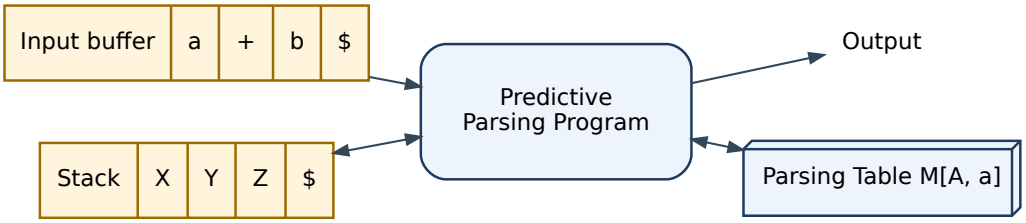


Fig. Model of a table-driven predictive parser

- **Input buffer:** the string to be parsed, followed by the end-marker $.
- **Stack:** grammar symbols, with $ at the bottom. At the start it holds $S (S on top).
- **Parsing table M[A, a]:** a 2-D array whose entries are productions or error.
- **Parsing program:** repeatedly looks at X (top of stack) and a (current input symbol).

```
push $ and then S onto the stack;  a = first input symbol;
while ( X = top of stack ) ≠ $ :
    if X == a            : pop X;  a = next input symbol        /* match */
    else if X is terminal: error()
    else if M[X, a] is error : error()
    else if M[X, a] = X → Y1 Y2 … Yk :
         output the production;  pop X;
         push Yk, …, Y2, Y1   (Y1 on top)
if a == $ : accept
```

Advantages: no recursion overhead, no backtracking, linear time O(n), and easy to generate automatically. **Limitation:** works only for LL(1) grammars, so left recursion must be removed and the grammar left-factored. (See 3.6 for a complete trace.)

Worked mini example

Grammar: S → aSb | c (FIRST(aSb) = {a}, FIRST(c) = {c}). Table: M[S, a] = S → aSb, M[S, c] = S → c.

Stack	Input	Action
$S	acb$	S → aSb (push b, S, a)
$bSa	acb$	match a
$bS	cb$	S → c
$bc	cb$	match c
$b	b$	match b
$	$	**accept**

Constructing the table (algorithm)

```
for each production A → α:
    for each terminal a in FIRST(α):  add A → α to M[A, a]
    if ε ∈ FIRST(α):
        for each b in FOLLOW(A):      add A → α to M[A, b]   ($ included)
every undefined entry = error
If some cell gets two productions, the grammar is NOT LL(1).
```

Recursive vs non-recursive predictive parsing

Recursive descent (predictive)	Non-recursive (table-driven)
One procedure per non-terminal; uses the system call stack implicitly	Explicit stack plus parsing table
Written by hand; grammar knowledge is in the code	Generic driver; grammar knowledge is in the table
Changing the grammar means rewriting code	Changing the grammar means rebuilding the table
Deep recursion can overflow the stack	Stack depth is under the parser's control

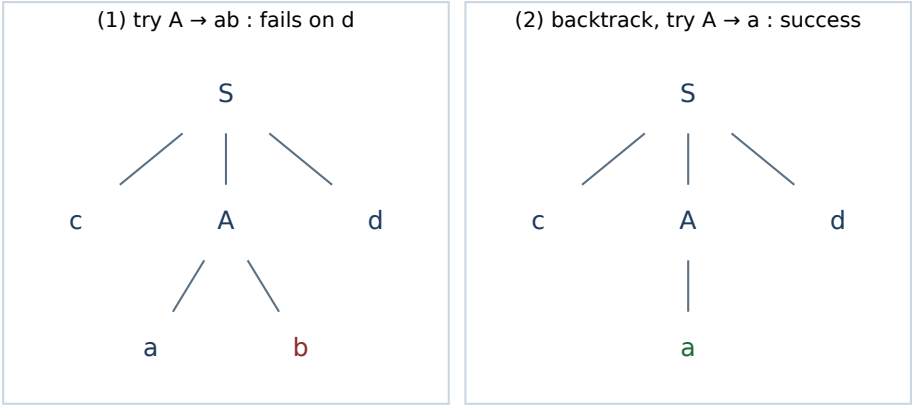
Meaning of "LL(1)": the input is scanned from **L**eft to right, a **L**eftmost derivation is produced, and **1** lookahead symbol is used.

3.8 What is backtracking in parsing? Why is it inefficient? 4 M

Apr-25 Q5c

Backtracking happens in recursive-descent parsing when the parser picks one alternative of a non-terminal, finds it does not match the input, and then *undoes* its work: it resets the input pointer and tries the next alternative.

Example: S → cAd, A → ab | a, input **cad**.



Why it is inefficient:

- In the worst case the running time is **exponential**, because the same input is re-scanned many times for different alternatives.
- Any semantic actions already performed (symbol-table entries, generated code) must be undone.
- Error reporting is poor: it is hard to tell where the real error is, because every failure looks like "try the next alternative".
- It cannot handle left-recursive grammars (infinite loop).

Predictive (LL(1)) and LR parsers avoid backtracking by using lookahead tokens and a parsing table.

Recursive descent with backtracking — code idea

```
bool S() { save = ip;
           if (match('c') && A() && match('d')) return true;
           ip = save; return false; }
bool A() { save = ip;
           if (match('a') && match('b')) return true;   /* try A → ab */
           ip = save;                                    /* BACKTRACK  */
           if (match('a')) return true;                  /* try A → a  */
           ip = save; return false; }
```

Trace on "cad"

1. S matches c; A tries ab: matches a, but expects b and sees d → fail.
2. The input pointer is reset to 'a' (backtrack). A tries a: match ✔.
3. S matches d ✔. Accept.

Ways to avoid backtracking

- **Left factoring:** A → aA', A' → b | ε, so the decision is delayed until it is clear.
- **Predictive parsing** with FIRST/FOLLOW sets and one lookahead (LL(1)).
- **Bottom-up LR parsing**, which postpones decisions until the whole handle is seen.

Backtracking parser	Predictive parser
Tries alternatives by trial and error	Chooses the right alternative from the lookahead
Exponential worst case	Linear time
Any non-left-recursive grammar	Only LL(1) grammars

3.9 Discuss error-recovery strategies in predictive parsing. 7 M

Aug-24 Q6c

An LL(1) parser detects an error when (a) the terminal on top of the stack does not match the input symbol, or (b) the entry M[A, a] for non-terminal A and input a is empty.

1. Panic-mode recovery

Skip input symbols until a token from a chosen **synchronizing set** appears. Heuristics for building the set of A:

- Put all symbols of **FOLLOW(A)** into the sync set. On finding one of them, **pop A** and continue.
- Add symbols of FIRST(A). On seeing one of them, resume parsing A.
- Add keywords that start statements (if, while, ;) to the sync sets of lower-level constructs.
- If a terminal on top of the stack does not match, pop it ("insert" the missing token) and report it.
- If A ⇒ ε, the ε-production can be used as a default.

Example (expression table of 3.6): mark M[A, b] = **synch** for every b ∈ FOLLOW(A) where the cell is empty, e.g. M[E,)] = M[E, $] = synch, M[F, +] = M[F, *] = synch. On input `) id * + id`: the ')' is skipped (error); at '+' with F on the stack, the entry is synch, so F is popped and parsing continues.

2. Phrase-level recovery

Fill the blank entries of the table with pointers to **error routines**. These may insert, delete or change input symbols, or pop the stack, and they issue a suitable message (e.g. "missing operand", "unbalanced parenthesis"). Care is needed to avoid infinite loops: every action should consume input or shrink the stack.

Parsing table with synch entries (expression grammar)

NT	id	+	*	(	)	$
E	E→TE'			E→TE'	synch	synch
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'	synch		T→FT'	synch	synch
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id	synch	synch	F→(E)	synch	synch

Rules: blank entry → skip the input symbol. synch → pop the non-terminal. Terminal mismatch → pop the terminal.

Trace on the erroneous input) id * + id

Stack	Input	Remark
$E	) id * + id $	error: M[E,)] = synch, but E is the start symbol, so skip)
$E	id * + id $	id ∈ FIRST(E): E → TE'
$E'T	id * + id $	T → FT'
$E'T'F	id * + id $	F → id
$E'T'id	id * + id $	match
$E'T'	* + id $	T' → *FT'
$E'T'F*	* + id $	match
$E'T'F	+ id $	**error: M[F, +] = synch → pop F** ("missing operand")
$E'T'	+ id $	T' → ε
$E'	+ id $	E' → +TE'
$E'T+	+ id $	match
$E'T	id $	T → FT'
$E'T'F	id $	F → id
$E'T'id	id $	match
$E'T'	$	T' → ε
$E'	$	E' → ε
$	$	finished (2 errors reported)

The parser reported both errors (the extra ')' and the missing operand after '*') and reached the end of the input. This is the goal of error recovery: find as many errors as possible in one run.

Summary: panic mode uses FOLLOW-based synch sets and is simple and safe. Phrase-level recovery uses custom error routines and gives more precise fixes, but must be designed carefully to avoid infinite loops.

UNIT IV

Bottom-Up Parsing, Syntax-Directed Translation & Yacc

Shift-reduce · handle pruning · LR(0) · CLR(1) · LALR(1) · SDD · type checking · Yacc

4.1 Explain bottom-up parsing in detail. Define augmented grammar and explain handle pruning. 6 M

Dec-25 Q8a Aug-24 Q8b

Bottom-up parsing builds the parse tree starting from the leaves (the input string) and works up to the root (the start symbol). At each step a substring that matches the right side of a production is **reduced** to the non-terminal on the left. Read in reverse, the sequence of reductions is a **rightmost derivation**.

Shift-reduce parsing is the general form. It uses a stack and an input buffer, both bottomed by $. There are four actions:

- **Shift** — move the next input symbol onto the stack.
- **Reduce** — the handle is on top of the stack; replace it by the left side of its production.
- **Accept** — the stack holds $S and the input is $.
- **Error** — call an error-recovery routine.

Handle: a substring that matches the right side of a production, and whose reduction is one step of the reverse rightmost derivation. Formally, if $S \Rightarrow^*_{rm} \alpha A w \Rightarrow_{rm} \alpha\beta w$, then $A \to \beta$ at the position after α is a handle of $\alpha\beta w$. **Handle pruning** means repeatedly finding the handle and replacing it with its left side, until only the start symbol is left. This produces the rightmost derivation in reverse.

Example ($E \to E+E \mid E*E \mid id$, input $id_1 * id_2$): the handles are id_1 ($E \to id$), then id_2 ($E \to id$), then $E*E$ ($E \to E*E$), giving E.

Augmented grammar: for a grammar G with start symbol S, G' is G plus a new start symbol S' and the production **S' → S**. It tells the parser exactly when to stop: the parser accepts only when it is about to reduce by S' → S. This avoids confusion when S also appears on the right side of other productions.

Conflicts: *shift/reduce* (the parser cannot decide whether to shift or reduce) and *reduce/reduce* (it cannot decide which production to use). LR parsers avoid them by using states and lookahead.

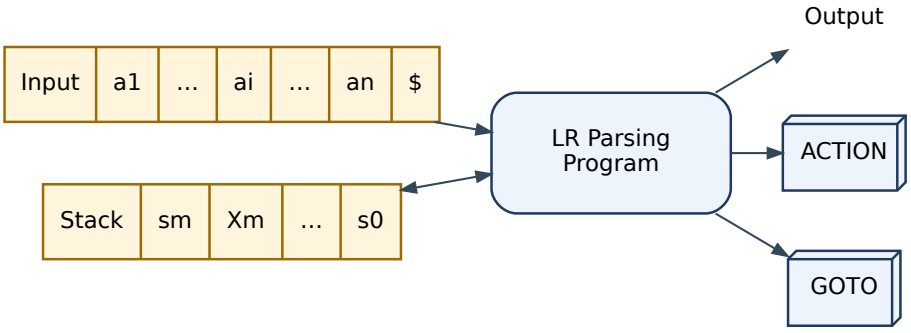
Shift-reduce trace — id * id

Grammar: E → E + T | T, T → T * F | F, F → id

Stack	Input	Action	Handle
$	id * id $	shift	
$id	* id $	reduce F → id	id
$F	* id $	reduce T → F	F
$T	* id $	shift (do *not* reduce E → T: the lookahead is *)	
$T *	id $	shift	
$T * id	$	reduce F → id	id
$T * F	$	reduce T → T * F	T * F
$T	$	reduce E → T	T
$E	$	**accept**	

Reading the reductions in reverse gives the rightmost derivation: E ⇒ T ⇒ T * F ⇒ T * id ⇒ F * id ⇒ id * id.

Model of an LR parser



The stack holds states $s_0\ X_1\ s_1 \ldots X_m\ s_m$. The state on top summarises the whole stack. ACTION[s_m, a_i] is one of shift s, reduce A → β, accept or error. GOTO[s, A] gives the state to enter after a reduction to A.

```
loop:  s = top state;  a = current input symbol
   if ACTION[s,a] = shift t : push a, t ; advance input
   else if ACTION[s,a] = reduce A → β : pop 2|β| symbols; t = top; push A, GOTO[t,A]; output
A → β
   else if ACTION[s,a] = accept : stop (success)
   else error()
```

Viable prefixes

The prefixes of right-sentential forms that can appear on the stack are called **viable prefixes**. They never extend past the right end of the handle. The set of viable prefixes is a **regular language**, which is why a DFA (the LR(0)/LR(1) item automaton) can recognise handles.

Conflicts — examples

- **Shift/reduce:** dangling else. On the stack *if E then S* with lookahead *else*, should the parser reduce to S or shift the else?

- **Reduce/reduce:** two productions with the same right side, e.g. a function call p(i) vs an array reference p(i). Which one to reduce?

Why LR parsers are preferred

- They recognise virtually all programming-language constructs that can be written as CFGs.
- They are the most general non-backtracking shift-reduce method.
- They detect syntax errors as soon as possible in a left-to-right scan.
- LR grammars are a proper superset of LL grammars.

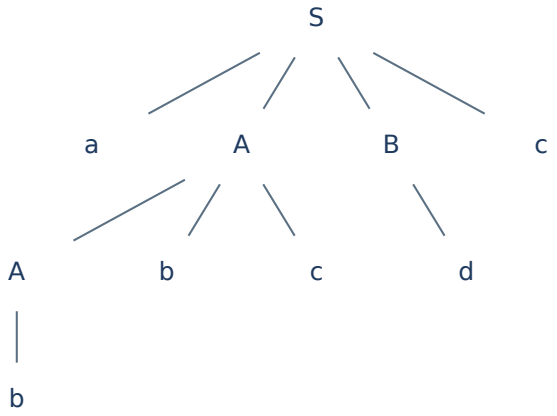
4.2 Trace the handle-pruning process while parsing the input "abbcdc" with a shift-reduce parser for S → aABc, A → Abc | b, B → d. List the handles found in each step and the final reduction sequence. 6 M

Apr-25 Q7c

#	Stack	Input	Action	Handle
1	$	abbcdc$	shift a	
2	$a	bbcdc$	shift b	
3	$ab	bcdc$	reduce A → b	**b**
4	$aA	bcdc$	shift b	
5	$aAb	cdc$	shift c	
6	$aAbc	dc$	reduce A → Abc	**Abc**
7	$aA	dc$	shift d	
8	$aAd	c$	reduce B → d	**d**
9	$aAB	c$	shift c	
10	$aABc	$	reduce S → aABc	**aABc**
11	$S	$	**accept**	

At step 5, b on top of the stack is *not* a handle. Reducing it to A would give aAA, which cannot lead back to S. The correct handle is Abc.

Handles in order: b, Abc, d, aABc
Reduction sequence: abbcdc ⇐ aAbcdc ⇐ aAdc ⇐ aABc ⇐ S
i.e. rightmost derivation: S ⇒ aABc ⇒ aAdc ⇒ aAbcdc ⇒ abbcdc



Parse tree for abbcdc, built bottom-up in the order b → A, Abc → A, d → B, aABc → S

Right-sentential forms and their handles

Right-sentential form	Handle	Reducing production
a b bcdc	b (at position 2)	A → b
a Abc dc	Abc	A → Abc
aA d c	d	B → d
aABc	aABc	S → aABc

Why not reduce the second b?

In "aAbcdc" the leftmost substring matching a right side is b (A → b). Reducing it would give aAAcdc. No right-sentential form of this grammar contains AA, so that reduction can never lead to S: it is not a handle. The handle is the substring whose reduction is one step of the **reverse rightmost derivation**, here Abc. An LR parser makes this choice correctly because its state shows that "aAb" is the prefix of the handle Abc.

Viable prefixes seen on the stack

a, ab, aA, aAb, aAbc, aA, aAd, aAB, aABc, S. Each one is a prefix of a right-sentential form that does not go past the handle.

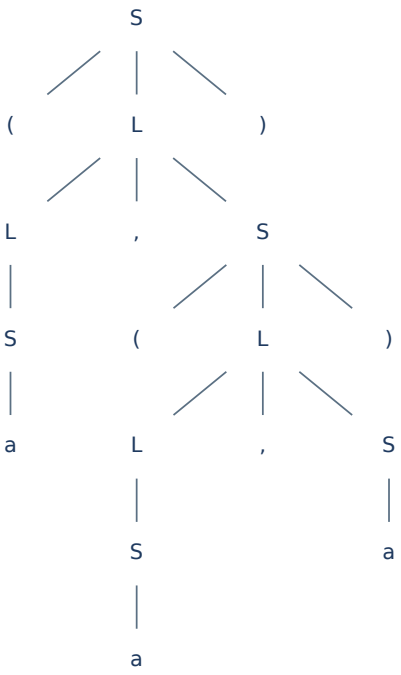
> **Handle pruning in one line:** find the handle, reduce it, and repeat until the start symbol is left. The handles found here, in order, are b, Abc, d, aABc.

4.3 Perform shift-reduce parsing for the input string (a,(a,a)) using S → (L) | a, L → L,S | S.

5 M

Aug-24 Q5c

Stack	Input	Action
$	(a,(a,a))$	shift (
$(	a,(a,a))$	shift a
$(a	,(a,a))$	reduce S → a
$(S	,(a,a))$	reduce L → S
$(L	,(a,a))$	shift ,
$(L,	(a,a))$	shift (
$(L,(	a,a))$	shift a
$(L,(a	,a))$	reduce S → a
$(L,(S	,a))$	reduce L → S
$(L,(L	,a))$	shift ,
$(L,(L,	a))$	shift a
$(L,(L,a	))$	reduce S → a
$(L,(L,S	))$	reduce L → L,S
$(L,(L	))$	shift)
$(L,(L)	)$	reduce S → (L)
$(L,S	)$	reduce L → L,S
$(L	)$	shift)
$(L)	$	reduce S → (L)
$S	$	**accept**



Parse tree for (a,(a,a))

Rightmost derivation (the trace above, read in reverse)

```
S ⇒ (L) ⇒ (L,S) ⇒ (L,(L)) ⇒ (L,(L,S)) ⇒ (L,(L,a)) ⇒ (L,(S,a))
  ⇒ (L,(a,a)) ⇒ (S,(a,a)) ⇒ (a,(a,a))
```

Handles found during the parse, in order

a (S→a), S (L→S), a (S→a), S (L→S), a (S→a), L,S (L→L,S), (L) (S→(L)), L,S (L→L,S), (L) (S→(L)).

Decisions explained

- After the stack becomes `$(L` with lookahead ',', the parser shifts. It does not reduce, because no production ends in "(L" without ')'.
- `$(L,(L,S` with lookahead ')': the handle is L,S, so reduce by L → L,S. This is left recursion handled naturally by a bottom-up parser.
- The parser accepts when the stack is `$S` and the input is `$`.

Language: nested, parenthesised lists of a's, such as (a), (a,a), ((a),a).

4.4 Differentiate between shift-reduce and operator-precedence parsers. 5 M

Aug-24 Q6a

Shift-reduce (LR family)	Operator-precedence parser
General bottom-up technique; works for a large class of CFGs (LR grammars)	A special shift-reduce parser that works only for **operator grammars** (no ε-productions, no two adjacent non-terminals)
Decisions come from parser states plus a lookahead (ACTION/GOTO tables)	Decisions come from precedence relations <·, ≐, ·> between terminals
Finds handles exactly using the DFA of viable prefixes	Handle = the string between <· and ·>; non-terminals are ignored when comparing
Detects errors as soon as possible; precise	Error detection is weaker; may accept some invalid strings
Hard to build by hand; generated by tools (Yacc)	Easy to build by hand for expressions
Can handle unary and binary uses of the same operator, using grammar states	Cannot easily handle an operator with two precedences (e.g. unary vs binary minus)

Operator-precedence relations — example

Grammar E → E+E | E*E | id, with * above + and both left-associative:

	id	+	*	$
id		·>	·>	·>
+	<·	·>	<·	·>
*****	<·	·>	·>	·>
$	<·	<·	<·	accept

Inserting the relations into $ id + id * id $ gives **$ <· id ·> + <· id ·> * <· id ·> $**. Each handle lies between <· and ·>. Each id is reduced first, then the * part (because + <· *), and finally the + part. So the precedence of * over + is respected.

Operator-precedence parsing algorithm

```
let a = top terminal on the stack, b = current input
if a = $ and b = $ : accept
if a <· b or a ≐ b : shift b
if a ·> b : pop until the terminal on top is related by <· to the last popped terminal
(reduce)
else error
```

Shift-reduce (LR) trace for comparison

An LR parser on the same input uses states, not precedence relations: s(id) r(F→id) r(T→F) r(E→T) s(+) s(id) r r s(*) s(id) r(F→id) r(T→T*F) r(E→E+T) accept.

Advantages and disadvantages of operator precedence

- ✔ Simple, small tables; easy to build by hand for expressions.

- ✘ Applies only to a small class of grammars; unary minus is hard to handle; the parser may accept some incorrect strings; there is no direct link between the grammar and the parser.

Precedence functions f and g can replace the n × n table with two vectors of size n, which saves space.

4.5 Construct the LR(1) / CLR(1) parsing table for S → AA, A → aA | b. (Also: S → CC, C → aC | d.)

7–10 M

Dec-25 Q6b, Q8b Apr-25 Q8b Aug-24 Q8a

Augmented grammar: (0) S' → S (1) S → AA (2) A → aA (3) A → b

An LR(1) item is [A → α•β, a], where a is the **lookahead**. **CLOSURE:** for [A → α•Bβ, a], add [B → •γ, b] for every b ∈ FIRST(βa). **GOTO(I, X)** = CLOSURE of the items with the dot moved past X.

I0 = CLOSURE([S' → •S, $]). For S → •AA, the lookahead of A is FIRST(A$) = {a, b}.

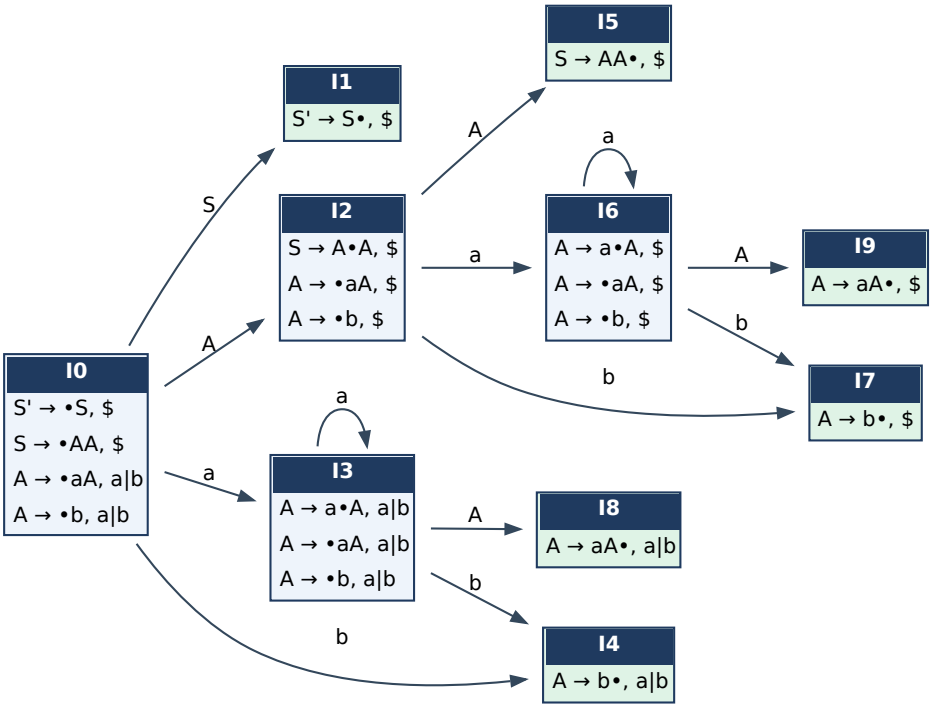


Fig. Canonical collection of LR(1) items (green = states containing a completed item)

CLR(1) parsing table

Rules: if [A → α•aβ, b] ∈ I_i and GOTO(I_i, a) = I_j, set ACTION[i, a] = shift j. If [A → α•, a] ∈ I_i, set ACTION[i, a] = reduce A → α (only on its lookahead a). If [S' → S•, $] ∈ I_i, set ACTION[i, $] = accept.

State	ACTION			GOTO	
	a	b	$	S	A
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

There are no multiply-defined entries, so the grammar is **CLR(1) / LR(1)**. (LR(1) and CLR(1) are the same parser.)

Parsing "aabb" (to check the table)

Stack	Input	Action
0	aabb$	s3
0 a 3	abb$	s3
0 a 3 a 3	bb$	s4
0 a 3 a 3 b 4	b$	r3 (A→b), goto(3,A)=8
0 a 3 a 3 A 8	b$	r2 (A→aA), goto(3,A)=8
0 a 3 A 8	b$	r2, goto(0,A)=2
0 A 2	b$	s7
0 A 2 b 7	$	r3, goto(2,A)=5
0 A 2 A 5	$	r1 (S→AA), goto(0,S)=1
0 S 1	$	accept

Aug-24 version (S → CC, C → aC | d): this is the same grammar with A renamed C and b renamed d. Use the same item sets and the same table, with column **b → d** and GOTO column **A → C**. Productions: (1) S → CC, (2) C → aC, (3) C → d. For example, I4 = [C → d•, a|d] gives r3 on a and d, and I7 = [C → d•, $] gives r3 on $.

Computing the closures in detail

- **I0:** start with [S' → •S, $]. The dot is before S, so add S-items with lookahead FIRST(ε$) = {$}: [S → •AA, $]. The dot is before the first A, followed by A$, so the lookaheads are FIRST(A$) = {a, b}: add [A → •aA, a|b] and [A → •b, a|b].

- **I2 = GOTO(I0, A):** the kernel is [S → A•A, $]. The dot is before A with nothing after it, so the lookahead is FIRST($) = {$}: add [A → •aA, $] and [A → •b, $].
- **I3 = GOTO(I0, a):** the kernel is [A → a•A, a|b]. Its closure adds the A-items with the same lookaheads a|b.
- **I6 = GOTO(I2, a):** the kernel is [A → a•A, $]. This has the same core as I3 but a different lookahead, so CLR keeps it as a separate state.

Table-construction algorithm (canonical LR)

1. Build C = {I0, I1, …, In}, the canonical collection of LR(1) items.
2. If [A → α•aβ, b] ∈ I_i and GOTO(I_i, a) = I_j, then ACTION[i, a] = shift j.
3. If [A → α•, a] ∈ I_i (A ≠ S'), then ACTION[i, a] = reduce A → α.
4. If [S' → S•, $] ∈ I_i, then ACTION[i, $] = accept.
5. GOTO[i, A] = j if GOTO(I_i, A) = I_j. All other entries are error.

Error detection — input "ab"

Stack	Input	Action
0	ab$	s3
0 a 3	b$	s4
0 a 3 b 4	$	**ACTION[4, $] is blank → error**

CLR detects the error **immediately**, without performing a wrong reduction. Compare this with LALR in 4.6.

CLR table written out for the Aug-24 grammar (S → CC, C → aC | d)

State	ACTION			GOTO	
	a	d	$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Productions: (1) S → CC, (2) C → aC, (3) C → d. Item sets: I0 = {[S'→•S,$], [S→•CC,$], [C→•aC,a|d], [C→•d,a|d]}, I3 = {[C→a•C,a|d], …}, I4 = {[C→d•,a|d]}, I6 = {[C→a•C,$], …}, I7 = {[C→d•,$]}, I8 = {[C→aC•,a|d]}, I9 = {[C→aC•,$]}.

Language: a*b a*b (or a*d a*d). LR(1) items carry lookahead, so reductions are placed only on the symbols that can really follow. This makes CLR the most powerful LR method.

4.6 Construct the LALR(1) parsing table for S → AA, A → aA, A → b. 8 M

Dec-25 Q7a Apr-25 Q7a

Method: build the canonical LR(1) collection (see 4.5). Then **merge states that have the same core** (the same LR(0) items, differing only in lookaheads), taking the union of their lookaheads.

Same core	Merged state	Items after merging
I3 and I6	**I36**	A → a•A, a|b|$; A → •aA, a|b|$; A → •b, a|b|$
I4 and I7	**I47**	A → b•, a|b|$
I8 and I9	**I89**	A → aA•, a|b|$

Unchanged: I0, I1, I2, I5. That gives 7 states in total (the same number as SLR/LR(0)).

State	ACTION			GOTO	
	a	b	$	S	A
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

No conflicts appear after merging (merging can only create reduce/reduce conflicts, and here there are none). So the grammar is **LALR(1)**. The LALR parser detects errors a little later than CLR, but it has far fewer states.

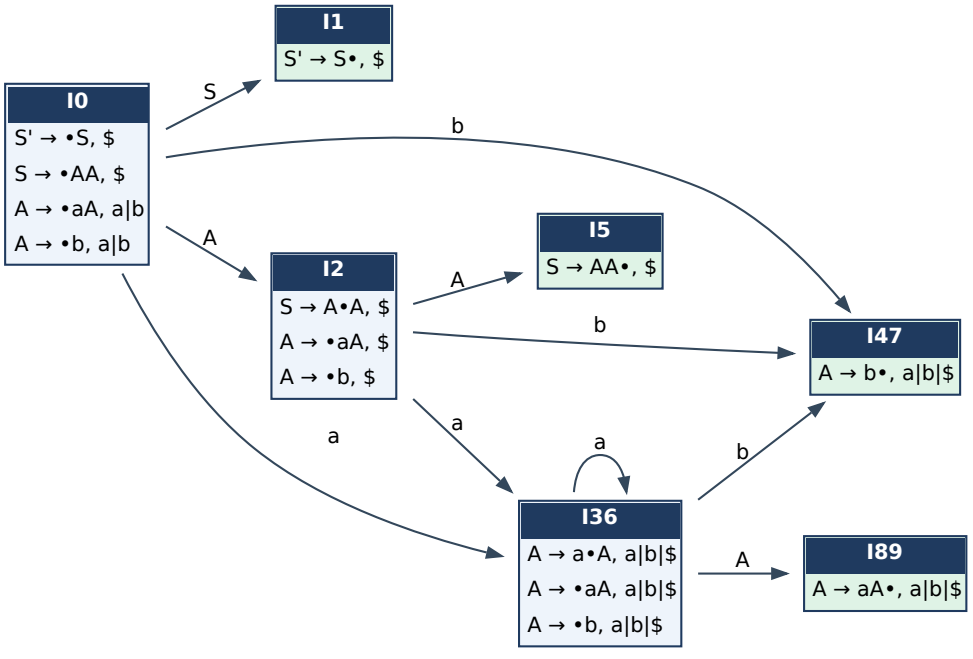


Fig. LALR(1) automaton after merging (7 states)

LALR construction algorithm

1. Construct the collection of LR(1) item sets.
2. Find all sets with the same core (the first components of the items) and replace each group by its union.
3. GOTO of a merged set is the union of the GOTOs (the cores are equal, so the targets also have equal cores).
4. Build the ACTION/GOTO table as for CLR. If a conflict appears, the grammar is not LALR(1).

Why merging can create only reduce/reduce conflicts

A shift action depends only on the core. If a merged state had a shift/reduce conflict, one of the original states would already have had it, so the conflict would already exist in CLR. New reduce/reduce conflicts can arise, because the lookaheads of different items are combined.

Delayed error detection — input "ab" (compare with CLR)

Stack	Input	Action
0	ab$	s36
0 a 36	b$	s47
0 a 36 b 47	$	r3 (A → b) — CLR would report an error here
0 a 36 A 89	$	r2 (A → aA)
0 A 2	$	**ACTION[2, $] blank → error**

LALR performs **two extra reductions** before it detects the error, but it never shifts an erroneous symbol. The error is still caught before any more input is consumed.

Parse of a valid string — "baab"

0 →b s47 → r3, GOTO(0,A) = 2 → a s36 → a s36 → b s47 → r3, GOTO(36,A) = 89 → r2 → 89 → r2, GOTO(2,A) = 5 → r1, GOTO(0,S) = 1 → accept.

Method	States for this grammar
LR(0) / SLR	7
CLR(1)	10
LALR(1)	7

Why LALR is used in practice: its tables are as small as SLR's, while it is almost as powerful as CLR. That is why Yacc and Bison generate LALR(1) parsers. For a typical language (C, Pascal), SLR/LALR need a few hundred states, while CLR needs thousands.

4.7 Construct the LR(0) parsing table for E → E*B | E+B | B, B → 0 | 1. — 8 M

Apr-25 Q8c

Augmented grammar: (0) S → E (1) E → E*B (2) E → E+B (3) E → B (4) B → 0 (5) B → 1

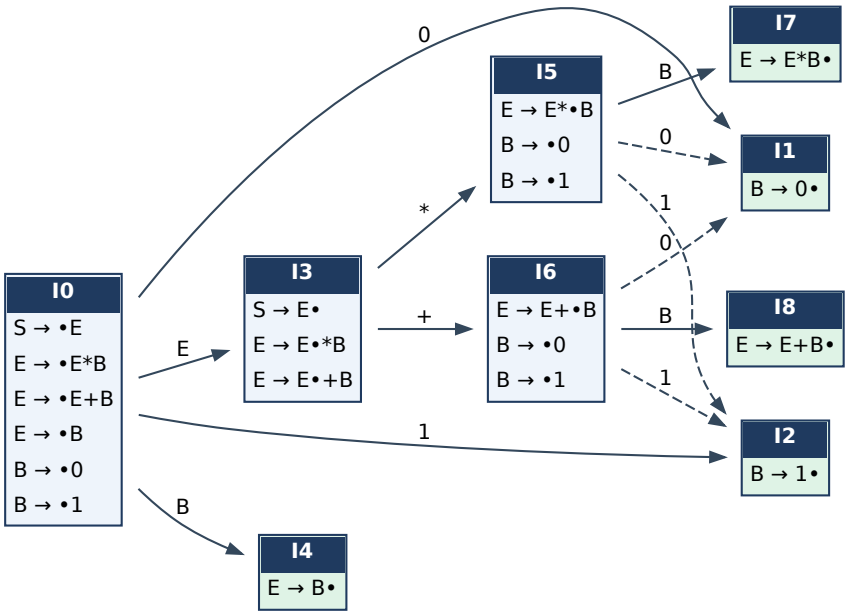


Fig. Canonical collection of LR(0) items and the GOTO graph

LR(0) rule: if a state contains a completed item A → α•, put the reduce action in **every** terminal column. There is no lookahead.

State	ACTION					GOTO	
	*	+	0	1	$	E	B
0			s1	s2		3	4
1	r4	r4	r4	r4	r4		
2	r5	r5	r5	r5	r5		
3	s5	s6			**acc**		
4	r3	r3	r3	r3	r3		
5			s1	s2			7
6			s1	s2			8
7	r1	r1	r1	r1	r1		
8	r2	r2	r2	r2	r2		

No state has both a shift and a reduce, or two reduces, so the grammar is **LR(0)**. Example: "1+1" → s2, r5 (B→1), goto 4, r3 (E→B), goto 3, s6, s2, r5, goto 8, r2 (E→E+B), goto 3, accept.

Building the item sets — closure and GOTO

- **CLOSURE(I):** for every item A → α•Bβ in I, add B → •γ for each B-production, and repeat.
- **GOTO(I, X):** the closure of { A → αX•β | A → α•Xβ ∈ I }.
- I0 = CLOSURE({S → •E}): since the dot is before E, add the three E-items. The dot is then before B in E → •B, so add B → •0 and B → •1.
- GOTO(I3, *) = I5 = {E → E*•B} plus its closure {B → •0, B → •1}.

Full parse of "1+1"

Stack	Input	Action
0	1+1$	s2
0 1 2	+1$	r5 (B→1), GOTO(0,B) = 4
0 B 4	+1$	r3 (E→B), GOTO(0,E) = 3
0 E 3	+1$	s6
0 E 3 + 6	1$	s2
0 E 3 + 6 1 2	$	r5, GOTO(6,B) = 8
0 E 3 + 6 B 8	$	r2 (E→E+B), GOTO(0,E) = 3
0 E 3	$	**accept**

Why the grammar is LR(0)

Every state that contains a completed item (I1, I2, I4, I7, I8) contains **only** that item, so there is no shift/reduce or reduce/reduce conflict. I3 contains S → E• together with shift items, but S → E• means accept on $, and the shifts are on * and +, so they do not clash.

SLR(1) version (for comparison)

FOLLOW(E) = FOLLOW(B) = {*, +, $}. In the SLR table, the reduce entries r1–r5 appear only in the *, + and $ columns, not under 0 or 1. That gives fewer entries and earlier error detection. For example, "11" is caught at once in SLR, while LR(0) first reduces B → 1.

Language: binary digits separated by * or +, e.g. 1, 0+1, 1*0+1. The operators are left-associative because of the left recursion.

4.8 Differences between SLR, CLR and LALR parsers. 4 M

Aug-24 Q8c

Feature	SLR(1)	CLR(1) / LR(1)	LALR(1)
Items used	LR(0) items	LR(1) items (core + lookahead)	LR(1) items with the same core merged
Reduce placed on	FOLLOW(A)	the exact lookahead of the item	the union of the merged lookaheads
Number of states	Small	Largest	Same as SLR
Power	Least	Most powerful	In between (SLR ⊂ LALR ⊂ CLR)
Conflicts	Most S/R conflicts	Fewest	May get R/R conflicts from merging
Error detection	Delayed	Immediate	May do a few extra reductions first
Used by	Teaching, simple tools	Rarely (table too large)	Yacc, Bison (practical choice)

Item types

- **LR(0) item:** A → α•β (just the position of the dot).
- **LR(1) item:** [A → α•β, a], with one lookahead terminal.

A grammar that is LALR but not SLR

```
S → L = R | R      L → * R | id      R → L
```

In SLR, the state containing {S → L•=R, R → L•} has a shift/reduce conflict on '=', because '=' ∈ FOLLOW(R). In LALR, the lookahead of R → L• in that state is only $, so there is no conflict. This shows that **SLR ⊂ LALR**.

A grammar that is CLR but not LALR

```
S → aAd | bBd | aBe | bAe      A → c      B → c
```

CLR has the states {[A→c•, d], [B→c•, e]} and {[A→c•, e], [B→c•, d]}. They have the same core. Merging them gives a reduce/reduce conflict on both d and e. So **LALR ⊂ CLR**.

Hierarchy

`LR(0) ⊂ SLR(1) ⊂ LALR(1) ⊂ LR(1) = CLR(1); also LL(1) ⊂ LR(1)`

Ease of construction	Table size	Power
SLR easiest → LALR → CLR hardest	SLR = LALR ≪ CLR	SLR < LALR < CLR

4.9 Explain Syntax-Directed Definitions (SDD) with an example. Differentiate between synthesized and inherited attributes. / Define syntax-directed translation with an example.

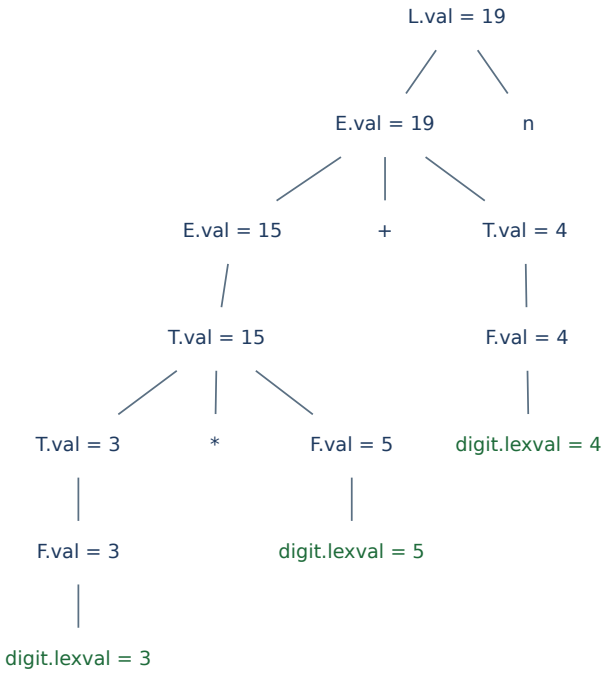
4-6 M

Dec-25 Q7b Apr-25 Q8a Aug-24 Q7b

A **Syntax-Directed Definition (SDD)** is a context-free grammar together with **attributes** attached to grammar symbols and **semantic rules** attached to productions. A rule computes the value of an attribute, e.g. E.val = E₁.val + T.val. A **Syntax-Directed Translation (SDT) scheme** instead embeds program fragments called *semantic actions* inside the production bodies, e.g. E → E₁ + T { print('+') }. An SDT also fixes the order in which the actions run.

Example — SDD of a desk calculator (S-attributed)

Production	Semantic rule
L → E n	L.val = E.val
E → E₁ + T	E.val = E₁.val + T.val
E → T	E.val = T.val
T → T₁ * F	T.val = T₁.val × F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

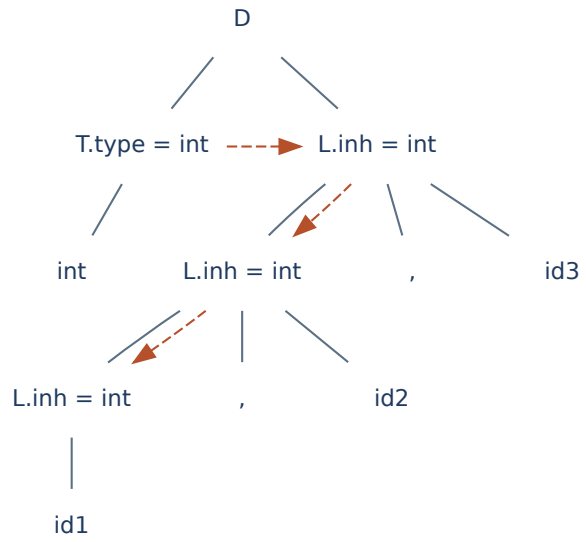


Annotated parse tree for 3 * 5 + 4 n

Synthesized vs inherited attributes

Synthesized attribute	Inherited attribute
Value at node N is computed from the attributes of N's **children** (and N itself)	Value at node N is computed from N's **parent**, its **siblings** and N itself
Information flows **bottom-up**	Information flows **top-down / sideways**
Defined by a rule of the production whose head is N	Defined by a rule of the production whose *body* contains N
Evaluated naturally during bottom-up (LR) parsing	Needs a dependency-graph order, e.g. top-down
An SDD with only these is **S-attributed**	Allowed in **L-attributed** SDDs (from the left only)
e.g. E.val, T.val above	e.g. L.inh in D → T L, rule L.inh = T.type

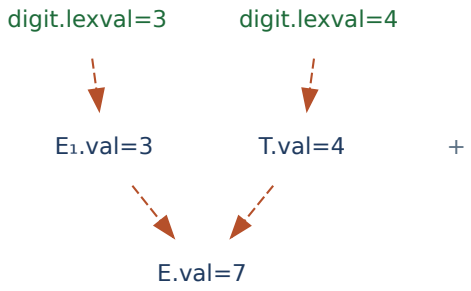
Production	Rule (type declaration)
D → T L	L.inh = T.type
T → int	T.type = integer
T → float	T.type = float
L → L₁ , id	L₁.inh = L.inh; addType(id.entry, L.inh)
L → id	addType(id.entry, L.inh)



int id1, id2, id3 — the type is passed down (inherited)

Dependency graph and evaluation order

A **dependency graph** has one node per attribute instance and an edge b → c whenever c's rule uses b. Attributes must be evaluated in a **topological order** of this graph. If the graph has a cycle, the attributes cannot be evaluated.



For 3 + 4: digit.lexval → E₁.val, digit.lexval → T.val, then E₁.val and T.val → E.val. One valid order is 3, 4, E₁, T, E.

S-attributed and L-attributed definitions

S-attributed SDD	L-attributed SDD
Uses only synthesized attributes	Synthesized attributes, plus inherited attributes that depend only on the parent's inherited attributes and on siblings to the **left**
Evaluated bottom-up (post-order); fits LR parsing	Evaluated in one depth-first, left-to-right pass; fits LL parsing
e.g. desk calculator	e.g. type declarations, T' → *FT'₁ with T'₁.inh = T'.inh × F.val

SDT example — infix to postfix

```
E → E₁ + T   { print('+') }
E → E₁ - T   { print('-') }
E → T
T → 0 { print('0') } | 1 { print('1') } | … | 9 { print('9') }
```

For **9 − 5 + 2**, the actions run in post-order and print **9 5 − 2 +**. The actions are placed at the end of the productions, so this is a "postfix SDT" that an LR parser can run on each reduction.

SDD vs SDT

SDD	SDT scheme
High-level specification: attributes plus rules	Implementation-oriented: actions embedded in productions
Evaluation order not specified (dependency graph)	The position of each action fixes when it runs
E → E₁ + T E.val = E₁.val + T.val	E → E₁ + T { E.val = E₁.val + T.val }

4.10 What are the applications of SDD? 6 M

Dec-25 Q8c

1. **Construction of syntax trees:** SDDs with rules such as E.node = new Node('+', E₁.node, T.node) build abstract syntax trees during parsing.
2. **Type checking:** attributes carry type information (E.type) and rules check compatibility, e.g. if E₁.type = int and E₂.type = int then E.type = int, else type_error.
3. **Intermediate code generation:** attributes such as E.addr and E.code generate three-address code, e.g. E.code = E₁.code ‖ E₂.code ‖ gen(E.addr '=' E₁.addr '+' E₂.addr).
4. **Evaluating expressions:** desk calculators (the val attribute), constant folding.
5. **Symbol-table management and declarations:** inherited attributes pass the type to a list of identifiers (D → T L), and the offset or width of variables is computed for storage layout.
6. **Translating control flow:** the attributes B.true, B.false and S.next generate jumps for if, while and Boolean expressions (backpatching).
7. **Infix to postfix translation:** e.g. E → E₁ + T { print('+') }.
8. **Other uses:** pretty printers, document formatting, and generating code for array references.

Application 1 in detail — building a syntax tree for a − 4 + c

Production	Semantic rule
E → E₁ + T	E.node = new Node('+', E₁.node, T.node)
E → E₁ − T	E.node = new Node('−', E₁.node, T.node)
E → T	E.node = T.node
T → id	T.node = new Leaf(id, id.entry)
T → num	T.node = new Leaf(num, num.val)

```
p1 = new Leaf(id, entry-a);  p2 = new Leaf(num, 4);  p3 = new Node('-', p1, p2);
p4 = new Leaf(id, entry-c);  p5 = new Node('+', p3, p4)          /* root */
```

Application 2 in detail — three-address code generation

Production	Semantic rules
S → id = E	S.code = E.code ‖ gen(top.get(id.lexeme) '=' E.addr)
E → E₁ + E₂	E.addr = new Temp(); E.code = E₁.code ‖ E₂.code ‖ gen(E.addr '=' E₁.addr '+' E₂.addr)
E → − E₁	E.addr = new Temp(); E.code = E₁.code ‖ gen(E.addr '=' 'minus' E₁.addr)
E → (E₁)	E.addr = E₁.addr; E.code = E₁.code
E → id	E.addr = top.get(id.lexeme); E.code = ''

For **a = b + −c** this produces t1 = minus c; t2 = b + t1; a = t2.

Application 3 in detail — Boolean expressions and control flow

S → if (B) S₁: B.true = newlabel(); B.false = S₁.next = S.next; S.code = B.code ‖ label(B.true) ‖ S₁.code. The labels are passed as inherited attributes.

Summary: SDDs give a single, clear way to attach meaning to syntax. That is why tools such as Yacc are built around them, and why most front-end tasks (trees, types, IR, error checks) are specified with them.

4.11 Write a note on Yacc. / Explain the working of the Yacc parser generator in detail.

6–8 M

Dec-25 Q7c Apr-25 Q7b Aug-24 Q7a

Yacc ("Yet Another Compiler-Compiler", S.C. Johnson; GNU version Bison) is an **LALR(1) parser generator**. It reads a grammar specification with semantic actions and produces a C program, `y.tab.c`, containing the function `yyparse()` and the LALR parsing tables.

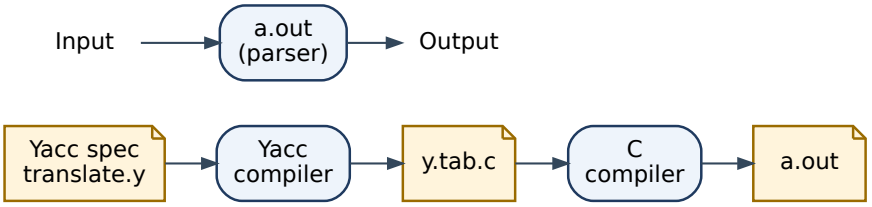


Fig. Creating an input/output translator with Yacc

Structure of a Yacc program

```
declarations
%%
translation rules
%%
supporting C routines
```

- **Declarations:** C code in `%{ … %}`; token declarations `%token DIGIT`; precedence and associativity (`%left '+' '-'`, `%right UMINUS`); `%start`.
- **Rules:** `head : body₁ {action} | body₂ {action} ;` where `$$` is the value of the head and `$1`, `$2`, … are the values of the body symbols.
- **Routines:** `main()`, `yylex()` (often generated by Lex), `yyerror()`.

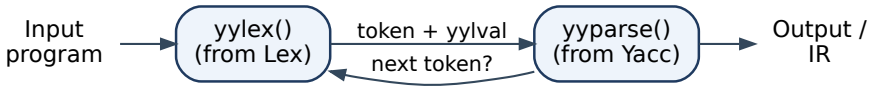
```
%{
#include <ctype.h>
%}
%token DIGIT
%left '+'
%left '*'
%%
line : expr '\n'        { printf("%d\n", $1); } ;
expr : expr '+' expr    { $$ = $1 + $3; }
     | expr '*' expr    { $$ = $1 * $3; }
     | '(' expr ')'     { $$ = $2; }
     | DIGIT ;
%%
yylex() { int c = getchar();
          if (isdigit(c)) { yylval = c - '0'; return DIGIT; }
          return c; }
```

Working

1. Yacc builds the LALR(1) item sets and the ACTION/GOTO tables from the rules.
2. `yyparse()` calls `yylex()` for tokens. Lex returns the token code, and its value goes in `yylval`.
3. The parser shifts tokens onto a state stack and a parallel value stack. On a reduce, it runs the rule's action, computing `$$` from `$1…$n`.
4. **Conflict resolution:** for a shift/reduce conflict, the default is to shift; for a reduce/reduce conflict, the earlier rule wins. The `%left/%right` declarations resolve conflicts in ambiguous grammars. `yacc -v` writes the conflicts to y.output.
5. **Error recovery:** the special `error` token, e.g. `stmt : error ';'`, together with `yyerror()`.

Commands: `lex calc.l` → lex.yy.c; `yacc -d calc.y` → y.tab.c, y.tab.h; `cc y.tab.c lex.yy.c -ly -ll` → a.out.

Lex and Yacc working together




```
/* calc.l */
%{
#include "y.tab.h"
extern int yylval;
%}
%%
[0-9]+   { yylval = atoi(yytext); return NUMBER; }
[ \t]    ;
\n       { return '\n'; }
.        { return yytext[0]; }
%%
```

```
/* calc.y */
%token NUMBER
%left '+' '-'
%left '*' '/'
%right UMINUS
%%
line : expr '\n'            { printf("= %d\n", $1); }
expr : expr '+' expr        { $$ = $1 + $3; }
     | expr '-' expr        { $$ = $1 - $3; }
     | expr '*' expr        { $$ = $1 * $3; }
     | expr '/' expr        { $$ = $1 / $3; }
     | '-' expr %prec UMINUS { $$ = -$2; }
     | '(' expr ')'         { $$ = $2; }
     | NUMBER ;
%%
int main() { return yyparse(); }
int yyerror(char *s) { fprintf(stderr, "%s\n", s); return 0; }
```

Run: input `2+3*4` → output `= 14`. The grammar is ambiguous, but `%left` makes * bind tighter than + and resolves the shift/reduce conflicts.

How Yacc resolves conflicts using precedence

- On a shift/reduce conflict between the rule `expr '+' expr` and the lookahead '*': * has higher precedence, so Yacc **shifts**.
- Rule `expr '+' expr` with lookahead '+': equal precedence and %left, so Yacc **reduces** (left associativity).
- `%prec UMINUS` gives that rule the precedence of UMINUS.

Files and options

File / option	Purpose
y.tab.c	Parser source (yyparse)
y.tab.h (-d)	Token codes shared with Lex
y.output (-v)	Item sets, parsing table and conflict report

4.12 Discuss type checking with suitable examples. 8 M

Aug-24 Q7c

Type checking is the part of semantic analysis that makes sure the type of every construct matches what its context expects, according to the language's **type system** (a set of rules assigning type expressions to program parts). Example: an arithmetic operator applied to an integer and a function is an error.

Static vs dynamic checking

- **Static:** done at compile time (C, Java). It catches errors early and adds no runtime cost.
- **Dynamic:** done at run time (Python, array-bound checks). It is more flexible but slower.
- A **strongly typed** language guarantees that accepted programs run without type errors.

Type expressions

Basic types: int, float, char, boolean, void, type_error. Constructors: array(I, T), pointer(T), product $T_1 \times T_2$, function $T_1 \to T_2$, record(…). For example, `int a[10]` has type array(0..9, int), and `float f(int, char)` has type int × char → float.

Type-checking rules (as an SDD)

Production	Semantic rule
E → num	E.type = integer
E → id	E.type = lookup(id.entry)
$E \to E_1 + E_2$	if E_1.type = int and E_2.type = int then int; else if either is float then float (coerce); else type_error
$E \to E_1 [E_2]$	if E_2.type = int and E_1.type = array(s, t) then t else type_error
$E \to *E_1$	if E_1.type = pointer(t) then t else type_error
$E \to E_1 (E_2)$	if E_1.type = s → t and E_2.type = s then t else type_error
S → id = E	if id.type = E.type then void else type_error
S → if E then S_1 / while E do S_1	if E.type = boolean then S_1.type else type_error

Key concepts

- **Type conversion / coercion:** in x = i * 2.5 with i an int, the compiler inserts inttofloat(i). This is an implicit (widening) conversion. An explicit conversion is a cast: (int) f.
- **Type equivalence:** *structural* (same structure) vs *name* equivalence (same type name).
- **Overloading:** '+' means integer addition, float addition or string concatenation depending on the operand types. It is resolved from the types.
- **Polymorphism and type inference:** e.g. a generic length: list(α) → int, where the type is inferred using unification.

Example of errors caught: `int x; char *p; x = p + 1.5;` → error: pointer + float is invalid; `if (x) …` where x is a struct → error: the condition is not a boolean or scalar.

Annotated example — type checking x = i * 2.5 + a[j]

Declarations: `int i, j; float x; float a[10];`

Sub-expression	Rule applied	Type
i	lookup	int
2.5	literal	float
i * 2.5	int × float: coerce i with inttofloat	float
a[j]	a : array(0..9, float), j : int	float
i*2.5 + a[j]	float + float	float
x = …	float = float	void (OK)

The intermediate code includes the inserted conversion: t1 = inttofloat(i); t2 = t1 * 2.5; t3 = j * 4; t4 = a[t3]; t5 = t2 + t4; x = t5.

Type checking of functions and statements

```
P → D ; S
D → D ; D | id : T                { addtype(id.entry, T.type) }
T → int | float | array [num] of T₁ | ↑T₁ | T₁ → T₂
S → id := E                       { S.type = (id.type == E.type) ? void : type_error }
S → while E do S₁                 { S.type = (E.type == boolean) ? S₁.type : type_error }
E → E₁ ( E₂ )                     { E.type = (E₂.type == s && E₁.type == s → t) ? t :
type_error }
```

Sound type systems

A type system is **sound** if a program that passes static checking can never cause a type error at run time. With a sound system there is no need for run-time checks. Java is mostly sound. C is not sound, because casts and unions allow arbitrary reinterpretation.

Error recovery in the type checker

Give an erroneous expression the type **type_error**, report it once, and treat type_error as compatible with everything afterwards. This avoids a cascade of messages caused by the same mistake.

UNIT V

Intermediate Code, Run-Time Storage & Optimization

Syntax trees & DAGs · three-address code · quadruples/triples · basic blocks & flow graphs · storage organization · optimization

5.1 Explain the different types of intermediate code with an example. / Briefly explain the variants of syntax trees.

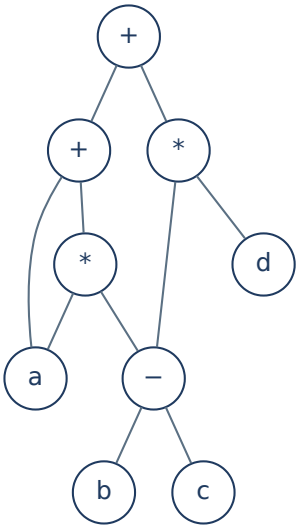
7–8 M

Aug-24 Q10a Apr-25 Q9a

The **intermediate representation (IR)** sits between the front end and the back end. It makes retargeting easy and allows machine-independent optimization. Common forms:

1. **Syntax tree (abstract syntax tree):** each interior node is an operator and its children are its operands. It is a condensed parse tree with no chain productions or punctuation.
2. **Directed Acyclic Graph (DAG):** like a syntax tree, but a common subexpression is represented by *one* shared node, which can have several parents.
3. **Postfix notation:** the operator comes after its operands, e.g. a + b * c → `a b c * +`. It can be evaluated with a stack.
4. **Three-address code (TAC):** a sequence of instructions x = y op z, with at most one operator on the right side. It is stored as **quadruples**, **triples** or **indirect triples**.
5. **Static Single Assignment (SSA):** TAC in which each name is assigned exactly once, using φ-functions where control flow joins.

Variants of syntax trees, for a + a * (b − c) + (b − c) * d



DAG: a and (b − c) are shared

Syntax tree vs DAG: the syntax tree would build two separate nodes for (b − c) and two leaves for a. The DAG detects that they are the same and reuses the node. So the DAG tells the code generator that b − c needs to be computed only **once**.

Building the DAG: before creating a node (op, l, r), the constructor *Node* checks whether an identical node already exists. If it does, the existing node is returned.

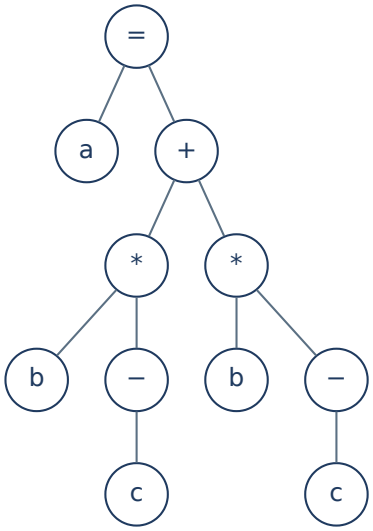
Value-number method

The nodes are stored in an array of records. Each node is referred to by its index, called its **value number**. A record is <op, left, right>. To find an existing node quickly, a hash table is keyed on the signature (op, l, r). For i = i + 10:

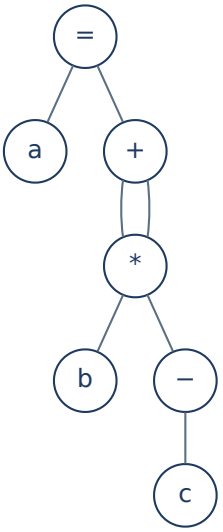
#	op	left	right
1	id	→ entry for i	
2	num	10	
3	+	1	2
4	=	1	3

TAC for the same expression: t1 = b − c; t2 = a * t1; t3 = a + t2; t4 = t1 * d; t5 = t3 + t4.
Postfix: a a b c − * + b c − d * +

One statement in every IR form — a = b * −c + b * −c



Syntax tree



DAG (b * −c is shared)

Form	Representation
Postfix	`a b c uminus * b c uminus * + assign`
TAC (from the syntax tree)	`t1 = minus c; t2 = b * t1; t3 = minus c; t4 = b * t3; t5 = t2 + t4; a = t5`
TAC (from the DAG)	`t1 = minus c; t2 = b * t1; t5 = t2 + t2; a = t5`

Quadruples				
#	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

Triples			
#	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Common three-address instruction forms

Form	Example
Assignment, binary op	x = y op z
Assignment, unary op	x = minus y, x = (float) y
Copy	x = y
Unconditional jump	goto L
Conditional jump	if x relop y goto L, ifFalse x goto L
Procedure call	param x_1 … param x_n; call p, n; return y
Indexed copy	x = y[i], x[i] = y
Address and pointer	x = &y, x = *y, *x = y

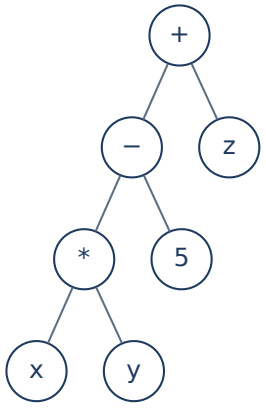
Choosing an IR

- **High-level IR** (syntax trees, DAGs): close to the source; good for type checking and source-level optimisation.
- **Low-level IR** (TAC, SSA): close to the machine; good for register allocation and instruction selection.
- Most compilers use several IR levels in sequence (e.g. AST → SSA-based IR → machine IR).

5.2 Construct the syntax tree for the expression x * y − 5 + z. 8 M

Dec-25 Q9b

* has higher precedence than − and +, and − and + are left-associative with equal precedence. So the expression groups as **((x * y) − 5) + z**.



Syntax tree for x*y−5+z

Construction steps (bottom-up, using Leaf and Node constructors):

```
p1 = new Leaf(id, entry-x)
p2 = new Leaf(id, entry-y)
p3 = new Node('*', p1, p2)
p4 = new Leaf(num, 5)
p5 = new Node('-', p3, p4)
p6 = new Leaf(id, entry-z)
p7 = new Node('+', p5, p6)   ← root
```

The SDD that builds it: E → E_1 + T { E.node = new Node('+', E_1.node, T.node) }; E → E_1 − T { E.node = new Node('−', E_1.node, T.node) }; T → T_1 * F { T.node = new Node('*', T_1.node, F.node) }; F → id { F.node = new Leaf(id, id.entry) }; F → num { F.node = new Leaf(num, num.val) }.

Corresponding TAC: t1 = x * y; t2 = t1 − 5; t3 = t2 + z. **Postfix:** x y * 5 − z +

Parse tree vs syntax tree

Using the grammar E → E + T | E − T | T, T → T * F | F, F → id | num, the **parse tree** for x*y−5+z has 20+ nodes, including chains such as E → T → F → id. The **syntax tree** keeps only the 3 operators (+, −, *) and the 4 operands (x, y, 5, z): 7 nodes in total. This compact form is what later phases use.

SDD with semantic actions — step-by-step construction during a bottom-up parse

Reduction	Action executed	Pointer
F → id (x)	new Leaf(id, x)	p1
F → id (y)	new Leaf(id, y)	p2
T → T * F	new Node('*', p1, p2)	p3
F → num (5)	new Leaf(num, 5)	p4
E → E − T	new Node('−', p3, p4)	p5
F → id (z)	new Leaf(id, z)	p6
E → E + T	new Node('+', p5, p6)	p7 (root)

Evaluating the postfix form with a stack (x=2, y=3, z=4)

Symbol	x	y	*	5	−	z	+
Stack	2	2 3	6	6 5	1	1 4	5

Result: 2·3 − 5 + 4 = 5 ✔.

DAG

There is no repeated sub-expression in x*y−5+z, so its DAG is identical to the syntax tree. For an expression like x*y − 5 + x*y, the DAG would share the x*y node.

Points to state: precedence and associativity decide the shape, interior nodes are operators, leaves are operands, and the tree is built bottom-up by Leaf/Node calls in the semantic actions.

5.3 Construct the quadruples, triples and indirect triples of the expression a * −(b + c). — 8 M

Dec-25 Q10b

Three-address code: t1 = b + c; t2 = minus t1; t3 = a * t2 (and a = t3 if the result is assigned)

Quadruples (op, arg1, arg2, result)

#	op	arg1	arg2	result
(0)	+	b	c	t1
(1)	minus	t1		t2
(2)	*	a	t2	t3

Triples (op, arg1, arg2)

#	op	arg1	arg2
(0)	+	b	c
(1)	minus	(0)	
(2)	*	a	(1)

Indirect triples

Statement	Pointer
(0)	(14)
(1)	(15)
(2)	(16)

#	op	arg1	arg2
(14)	+	b	c
(15)	minus	(14)	
(16)	*	a	(15)

Quadruples	Triples	Indirect triples
Temporaries are named explicitly, which needs symbol-table entries	No temporaries: a result is referred to by its position (i)	A separate list of pointers to the triples
Easy to move code during optimization	Moving a statement changes every reference to it	Reorder the pointer list only, so code is easy to move
Uses more space	Uses less space	Space in between

Generating the TAC — SDD trace

Node	Rule	Code generated
b + c	E.addr = new Temp()	t1 = b + c
−(b + c)	$E \to -E_1$	t2 = minus t1
a * …	$E \to E_1 * E_2$	t3 = a * t2

If the statement is an assignment: x = a * −(b + c)

#	op	arg1	arg2	result
(0)	+	b	c	t1
(1)	minus	t1		t2
(2)	*	a	t2	t3
(3)	=	t3		x

As a triple: (3) = x (2). Note that in triples an assignment uses the variable as arg1 and the position as arg2.

Why indirect triples are useful for optimisation

Suppose the optimiser moves the computation of b + c out of a loop. With **triples**, every reference "(0)" in later instructions would have to change. With **indirect triples**, only the order of the pointer list changes; the triples and their references stay the same. **Quadruples** also allow free reordering, because results are named by temporaries, but those temporaries need symbol-table space.

Space comparison (fields per instruction)

Representation	Fields	Temporaries in the symbol table?
Quadruples	4	Yes
Triples	3	No
Indirect triples	3 + pointer list	No

Remember: unary minus uses only arg1, so arg2 is left empty. Always write the TAC first, then the three tables, then a short comparison. This structure earns full marks.

5.4 Write the three-address code, quadruples, triples and indirect triples for: (1) a + b + (c + d) + d + e + a (2) a = (−c * b) + (−c * d) — 10 M

Apr-25 Q10b

(1) a + b + (c + d) + d + e + a (+ is left-associative)

```
t1 = a + b
t2 = c + d
t3 = t1 + t2
t4 = t3 + d
t5 = t4 + e
t6 = t5 + a
```

#	op	arg1	arg2	result
(0)	+	a	b	t1
(1)	+	c	d	t2
(2)	+	t1	t2	t3
(3)	+	t3	d	t4
(4)	+	t4	e	t5
(5)	+	t5	a	t6

Triples

#	op	arg1	arg2
(0)	+	a	b
(1)	+	c	d
(2)	+	(0)	(1)
(3)	+	(2)	d
(4)	+	(3)	e
(5)	+	(4)	a

Indirect triples

Stmt	Ptr
(0)	(20)
(1)	(21)
(2)	(22)
(3)	(23)
(4)	(24)
(5)	(25)

Triples (20)–(25) are the same as (0)–(5) above, with references renumbered: (22) = + (20) (21), and so on.

(2) a = (−c * b) + (−c * d)

```
t1 = minus c
t2 = t1 * b
t3 = minus c
t4 = t3 * d
t5 = t2 + t4
a  = t5
```

#	op	arg1	arg2	result
(0)	minus	c		t1
(1)	*	t1	b	t2
(2)	minus	c		t3
(3)	*	t3	d	t4
(4)	+	t2	t4	t5
(5)	=	t5		a

Triples

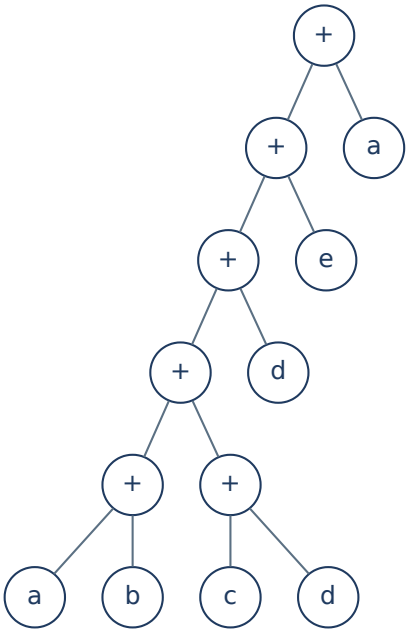
#	op	arg1	arg2
(0)	minus	c	
(1)	*	(0)	b
(2)	minus	c	
(3)	*	(2)	d
(4)	+	(1)	(3)
(5)	=	a	(4)

Indirect triples

Stmt	Ptr
(0)	(35)
(1)	(36)
(2)	(37)
(3)	(38)
(4)	(39)
(5)	(40)

Optimization note: minus c is a common subexpression, so t3 can be replaced by t1.

Syntax tree for expression (1)



Because + is left-associative, the tree leans to the left: ((((a+b)+(c+d))+d)+e)+a. Each interior node becomes one TAC instruction, giving 6 instructions in post-order.

Optimisation opportunities

- **Expression (1):** + is commutative and associative, so the expression can be rewritten as 2a + b + c + 2d + e. That needs fewer operations if a multiply by 2 is a cheap shift (t1 = a + a; ...). The compiler may do this only when overflow and floating-point rules allow it.
- **Expression (2):** "minus c" appears twice. Common subexpression elimination removes the second one: t1 = minus c; t2 = t1 * b; t4 = t1 * d; t5 = t2 + t4; a = t5. Algebraically, a = −c·(b + d), which needs only 3 operations: t1 = b + d; t2 = minus c; a = t2 * t1.

Quadruples vs triples — reminder

Aspect	Quadruple	Triple	Indirect triple
Result field	Explicit temp	Implicit (position)	Implicit
Moving code	Easy	Hard	Easy
Space	Most	Least	Medium

Exam format: for each expression write (a) the TAC, (b) the quadruple table, (c) the triple table, (d) the indirect triples (pointer list plus triples). This answer gives all four for both expressions, which earns the full 10–15 marks.

5.5 (i) Translate A[i] := b and a := B[i] into triple representation. (ii) Generate the three-address code, compute the basic blocks and draw the flow graph for: prod = 0; i = 1; do { prod = prod + a[i] * b[i]; i = i + 1; } while (i ≤ 10); 8 M

Aug-24 Q9c

(i) An indexed assignment needs **two** triples, because a triple has only two arguments:

A[i] := b			
#	**op**	**arg1**	**arg2**
(0)	[ ]=	A	i
(1)	assign	(0)	b

a := B[i]			
#	**op**	**arg1**	**arg2**
(0)	=[ ]	B	i
(1)	assign	a	(0)

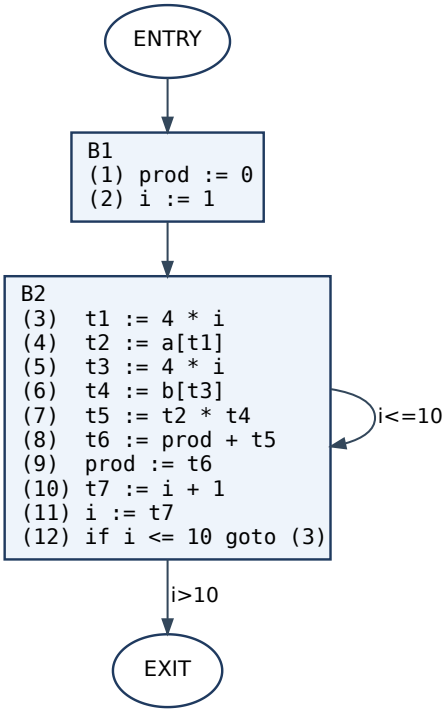
(ii) TAC (each array element is 4 bytes wide):

```
(1)  prod := 0
(2)  i := 1
(3)  t1 := 4 * i
(4)  t2 := a[t1]
(5)  t3 := 4 * i
(6)  t4 := b[t3]
(7)  t5 := t2 * t4
(8)  t6 := prod + t5
(9)  prod := t6
(10) t7 := i + 1
(11) i := t7
(12) if i <= 10 goto (3)
```

Leaders (rules: the first statement; the target of a jump; the statement after a jump):

- (1) — first statement
- (3) — target of goto in (12)

Basic blocks: B1 = (1)–(2), B2 = (3)–(12)



Leader algorithm (used to find the blocks)

1. The first three-address instruction is a leader.
2. Any instruction that is the target of a conditional or unconditional jump is a leader.
3. Any instruction that immediately follows a jump is a leader.

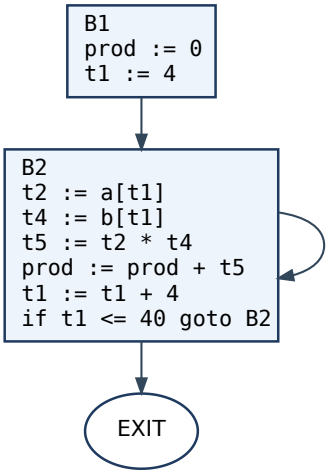
Each basic block consists of a leader and every instruction up to, but not including, the next leader.

Why 4 * i appears

Each array element is assumed to take 4 bytes. a[i] is at address base(a) + 4·i, so the TAC first computes the byte offset t1 = 4 * i and then reads a[t1]. In the source the loop starts at i = 1; with 0-based arrays the address would be base + 4·(i − 1), but the question uses the simple form.

Optimising the loop (B2)

1. **Common subexpression:** t3 = 4 * i is the same as t1, so b[t3] becomes b[t1].
2. **Copy propagation and dead code:** prod = t6 becomes prod = prod + t5, and i = t7 becomes i = i + 1.
3. **Strength reduction:** t1 = 4 * i becomes t1 = t1 + 4 (starting from t1 = 4).
4. **Induction-variable elimination:** i ≤ 10 is equivalent to t1 ≤ 40, so i is no longer needed.



Optimised flow graph: the loop body shrinks from 10 instructions to 6, and the multiplication by 4 is gone

Loop detection

The edge B2 → B2 is a **back edge**: its target dominates its source. So B2 forms a natural loop. The loop optimisations above apply to it.

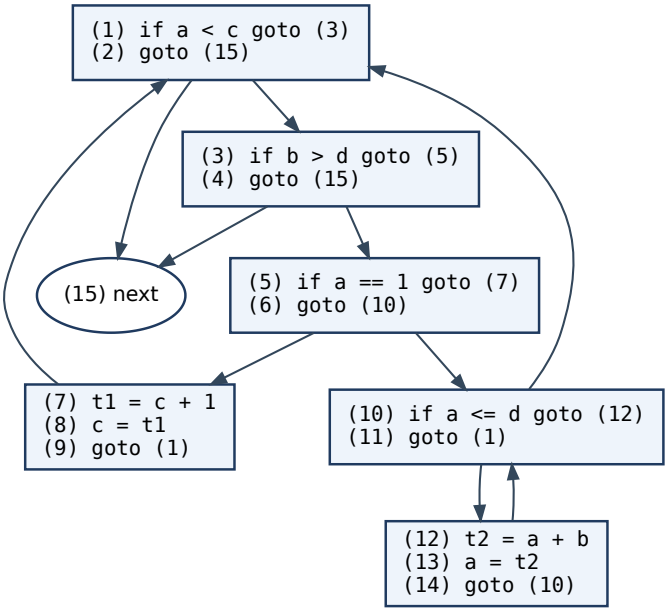
5.6 Generate three-address code for: while (a < c and b > d) do if a = 1 then c = c + 1 else while a <= d do a = a + b

8 M

Apr-25 Q9b

The Boolean **and** is translated with short-circuit (jumping) code. If *a < c* is false, the loop ends at once without testing *b > d*.

```
(1)  if a < c goto (3)
(2)  goto (15)
(3)  if b > d goto (5)
(4)  goto (15)
(5)  if a == 1 goto (7)
(6)  goto (10)
(7)  t1 = c + 1
(8)  c = t1
(9)  goto (1)
(10) if a <= d goto (12)
(11) goto (1)
(12) t2 = a + b
(13) a = t2
(14) goto (10)
(15) next ...
```



(1)–(4) test the outer condition; (5)–(6) the if; (7)–(9) the then-part, which jumps back to the outer loop test; (10)–(14) the inner while loop, whose exit (11) goes back to the outer loop test.

The translation rules used (SDD for control flow)

Construct	Code pattern

while (B) S_1	`begin: B.code (true → L1, false → S.next)`
	`L1: S1.code`
	`goto begin`

if (B) S_1 else S_2	`B.code (true → L1, false → L2)`
	`L1: S1.code; goto S.next`
	`L2: S2.code`

B_1 and B_2	`B1.true = newlabel (test B2 there); B1.false = B.false` `B2.true = B.true; B2.false = B.false`

Label-based version (easier to read)

```
L1:  if a < c goto L2
     goto Lnext
L2:  if b > d goto L3
     goto Lnext
L3:  if a == 1 goto L4
     goto L5
L4:  t1 = c + 1
     c = t1
     goto L1
L5:  if a <= d goto L6
     goto L1
L6:  t2 = a + b
     a = t2
     goto L5
Lnext: …
```

Basic blocks of the generated code

Leaders: (1), (2), (3), (4), (5), (6), (7), (10), (11), (12), (15). The flow graph above groups each test with its jump for readability. The inner loop is (10)–(14) with the back edge (14) → (10). The outer loop has back edges from (9) and (11) to (1).

Backpatching (alternative approach)

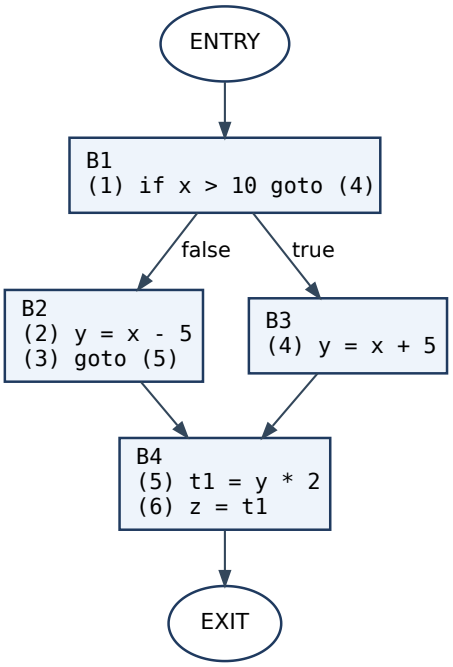
In one pass, jump targets are not known when a jump is generated. The compiler emits `goto _` and keeps lists (truelist, falselist, nextlist) of unfilled jumps. When the target address becomes known, these lists are **backpatched** with it.

5.7 Construct a basic-block flow graph for: if (x > 10) y = x + 5; else y = x − 5; z = y * 2; and identify the number of basic blocks. 7 M

Dec-25 Q9a

Three-address code

```
(1) if x > 10 goto (4)
(2) y = x - 5
(3) goto (5)
(4) y = x + 5
(5) t1 = y * 2
(6) z = t1
```



Finding leaders

- (1) is the first statement
- (2) follows a conditional jump
- (4) is the target of the jump in (1)
- (5) is the target of the goto in (3)

Basic blocks: B1 = {1}, B2 = {2, 3}, B3 = {4}, B4 = {5, 6}

Number of basic blocks = 4. Edges: B1→B2 (condition false), B1→B3 (condition true), B2→B4, B3→B4.

Alternative TAC using ifFalse (same 4 blocks)

```
(1) ifFalse x > 10 goto (4)
(2) y = x + 5
(3) goto (5)
(4) y = x - 5
(5) t1 = y * 2
(6) z = t1
```

Leader rules (with justification for this code)

Rule	Leader found
The first statement is a leader	(1)
The target of a jump is a leader	(4) from statement 1, (5) from statement 3
The statement after a jump is a leader	(2) after the conditional jump (1); (4) after the goto (3)

Properties of the flow graph

- **Entry block:** B1. **Exit block:** B4.
- B2 and B3 are the two *branches*; B4 is the *join* (merge) point, with predecessors B2 and B3.
- There are no back edges, so the graph has **no loops**.
- **Dominators:** B1 dominates every block. B4 is dominated by B1 only, because it can be reached through either B2 or B3.

Use in optimisation

Global data-flow analysis on this graph shows that y is defined in both B2 and B3. Both definitions reach B4 (reaching definitions). So y cannot be replaced by a constant in B4. The common term x is used in both branches and is available on entry to each.

Answer: **4 basic blocks** (B1: condition, B2: else-part + jump, B3: then-part, B4: z = y * 2), connected by the edges B1→B2, B1→B3, B2→B4 and B3→B4.

5.8 Define a flow graph. Differentiate between a basic block and a flow graph. Explain the structure-preserving transformations for optimizing a basic block.

4–5 M

Dec-25 Q10c Aug-24 Q9a

Basic block: a maximal sequence of consecutive three-address statements that control enters only at the first statement and leaves only at the last, with no halt or branch in between. **Flow graph:** a directed graph whose nodes are the basic blocks. There is an edge B1 → B2 if B2 can immediately follow B1 during execution: either B1 ends in a jump to B2's leader, or B2 comes right after B1 and B1 does not end in an unconditional jump.

Basic block	Flow graph
A straight-line sequence of instructions	A graph made of basic blocks connected by control-flow edges
Single entry, single exit; no internal branches	Contains branches, loops and join points
Found using the *leader* algorithm	Built by adding edges between the blocks
Unit of **local** optimization (DAG, CSE)	Used for **global** optimization (data-flow and loop analysis)

Structure-preserving transformations (local, within a block)

1. **Common subexpression elimination:** a = b + c; b = a − d; c = b + c; d = a − d ⟹ the second a − d is replaced: d = b.
2. **Dead-code elimination:** remove x = y + z if x is never used afterwards (x is dead).
3. **Renaming temporary variables:** t = b + c can become u = b + c (with all uses of t changed). This gives a normal-form block in which each temporary is defined once.
4. **Interchange of statements:** t1 = b + c; t2 = x + y can be swapped if neither uses the other's result. This helps scheduling.

Partitioning into basic blocks — algorithm

```
Input: a sequence of three-address instructions
1. Determine the leaders (first instruction, jump targets, instructions after jumps)
2. For each leader, its block = the leader plus all instructions up to (not including) the
next leader
Flow-graph edges: B → C if C's leader follows B's last instruction and B does not end in
goto,
                  or if B's last instruction jumps to C's leader
```

Algebraic transformations (also local)

- Removing identities: x = x + 0, x = x * 1 → removed.
- Strength reduction: x^2 → x * x, 2 * x → x + x.
- Constant folding: 2 * 3.14 → 6.28.
- Commutativity: x * y and y * x are recognised as the same expression in a DAG.

Example of all four structure-preserving transformations on one block

Before	After	Transformation

`t1 = b + c`
`t2 = t1 * d`
`t3 = b + c`
`t4 = t3 * e`
`u = t2 (u never used)`
	`t1 = b + c`
	`t2 = t1 * d`
	
	`t4 = t1 * e`
		CSE removes t3; dead-code elimination removes u = t2

Renaming temporaries (t4 → t3) and interchanging independent statements (computing t4 before t2) are also allowed, because they do not change the result.

Uses of the flow graph

Detecting loops (back edges), data-flow analysis (reaching definitions, live variables, available expressions), global register allocation, and finding unreachable blocks.

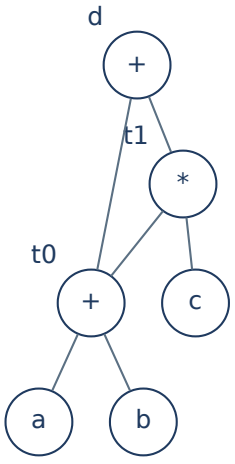
5.9 What is a Directed Acyclic Graph (DAG)? Draw the DAG for t0 = a + b; t1 = t0 * c; d = t0 + t1. 7 M

Aug-24 Q9b

A **DAG** for a basic block is a directed graph with no cycles in which:

- **leaves** are labelled by identifiers or constants (the initial values of names, e.g. a_0);
- **interior nodes** are labelled by operators;
- each node can also carry a list of **identifiers** (attached names) that hold its value.

Uses: finding common subexpressions automatically, finding which names are used inside the block but defined outside, finding which statements compute values used outside the block, and generating optimized code.



DAG: node (+ a b) is shared by t1 and d

Construction:

1. t0 = a + b → create leaves a, b and node + (label t0).
2. t1 = t0 * c → leaf c; node * (t0, c), label t1.
3. d = t0 + t1 → node + (t0-node, t1-node), label d.

The t0 node has two parents. This shows that a + b is computed once and reused.

DAG construction algorithm (for a basic block)

```
for each statement  x = y op z :
    1. if node(y) is undefined, create a leaf y₀;  same for z
    2. look for an existing node n labelled op with children node(y), node(z)
       if none exists, create one
    3. remove x from the label list of its old node; attach x to n;  set node(x) = n
for x = y :   attach x to node(y)
```

Code generated from the DAG

```
t0 = a + b
t1 = t0 * c
d  = t0 + t1
```

If t0 and t1 are not live after the block, the code can be written as R1 = a + b; R2 = R1 * c; d = R1 + R2, keeping the values in registers.

Applications of DAGs

- Automatic detection of **common subexpressions**.
- Finding which names are used inside the block but evaluated outside it (the leaves).
- Finding which statements compute values that can be used outside the block (the labelled nodes).
- Simplifying the list of quadruples by removing redundant ones, and choosing a good evaluation order to reduce register usage.

Syntax tree vs DAG

Syntax tree	DAG
Each node has one parent	A node can have several parents (shared)
Repeated sub-expressions are duplicated	Repeated sub-expressions are represented once
Built for whole expressions	Built for expressions or whole basic blocks

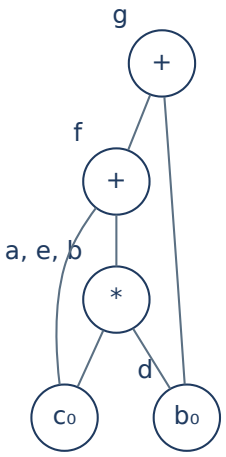
5.10 Construct the DAG for the block: a := b*c; d := b; e := d*c; b := e; f := b + c; g := f + d

10 M

Apr-25 Q10a

Algorithm: for each statement x := y op z: find or create nodes for y and z (by their current values), look for an existing node (op, y, z) and create one only if none exists, then attach the label x to it (removing x from any node it labelled before). For x := y, just attach x to y's node.

Statement	Action on the DAG
a := b * c	Create leaves b_0, c_0 and node n1 = (* b_0 c_0); label **a**
d := b	Copy: attach label **d** to leaf b_0
e := d * c	d is the node b_0, so this is (* b_0 c_0), which is n1 — **common subexpression**. Attach **e** to n1
b := e	Attach **b** to n1. b now means n1; leaf b_0 keeps the old value
f := b + c	Node n2 = (+ n1 c_0); label **f**
g := f + d	Node n3 = (+ n2 b_0); label **g**



Final DAG

Optimized code from the DAG (assuming all names are live on exit):

```
a := b * c
d := b
f := a + c
g := f + d
e := a
b := a
```

The second multiplication `d * c` is removed (common subexpression). If e is not live after the block, `e := a` is also removed (dead code). Note that `d := b` must come before `b := a`.

Points to note in this DAG

- **Common subexpression:** e := d * c is detected as equal to a := b * c, because d holds the original value of b (d := b). So no new node is created.
- **Killing a value:** after b := e, the name b refers to node n1. The leaf b_0 still represents the *old* value of b, which d still holds. So g := f + d uses b_0, not n1.
- The node n1 carries three labels (a, e, b): all three names hold the same value at the end of the block.

Reconstructing code — ordering rules

1. Evaluate each interior node only after its children.
2. Assign the value to **one** live name first; copy it to the other live names afterwards.
3. Do not overwrite a name whose old value is still needed: d := b must be emitted before b := a.
4. Labels that are not live on exit (e.g. temporaries) need no assignment.

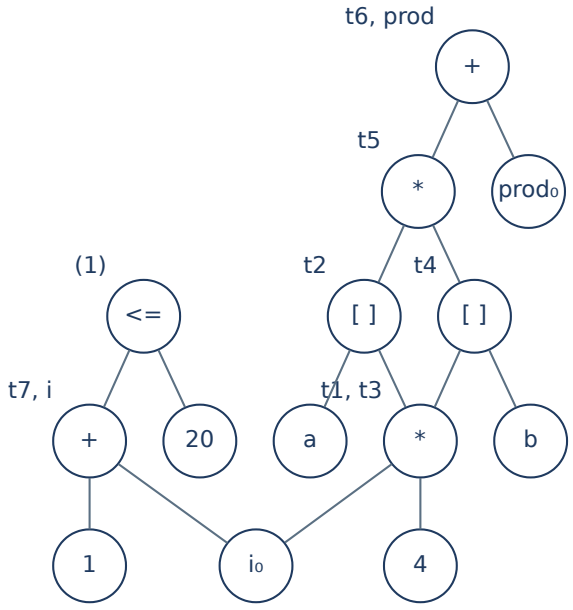
Result: operations saved

	Original	From DAG
Multiplications	2	1
Additions	2	2
Copies	2	3 (but copies are cheap and often disappear in register allocation)

If only g is live on exit, the block reduces to: t = b * c; f = t + c; g = f + b. Only 3 operations are needed.

5.11 Construct the DAG for the basic block: t1 := 4*i; t2 := a[t1]; t3 := 4*i; t4 := b[t3]; t5 := t2*t4; t6 := prod + t5; prod := t6; t7 := i + 1; i := t7; if i <= 20 goto (1) — 6 M

Aug-24 Q10b



DAG for the basic block

- t1 := 4*i creates node (* 4 i_0), label t1.
- t3 := 4*i finds the **same** node, so it gets labels t1, t3.
- t2 := a[t1] and t4 := b[t3] become two [ ] nodes sharing the (* 4 i_0) child.
- t5 = (* t2 t4); t6 = (+ $prod_0$ t5); prod := t6 attaches prod to it.
- t7 = (+ i_0 1); i := t7 attaches i.
- (<= t7-node 20) is the conditional jump back to (1).

Optimized block:

```
t1 := 4 * i
t2 := a[t1]
t4 := b[t1]
t5 := t2 * t4
prod := prod + t5
i := i + 1
if i <= 20 goto (1)
```

Statement-by-statement DAG construction

Statement	Effect on the DAG
1) t1 := 4*i	leaves 4, i_0; node n1 = (* 4 i_0) labelled t1
2) t2 := a[t1]	leaf a; node n2 = ([ ] a n1) labelled t2
3) t3 := 4*i	(* 4 i_0) already exists as n1 → add label t3 (CSE)
4) t4 := b[t3]	leaf b; node n3 = ([ ] b n1) labelled t4
5) t5 := t2*t4	n4 = (* n2 n3) labelled t5
6) t6 := prod+t5	leaf $prod_0$; n5 = (+ $prod_0$ n4) labelled t6
7) prod := t6	add label prod to n5
8) t7 := i+1	leaf 1; n6 = (+ i_0 1) labelled t7
9) i := t7	add label i to n6
10) if i <= 20 goto (1)	leaf 20; n7 = (<= n6 20); its target is statement (1)

Further loop optimisation (beyond the DAG)

Across iterations, t1 = 4 * i is an induction variable. Strength reduction replaces it with t1 = t1 + 4, and the test i ≤ 20 becomes t1 ≤ 80. The DAG optimises *within* one iteration; loop optimisation works *across* iterations.

Observations to write: (1) 4*i is computed once (t1 and t3 share a node); (2) prod and i are both updated in place; (3) the leaves i_0 and $prod_0$ are values coming from outside the block; (4) the optimised block has 7 instructions instead of 10.

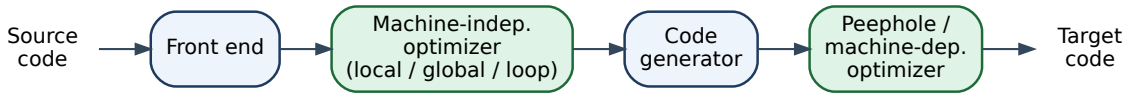
5.12 Explain the principal sources of optimization. 5 M

Dec-25 Q9c

A transformation must **preserve the meaning** of the program, make it measurably faster or smaller, and be worth the effort. The main function-preserving transformations are:

1. **Common subexpression elimination:** if E was computed before and its operands have not changed, reuse the value. `t1 = 4*i; … t3 = 4*i` ⟹ `t3 = t1`.
2. **Copy propagation:** after x = y, use y in place of x. This often makes x dead.
3. **Dead-code elimination:** remove statements whose results are never used, or code that can never be reached (`if (debug)` with debug = false).
4. **Constant folding and propagation:** evaluate constant expressions at compile time: `x = 2 * 3.14` ⟹ `x = 6.28`.
5. **Loop optimizations** (most programs spend most of their time in inner loops):
   - *Code motion:* move loop-invariant computations out of the loop: `while (i <= limit-2)` ⟹ `t = limit-2; while (i <= t)`.
   - *Induction-variable elimination:* when i and t = 4*i change in step, keep only one of them.
   - *Strength reduction:* replace an expensive operation with a cheaper one: `t = 4*i` inside a loop ⟹ `t = t + 4`.

Where optimisation happens



Levels and scope of optimisation

Level	Scope	Examples
Local	Within a basic block	CSE with DAGs, constant folding
Global	Across blocks in a function (flow graph)	Global CSE, copy propagation, dead code
Loop	Inside loops	Code motion, strength reduction, induction variables
Peephole	A small window of target instructions	Redundant loads and stores, jump chains, algebraic simplification
Interprocedural	Across functions	Inlining, constant propagation through calls

Worked example — before and after (quicksort fragment style)

Before	After

`i = 0`
`L: t1 = 4 * i`
`t2 = a[t1]`
`t3 = 4 * i`
`t4 = limit - 2`
`x = t2 + t4`
`t5 = 4 * i`
`b[t5] = x`
`i = i + 1`
`if i < n goto L`
	`t4 = limit - 2 ← code motion`
	`t1 = 0`
	`L: t2 = a[t1] ← CSE`
	`x = t2 + t4`
	`b[t1] = x`
	`t1 = t1 + 4 ← strength reduction`
	`if t1 < 4*n goto L ← induction-variable elimination`
	`(4*n computed once, before L)`

Peephole optimisation (machine-dependent)

- Redundant load/store: `MOV R0, a; MOV a, R0` → delete the second instruction.
- Jumps to jumps: `goto L1; … L1: goto L2` → `goto L2`.
- Unreachable code after an unconditional jump.
- Machine idioms: `x = x + 1` → `INC x`.

Criteria for a transformation: it must preserve meaning, it must speed up programs by a measurable amount on average, and it must be worth the effort.

5.13 Discuss copy propagation, dead-code elimination and strength reduction in detail.

4–7 M

Apr-25 Q9c Aug-24 Q10c

Copy propagation

A **copy statement** has the form f = g. Copy propagation replaces later uses of f with g, as long as neither f nor g is reassigned in between. It does not save time by itself, but it creates dead code.

Original	After copy propagation	After dead-code elimination (x unused)

`x = a;`
`y = x * 2;`
`z = x + y;`
	`x = a;`
	`y = a * 2;`
	`z = a + y;`
		`y = a * 2;`
		`z = a + y;`

Dead-code elimination

A variable is **live** at a point if its value may be used later; otherwise it is **dead**. Statements that compute values that are never used, and code that cannot be reached, can be removed safely. This makes the program smaller and faster.

```
debug = false;
if (debug) printf("value=%d", x);   ⇒  the whole if is removed (always false)
t = a + b;  /* t never used */       ⇒  removed
return x;  x = x + 1;               ⇒  x = x + 1 is unreachable, removed
```

Dead code often comes from earlier optimizations (copy propagation, constant folding) or from macros and debug code. It is found using liveness (data-flow) analysis and by checking which

blocks are reachable in the flow graph.

Strength reduction

Replace an **expensive** operation with an equivalent **cheaper** one, especially for induction variables inside loops:

Before	After
x * 2	x + x or x << 1
x ** 2	x * x
x / 8 (unsigned)	x >> 3

`for (i = 1; i <= n; i++)`
`  t = 4 * i;  /* multiply each time */`
	`t = 0;`
	`for (i = 1; i <= n; i++)`
	`  t = t + 4;  /* add only */`

Since t = 4·i increases by 4 each time i increases by 1, the multiplication is replaced by an addition. If i is then used only to control the loop, induction-variable elimination can remove i as well.

Liveness — the basis of dead-code elimination

A variable x is **live** at point p if there is a path from p to a use of x along which x is not redefined. Liveness is computed by backward data-flow analysis on the flow graph:

```
OUT[B] = ∪ IN[S]  over successors S of B
IN[B]  = use[B] ∪ (OUT[B] − def[B])
```

A statement x = … is dead if x is not live immediately after it.

Copy propagation followed by dead-code elimination — a longer example

Original	After copy propagation	After dead-code elimination

`t = i`
`a[t] = x`
`y = t + 1`
`t = y`
`return y`
	`t = i`
	`a[i] = x`
	`y = i + 1`
	`t = y`
	`return y`
		`a[i] = x`
		`y = i + 1`
		`return y`

Strength reduction — table of common replacements

Expensive	Cheaper	Condition
$x * 2^n$	x << n	integer x
$x / 2^n$	x >> n	unsigned x
$x \% 2^n$	$x \,\&\, (2^n - 1)$	unsigned x
pow(x, 2)	x * x	—
i * c in a loop	t = t + c	i is an induction variable

Strength reduction on a loop — full example

```
Before:                          After:
for (i = 0; i < n; i++)          t = 0;
    a[i] = i * 7;                for (i = 0; i < n; i++) {
                                     a[i] = t;
                                     t = t + 7;
                                 }
```

Benefits: multiplication is usually slower than addition (and division slower still), so replacing it inside a loop that runs n times saves n multiplications. These optimisations are machine-independent and are performed on the IR.

5.14 Explain storage organization in detail. 8 M

Dec-25 Q10a

At run time the target program executes in its own logical address space. The compiler, OS and target machine together decide how that space is divided. A typical layout is:

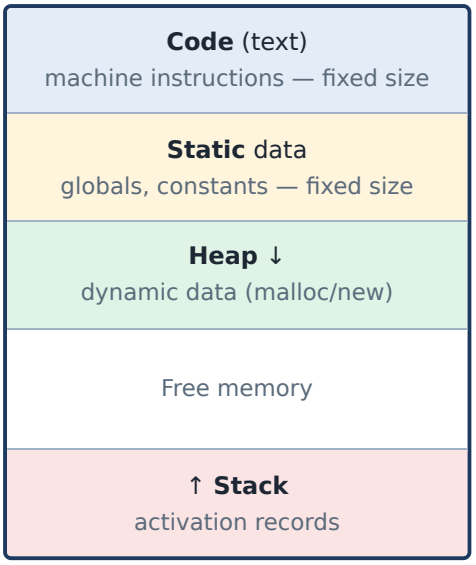


Fig. Run-time memory subdivision

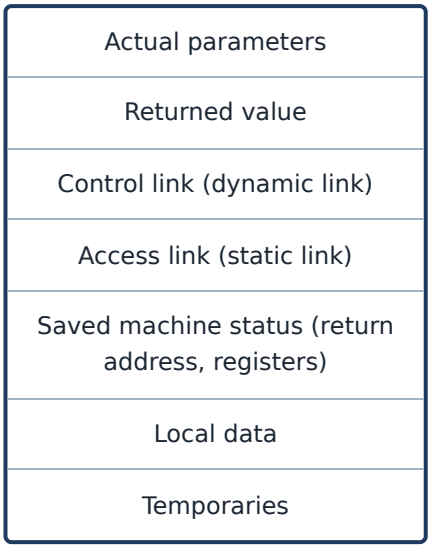


Fig. A general activation record

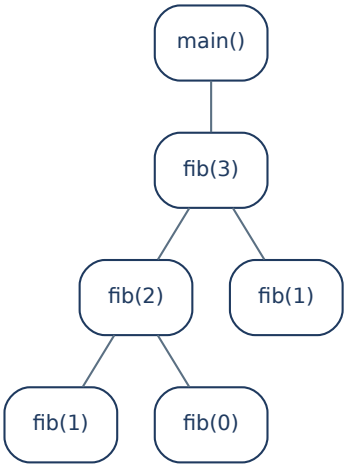
- **Code area:** the size of the generated code is fixed at compile time, so it is placed in a statically determined area, usually at the low end of memory.
- **Static area:** global variables, static locals and constants. Their size is known at compile time, and their addresses can be compiled into the code.
- **Stack:** holds the **activation records** of procedure calls. A record is pushed when a procedure is called and popped when it returns. Local variables live only as long as the call.
- **Heap:** data whose lifetime is not tied to calls, such as objects created with malloc/new and linked lists. It is freed explicitly (free/delete) or by garbage collection. The stack and the heap grow towards each other.

Storage-allocation strategies

Strategy	How it works	Limitations
Static allocation	Names are bound to storage at compile time; the binding never changes.	No recursion; data structure sizes must be known at compile time; no dynamic data.
Stack allocation	An activation record is pushed on each call and popped on return. Locals are addressed relative to the frame pointer.	Local values are lost when the call ends (no dangling locals).
Heap allocation	Blocks are allocated and freed in any order (first-fit, best-fit; garbage collection).	Slower; fragmentation; memory leaks.

Activation record fields: *actual parameters* passed by the caller; *returned value*; *control link*, which points to the caller's record; *access link*, which points to the enclosing scope's record so non-local data can be reached; *saved machine status*, i.e. the return address and registers; *local data*; and *temporaries* used while evaluating expressions. The **calling sequence** allocates the record and fills these fields. The **return sequence** restores the machine state and deallocates the record.

Activation tree and control stack



Activation tree for main() calling fib(3)

Each node is one activation (call) of a procedure. The children of a node are the calls it makes, in order. At any moment, the **control stack** holds the activation records of the calls on the path from the root to the active call. When fib(1) under fib(2) is running, the stack is main → fib(3) → fib(2) → fib(1).

Calling and return sequences

Calling sequence	Return sequence
1. The caller evaluates the actual parameters and stores them. 2. The caller stores the return address and the old frame pointer, then jumps to the callee. 3. The callee saves registers and initialises its local data.	1. The callee places the return value. 2. It restores the registers and the old frame pointer. 3. It jumps to the return address; the caller then reads the return value.

Heap management

- **Allocation strategies:** first-fit, best-fit and next-fit; a free list of available blocks; coalescing of adjacent free blocks to reduce fragmentation.
- **Deallocation:** manual (free/delete), which risks dangling pointers and leaks, or automatic **garbage collection**: reference counting, mark-and-sweep, copying collectors.

Access to non-local data

In languages with nested procedures (Pascal), the **access link** points to the activation record of the lexically enclosing procedure. A **display** (an array of pointers, one per nesting level) gives faster access. In C, non-local data is only global (static), so no access link is needed.

Parameter passing (often asked together)

Call by value (a copy is passed), call by reference (an address is passed), call by value-result (copy in, copy out), and call by name (textual substitution, as in Algol).

> **Answer plan for 10–15 marks:** the memory-layout diagram, the four areas, the activation record with all its fields, the three allocation strategies with their limitations, and the calling and return sequences.